LOW-LEVEL DESIGN · INTERVIEW HANDBOOK

LLD Notes Foundations to Interview Playbook

OOP, UML, SOLID, 22 design patterns, 11 full case studies and a step-by-step method for cracking any LLD round.

Built from the Coder Army LLD playlist
(Notion notes + NotebookLM + lecture transcripts + whiteboard UML)

Code & repo → **https://github.com/adityatandon15/Low-Level-Design-Course**

Contents

0 How to use this book 4

PART 1 · FOUNDATIONS

1 LLD, DSA and HLD 7
2 Object-Oriented Programming 8
3 UML: drawing a design 10
4 SOLID and design principles 12
5 Worked example: Document Editor (bad → good → trade-offs) 15

PART 2 · DESIGN PATTERNS

6 The pattern mindset, and Strategy 18
- Strategy 18

7 Creational patterns 20
- Singleton 20
- Factory family 21
- Builder 23
- Prototype 24

8 Structural patterns 25
- Adapter 25
- Facade 25
- Decorator 26
- Proxy 28
- Composite 30
- Bridge 31
- Flyweight 32

9 Behavioral patterns 34
- Observer 34
- Command 35
- State 36
- Template Method 38
- Chain of Responsibility 39
- Iterator 40
- Mediator 41
- Memento 42
- Visitor 43

10 Look-alike patterns: choosing between them 46
11 Null Object and anti-patterns 47
- Null Object 47

PART 3 · CASE STUDIES

12 Reading the case studies 49
13 Zomato: food delivery (user flow) 50
14 Notification system 52
15 Spotify: music player 54
16 Payment gateway 56

17 Discount / coupon system 58
18 Zepto: quick commerce 60
19 Tinder: dating app 62
20 Splitwise: expense sharing 64
21 Tic-Tac-Toe 66
22 Snake & Ladder 68
23 Chess (online, multi-user) 70

PART 4 · INTERVIEW PLAYBOOK

24 The 9-step approach to any LLD problem 74
25 Recognising the pattern from the problem statement 77
26 Decision cheat sheet 79
27 Trade-offs, one-liners and the final checklist 80

0 How to use this book

These notes are built to do two jobs: **teach** every concept from the ground up, and **revise** fast the night before an interview. Everything from the playlist is here, but each idea is explained once and then referred to, not repeated.

The 3-pass revision plan

Time you have	What to read
15 minutes	Part 4 only: the *9-step approach*, the *pattern recognition table* and the *trade-off one-liners*.
1 hour	Part 4, plus the grey *"Recognise it when"* strip and red *"Interview trap"* boxes of every pattern in Part 2.
Full prep	Everything in order. Re-draw one case study from Part 3 on paper without looking.

Reading the boxes

> **REMEMBER**
> The core idea or definition worth memorising verbatim.

> **INTERVIEW TRAP**
> What interviewers probe, common wrong answers, and the follow-up question they will ask.

> **SAY THIS**
> A ready-made sentence or justification to use out loud.

> **REAL-WORLD USE**
> Where the idea shows up in production systems (great for "give me an example").

Recognise it when… the trigger phrases in a problem statement that point to a pattern.

Conventions

- **Code is C++-flavoured pseudo-code** (as in the course): `virtual … = 0` is a pure virtual method, `IName` is an interface, `Name*` is a pointer/reference. Everything maps 1:1 to Java/C#/Python.
- **Diagrams** are of two kinds: your hand-drawn whiteboard UML from the lectures (dark background) and clean, light diagrams added to fill gaps (Singleton, Builder, Flyweight, Prototype, Memento, Chess, the approach flowchart).
- Pattern names are tagged CREATIONAL STRUCTURAL BEHAVIORAL.

Playlist video → chapter map

Video numbers are the position in the Coder Army LLD playlist (1–40). Every pattern header and case-study title also carries its video tag.

#	Video topic	Chapter	#	Video topic	Chapter
1	Introduction to System Design	1 · LLD vs DSA/HLD	**21**	Proxy (virtual / protection / remote)	8 · Proxy
2	OOP pillars: abstraction, encapsulation	2 · OOP	**22**	Chain of Responsibility (ATM)	9 · CoR
3	Inheritance & polymorphism	2 · OOP	**23**	Payment gateway	16 · Case study
4	UML: class & sequence diagrams	3 · UML	**24**	Discount coupon engine	17 · Case study
5	SOLID part 1	4 · SOLID	**25**	Bridge	8 · Bridge
6	SOLID part 2	4 · SOLID	**26**	Zepto	18 · Case study
7	Document Editor (Google Docs)	5 · Worked example	**27**	Tinder	19 · Case study
8	Strategy	6 · Strategy	**28**	Builder (classic + step)	7 · Builder
9	Factory (Simple / Method / Abstract)	7 · Factory	**29**	Iterator	9 · Iterator
10	Singleton	7 · Singleton	**30**	Flyweight	8 · Flyweight
11	Zomato	13 · Case study	**31**	Splitwise	20 · Case study
12	Observer	9 · Observer	**32**	State (vending machine)	9 · State
13	Decorator	8 · Decorator	**33**	Tic-Tac-Toe	21 · Case study
14	Notification engine	14 · Case study	**34**	Snake & Ladder	22 · Case study
15	Command	9 · Command	**35**	Mediator (chat room)	9 · Mediator
16	Adapter	8 · Adapter	**36**	Prototype	7 · Prototype
17	Facade	8 · Facade	**37**	Chess	23 · Case study
18	Spotify	15 · Case study	**38**	Visitor	9 · Visitor
19	Composite (file system)	8 · Composite	**39**	Memento	9 · Memento
20	Template Method	9 · Template Method	**40**	Null Object & anti-patterns	11 · Null Object

The whole book in one picture

Part	What you learn	Why it matters in the interview
1 · Foundations	LLD vs DSA/HLD, OOP, UML, SOLID, Law of Demeter	The vocabulary and the principles you justify every decision with.
2 · Patterns	22 patterns + anti-patterns, with look-alike comparisons	Pattern choice is how you show design maturity.
3 · Case studies	11 solved LLD problems end to end	Reusable templates: most new problems are a remix of these.
4 · Playbook	9-step method, recognition tables, checklists	A repeatable script for any problem you have never seen.

PART 1

Part 1 · Foundations

What LLD actually is, the OOP toolkit, how to draw designs in UML, and the SOLID principles that every design decision is judged against.

1 LLD, DSA and HLD

▶ Video 1 of the playlist

Three different questions get asked about the same product. Know which one you are being asked.

DSA — the brain

Solves an *isolated computational* problem: Dijkstra for shortest path, a min-heap to pop the closest rider.

LLD — the skeleton

Defines **code structure**: classes, objects, relationships, interfaces, patterns. Done *before* algorithms are plugged in.

HLD — the system

Macro decisions, almost no code: tech stack, SQL vs NoSQL, scaling servers, cost (AWS bill), load balancers.

The three pillars of a good LLD

Pillar	Meaning
Scalability	Structure survives huge growth in users, features and data without being rewritten.
Maintainability	Easy to debug; **adding a feature must not break existing features**.
Reusability	Plug-and-play, *not tightly coupled* to one app. A rider-mapping algorithm should drop into Zomato, Amazon or Blinkit unchanged.

INTERVIEW TRAP: THE "QUICK RIDE" CASE

Asked to design a ride-hailing app, most candidates jump straight to "map the city as a graph, run Dijkstra". That ignores real constraints: masking phone numbers between rider and user, payments, notifications.

Correct order: domain model first (User, Rider, Location, Notification, Payment) → relationships → security/scalability → *only then* the DSA that powers routing.

SAY THIS

"I'll first fix the entities and their responsibilities, then plug the routing algorithm in behind an interface so it stays swappable."

2 Object-Oriented Programming

▶ Video 2, 3 of the playlist

Why OOP exists

- **Machine language** (0s and 1s): talks directly to the CPU, error-prone, does not scale.
- **Assembly** (mnemonics like `MOV`): still welded to the hardware.
- **Procedural (C)**: functions, loops, conditionals: a "recipe book" of steps. Breaks down at enterprise scale because it cannot model the real world and gives no data security.
- **OOP**: solve problems by modelling code *exactly like the real world*.

> **BUILDING BLOCKS**
> **Object** = real-world thing with **characteristics** (state / variables: brand, model) and **behaviours** (methods: startEngine, accelerate). **Class** = the blueprint used to create objects.

Pillar 1 · Abstraction (manage complexity)

Hide unnecessary internals, show only what the user needs. To drive a car you use the steering wheel and pedals; you do not need to know how combustion works. In code it is delivered by **abstract classes / interfaces / virtual functions**. Even `if` and `for` are abstractions over binary machine code.

Pillar 2 · Encapsulation (protect integrity)

Two ideas in one: **bundle** data and the methods that operate on it into one class, and **restrict** direct access to sensitive data. Nobody can set a car's speed to 500 km/h or roll back the odometer; they must go through behaviours (press the accelerator).

```
class Car {
  private: int speed;                       // hidden
  public:
    int  getSpeed()         { return speed; }
    void setSpeed(int s)    { if (s >= 0 && s <= 220) speed = s; }  // validation lives here
};
```

Modifier	UML	Visible to
`public`	`+`	Everyone
`private`	`-`	Only the class itself
`protected`	`#`	The class and its child classes (not outsiders)

> **ABSTRACTION VS ENCAPSULATION: THE ONE-LINE DIFFERENCE**
> **Abstraction = complexity management** (hiding the engine only avoids confusing the driver; seeing it would not break the car). **Encapsulation = integrity protection** (exposing the odometer *would* break the system). Getters/setters are the mechanism; setters matter because they let you **validate**.

Pillar 3 · Inheritance (hierarchy + reuse)

Child classes automatically get the parent's state and behaviour. The parent holds universal logic (`Car::startEngine()`), each child adds only its specialities (`ElectricCar::chargeBattery()`,

`ManualCar::shiftGear()`).

- **Interview keyword:** `protected`. Make parent fields `protected`, not `private`, so children can use them while outsiders cannot.
- Models an **Is-A** relationship (ElectricCar *is a* Car).

> **INTERVIEW TRAP: PUBLIC VS PRIVATE INHERITANCE**
> In LLD we use **public inheritance 99% of the time**. With *private* inheritance every public/protected parent member becomes `private` in the child, which **breaks the chain downstream**: if `ManualCar` inherits `Car` privately, then `WagonR : ManualCar` cannot reach `Car`'s methods at all.

Pillar 4 · Polymorphism (one stimulus, many forms)

	Static (compile-time) = Overloading	Dynamic (run-time) = Overriding
Idea	The *same object* reacts differently because the *situation (arguments)* changed.	*Different objects of one family* react differently to the *same call*.
Rule	Same name, **different number/type of parameters**.	**Identical full signature** (name, return type, params) in parent and child; parent method is `virtual`.
Example	`accelerate()` vs `accelerate(int speed)`	`Car::accelerate()` is virtual; `ManualCar` adds +20, `ElectricCar` adds +15.

> **WHY DYNAMIC POLYMORPHISM IS THE ENGINE OF LLD**
> Client code holds a `Car*`, calls `accelerate()`, and the right child runs. Almost every design pattern in Part 2 is just *"hold an interface, call it, let polymorphism pick the implementation"*.

3 UML: drawing a design

▶ **Video 4 of the playlist**

UML has 14 diagram types, but LLD interviews use two almost exclusively.

Structural (static)
The blueprint: components and how they relate. **Class diagram**.

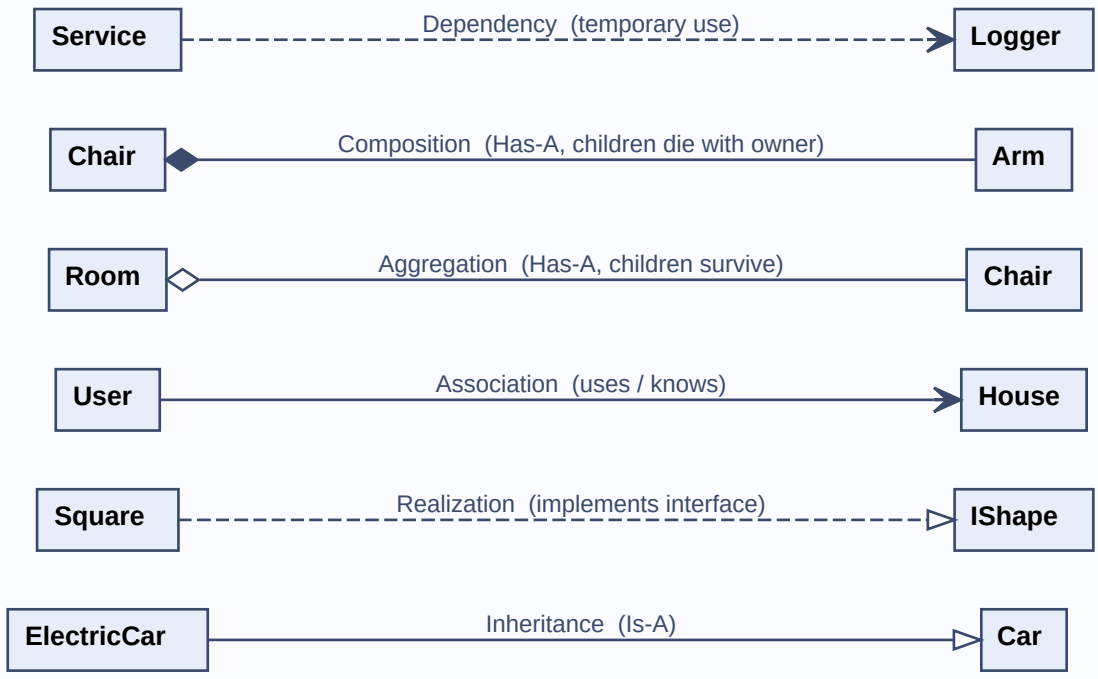
Behavioral (dynamic)
Objects interacting over time. **Sequence diagram** (and the State machine diagram, see the State pattern).

Class diagrams

A class is a rectangle in three tiers: **Name** → **Variables (state)** → **Methods (behaviour)**. Visibility prefixes: `+` public, `–` private, `#` protected. Abstract classes/interfaces carry `<<abstract>>` / `<<interface>>` above the name.

Relationships: the most-asked UML topic

Interviewers care how you **connect** classes. There are two families: **Is-A** (inheritance) and **Has-A** (the object holds a reference/pointer to another).



The six relationship notations. Memorise the arrowheads: hollow triangle = Is-A, diamond = Has-A (hollow = weak, filled = strong).

Relationship	Strength	Meaning and test question
Inheritance	Is-A	ElectricCar *is a* Car. Hollow closed arrow → parent.
Association	Weakest Has-A	Objects know and use each other, no ownership. "User lives in a House".
Aggregation	Container, **independent**	Room contains Chair and Bed. Destroy the room, **the chair still exists**. Hollow diamond on the container.
Composition	Strongest, **dependent**	Chair is made of Arms and Wheels. Destroy the chair and **those parts cease to exist**. Filled diamond on the whole.

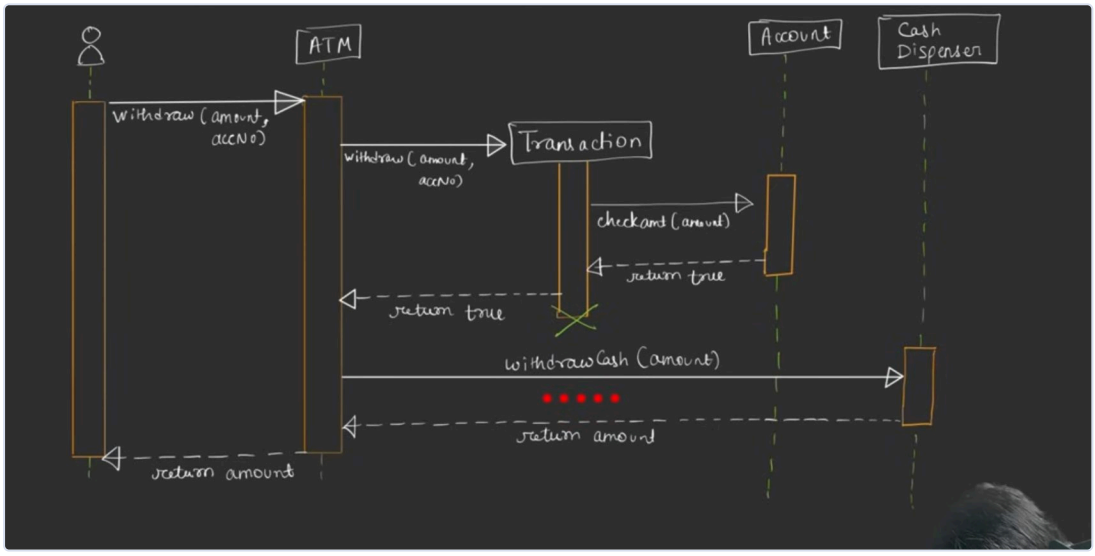
INTERVIEW TRAP: AGGREGATION OR COMPOSITION? (IT IS SUBJECTIVE)

There is no universal answer, so **state your assumption** and ask the lifecycle question: *"Does a Menu exist if the Restaurant burns down?"* Yes → aggregation. No → composition. Interviewers reward the reasoning, not the symbol.

Sequence diagrams

Golden rule: never draw one for the whole app; draw it for **one use-case** (e.g. ATM cash withdrawal).

Element	Meaning
Lifeline	Dashed vertical line: how long an object lives during the use-case.
Activation bar	Thick rectangle on a lifeline: the object is awake and processing.
Create / Destroy	Message that starts a new lifeline; an **X** ends one.
Synchronous call	Solid line, **closed** arrow; sender waits. The reply is a **dashed** line.
Asynchronous call	Solid line, **open** arrow; fire-and-forget.
`alt` / `opt` / `loop`	if-else / plain if / for-while fragments.



ATM withdrawal sequence: User → ATM → Transaction → Account, then Cash Dispenser. Note the dashed returns and the Transaction lifeline ending with an X.

4 SOLID and design principles

▶ Video 5, 6 of the playlist

As an app grows to thousands of classes, tangled and tightly-coupled code causes severe bugs and debugging cost. SOLID (Robert C. “Uncle Bob” Martin, 2000) keeps code **clean, readable, maintainable and loosely coupled**.

S · Single Responsibility Principle

> **DEFINITION**
> A class should have **only one reason to change**.

Violation: a `ShoppingCart` that totals prices, prints invoices *and* saves to a database. A change to pricing, invoice format or storage all hit the same class and risk regressions.

Fix: split by responsibility and connect with composition: `ShoppingCart` (totals) · `CartInvoicePrinter` (rendering) · `CartDbStorage` (persistence).

> **TRAP: THE "ONE METHOD MYTH"**
> SRP does **not** mean one method per class. A class may have many methods as long as they all serve **one** responsibility.

O · Open-Closed Principle

> **DEFINITION**
> Open for **extension**, closed for **modification**: add features without editing battle-tested files.

Violation: adding `saveToMongo()` / `saveToFile()` inside `CartDbStorage`.
Fix: abstract `Persistence { virtual save() = 0 }` with `SqlPersistence`, `MongoPersistence`, `FilePersistence`. Need Cassandra tomorrow? New class `CassandraPersistence`; nothing existing is touched.

L · Liskov Substitution Principle

> **DEFINITION**
> A child must be usable **anywhere its parent is expected**, without breaking or throwing surprises.

Violation: `Account` has `deposit()` and `withdraw()`. A `FixedDepositAccount` overrides `withdraw()` to throw. A client looping over accounts and calling `withdraw()` now crashes.

Bad fix (trap): `if (acc is FixedDepositAccount) skip;` in the client. This couples the client to concrete classes and breaks OCP with every new account type.

Good fix: restructure the hierarchy so the invalid call is impossible:

```
DepositOnlyAccount        { deposit() }
 ├─ FixedDepositAccount
 └─ WithdrawableAccount   { withdraw() }       // extends DepositOnlyAccount
     ├─ SavingsAccount
     └─ CurrentAccount
```

The client keeps one list per interface, so it can never ask an FD account to withdraw.

The three formal LSP rules

Rule	Requirement for a child	Example
Signature	Argument types: identical or **broader** (contravariant)	C++ forces identical; changing it is a compile error.
	Return type: identical or **narrower** (covariant)	Parent returns `Animal` → child may return `Dog`, not `Organism`.
	Exceptions: same or **narrower**	Parent throws `LogicError` → child may throw `OutOfRange`, not `RuntimeException` (client's catch would miss it).
Property	**Class invariant** must always hold	`BankAccount`: `balance ≥ 0`. A `CheatAccount` allowing negatives violates LSP.
	History constraint: respect parent's state transitions	Parent always allowed withdraw; FD that throws breaks history. A parent's immutable (`final`) field cannot be made mutable.
Method	**Pre-conditions**: may be **weakened**, never strengthened	Parent needs password ≥ 8. Admin child accepts ≥ 6 (ok). Requiring ≥ 10 breaks clients passing 8.
	Post-conditions: may be **strengthened**, never weakened	`Car::brake()` guarantees speed drops. `ElectricCar` also guarantees charge rises (ok); "speed unchanged" is not.

> **MEMORY HOOK**
> Pre-conditions: child may ask for *less*. Post-conditions: child may promise *more*. Inputs go broader, outputs go narrower.

I · Interface Segregation Principle

> **DEFINITION**
> Many **client-specific** interfaces beat one general-purpose interface. Never force a class to implement methods it does not need.

Violation: a `Shape` interface with `area()` and `volume()`; `Square` must implement `volume()` just to throw "not applicable".
Fix: `TwoDimensionalShape { area() }` and `ThreeDimensionalShape { area(); volume() }`. Square implements the first, Cube the second.

D · Dependency Inversion Principle

> **DEFINITION**
> High-level modules must not depend on low-level modules. **Both depend on abstractions.** (High-level = business logic like `UserService`; low-level = databases, APIs, file systems.)

Violation: `UserService` directly instantiates `MySqlDatabase` / `MongoDatabase`; swapping to Cassandra means refactoring the service.
Fix: a `Database` interface with `save()`; both DBs implement it; `UserService` holds a `Database*` that is **injected through the constructor** (Dependency Injection).

> **FAMOUS MANTRA**
> **If OCP is the target, DIP is the solution.**

Beyond SOLID: principles you will cite constantly

Principle	Meaning
Composition over inheritance	"The solution to inheritance is *not* more inheritance." Prefer Has-A with swappable parts (see Strategy, Decorator, Bridge).
DRY	Don't Repeat Yourself: duplicated logic goes into one reusable place.
Law of Demeter Least Knowledge	"Talk only to your immediate friends." A method may only call methods on: **(1)** its own object, **(2)** objects passed as parameters, **(3)** objects it creates, **(4)** its direct member variables. Avoid `a.getB().getC().doIt()` (friends of friends); expose a delegating method instead (`a.doIt()`).
Principles are ideals	Not physical laws. Like time-vs-space in DSA, real designs trade principles against each other (see next chapter). Minimise violations, but know which you are bending and why.

MODELLING THE REAL WORLD

Real objects are far too rich to model fully (a Human eats, sleeps, works…). Extract **only the behaviours relevant to this application**: a `User` books rides in Ola but orders food in Swiggy.

5 Worked example: Document Editor (bad → good → trade-offs)

▶ **Video 7 of the playlist**

A classic that exercises every SOLID principle. It also introduces the two modelling directions.

Top-down
Design the top wrapper class first, then the internals and their dependencies.

Bottom-up industry preferred
Design the smallest entities first (elements, persistence), then assemble into containers.

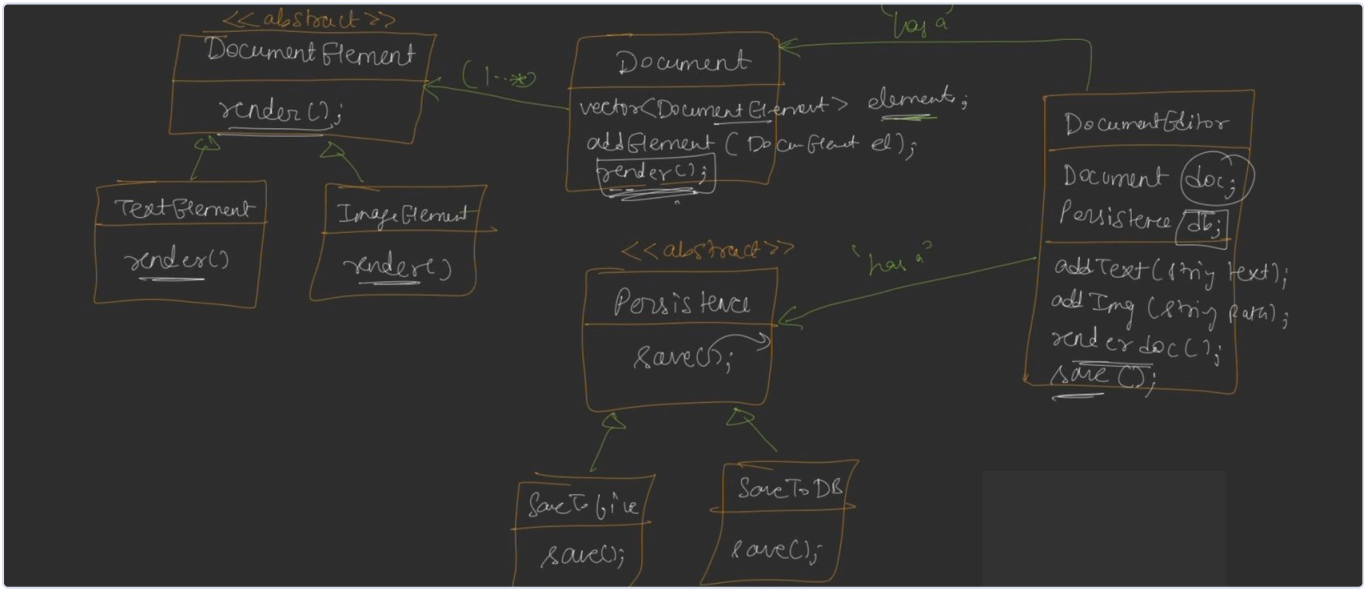
The bad design

One `DocumentEditor` holds a `vector<string>` for *all* content, a cached rendered string, and does `addText`, `addImage`, `renderDocument` and `saveToFile`. Rendering guesses "is this an image?" by checking whether the string ends in `.jpg/.png`.

- **SRP broken:** changes to I/O, rendering rules or element types all hit one class.
- **OCP broken:** a new `Table` / `Video` needs a new `add…()` and more branches in the hacky conditional.

The scalable design

- **Element hierarchy:** abstract `DocumentElement { render() }` with `TextElement`, `ImageElement`, `NewLineElement`, `TabSpaceElement`. A `BoldElement` is just another subclass.
- **Document:** 1-to-many with elements; `addElement()`; its `render()` just loops and calls `element->render()` (polymorphic delegation).
- **Persistence:** abstract `Persistence { save(content) }` with `FileStorage`, `DbStorage` (DIP).
- **DocumentEditor:** a facade holding `Document*` and `Persistence*`, exposing `addText/addImage/render/save`.

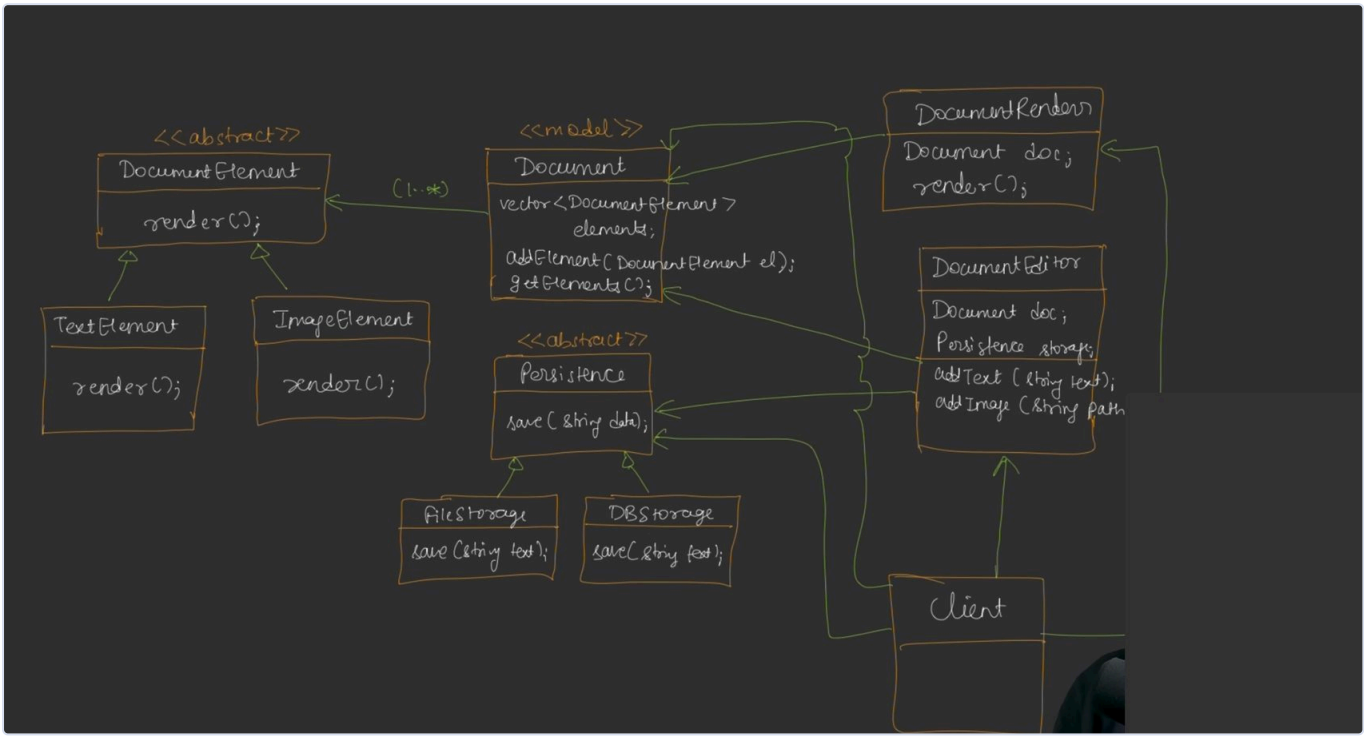


Version 1: DocumentElement hierarchy, Document aggregating elements, Persistence abstraction, and the DocumentEditor facade tying them together.

Evolving it, and the trade-offs interviewers push on

Trade-off 1 · SRP vs client simplicity

The editor still knows about rendering and saving; if `render()` 's signature changes, the editor changes. Fix: strip both out. A `DocumentRenderer` does formatting, `Document` becomes a pure model (CRUD on elements), and the **Client** coordinates Document → Editor → Renderer → Persistence directly.



Version 2: Document is a pure model; DocumentRenderer and Persistence are separate; the Client coordinates them.

Trade-off 2 · Law of Demeter vs SRP

`DocumentRenderer::render()` calls `doc->getElements()` then loops over them: it is talking to a friend of a friend. Moving the loop into `Document` would satisfy Demeter but give `Document` two jobs (storage + rendering), violating SRP.

> **THE GOLDEN ANSWER**
> "Low-level design has no perfect solutions, only trade-offs. I'm choosing to violate the Law of Demeter here to preserve SRP, so rendering is not coupled to the document's state model."

PART 2

Part 2 · Design Patterns

Proven answers to recurring design problems. Learn to recognise the *problem shape*, not to memorise class names.

6 The pattern mindset, and Strategy

WHY PATTERNS EXIST

Patterns are accumulated solutions to problems every application meets. **Change is the only constant**, so the golden rule behind all 23 GoF patterns is: **find what changes, separate it from what stays the same.**

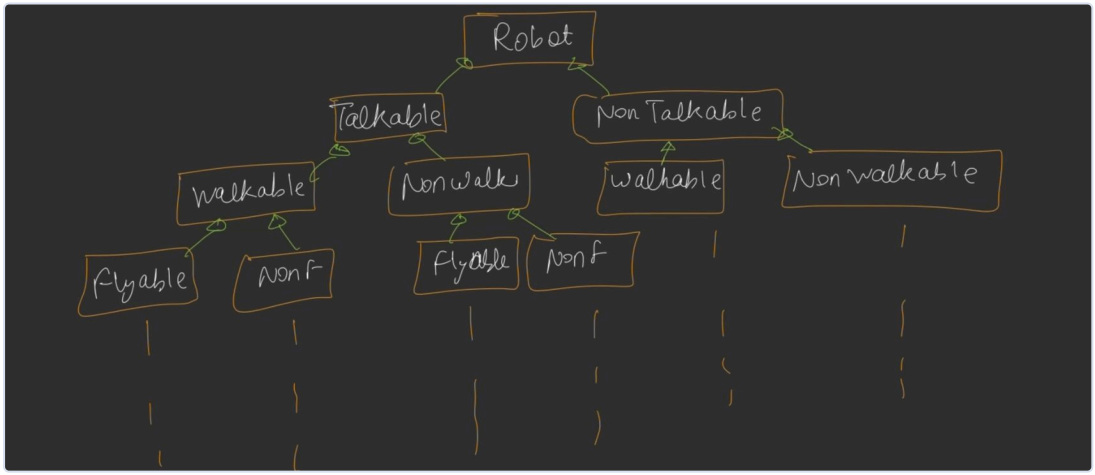
The three families

Family	Question it answers	Patterns in these notes
CREATIONAL	How do I **create** objects cleanly?	Singleton · Factory (Simple / Method / Abstract) · Builder · Prototype
STRUCTURAL	How do I **compose** classes into bigger structures?	Adapter · Facade · Decorator · Proxy · Composite · Bridge · Flyweight
BEHAVIORAL	How do objects **talk and share work**?	Strategy · Observer · Command · State · Template Method · Chain of Responsibility · Iterator · Mediator · Memento · Visitor (+ Null Object idiom)

Why inheritance alone fails: the Robot simulator

A `Robot` base class has `walk()`, `talk()`, abstract `projection()`. Now some robots fly:

- **Copy `fly()` into each flying robot:** duplicated code, breaks **DRY**.
- **Put `fly()` in `Robot`:** companion and worker robots now fly. Wrong.
- **Add layers (`FlyableRobot`, `NonFlyableRobot`, `FlyWithWings`, `FlyWithJet`, `Walkable`…):** a permutation explosion that breaks OCP and reuse.



Inheritance explosion: every capability multiplies the number of subclasses.

BEHAVIORAL Strategy

▶ Video 8

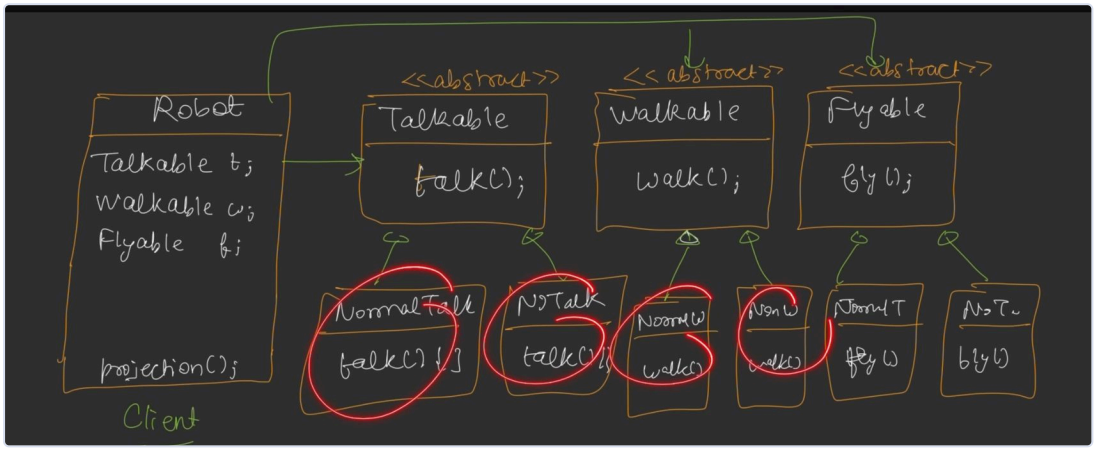
Define a family of algorithms, encapsulate each in its own class, and make them interchangeable at run-time.

Recognise it when… "support multiple ways to do X" (payment modes, sort algorithms, discount types, play modes, split types) or you catch yourself writing `if (type == …)` around behaviour.

Mantra: favour composition over inheritance. Steps:

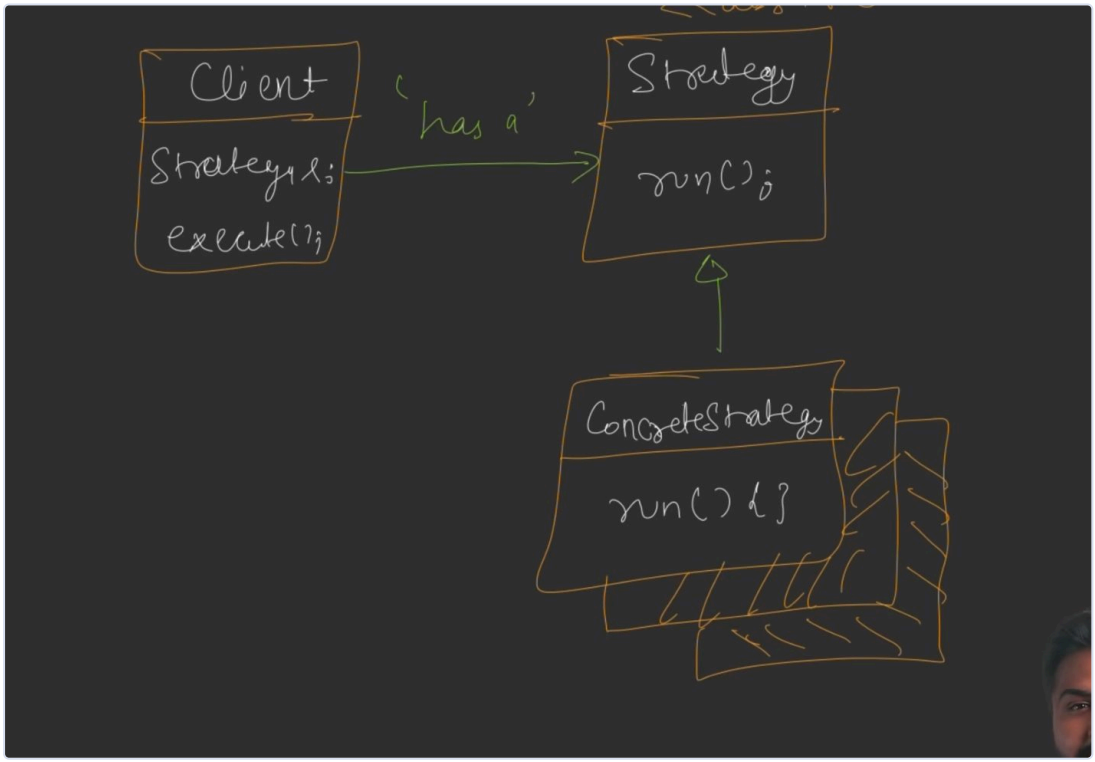
1. **Extract** each varying behaviour into an interface (`Talkable` , `Walkable` , `Flyable`).
2. **Concrete strategies** for each variant (`NormalTalk` , `NoTalk` , `FlyWithJet` , `NoFly`). Doing nothing (`NoFly`) is a valid strategy.
3. The main class **Has-A** reference to each interface.
4. **Delegate:** `Robot::walk() { walkStrategy->walk(); }` . The context is a "dumb object".

```
Robot r(new NormalTalk(), new NoWalk(), new FlyWithJet(), new CompanionProjection());  // any
combination, zero new classes
```



Robot with Talkable, Walkable and Flyable strategies injected by composition.

Going further: if `projection()` stays abstract you still subclass `Robot` . Extract a `Projectable` strategy too and `Robot` becomes a concrete class with 4 strategy references; **zero inheritance**.



Generic Strategy UML: Client/Context → Strategy interface ← ConcreteStrategies.

> **REAL-WORLD USE**
> `PaymentService` + `PaymentStrategy` (UPI / Card / NetBanking) · `Sorter` + `SortingStrategy` (Quick / Merge / Heap).

7 Creational patterns

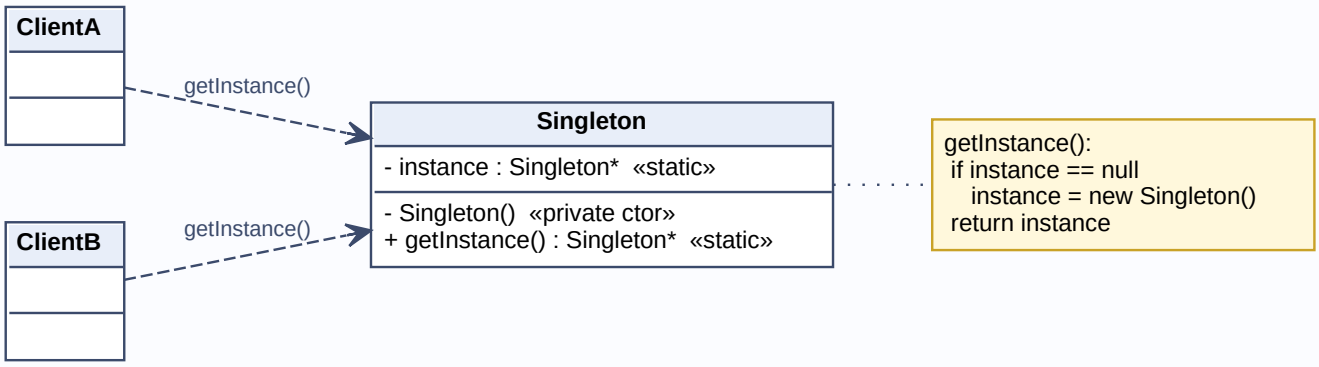
CREATIONAL Singleton

▶ Video 10

Exactly one instance of a class across the whole application, with a global access point.

Recognise it when… "single source of truth": a manager, logger, config, connection pool, in-memory registry of everything.

Three steps: (1) **private constructor** so nobody can `new` it; (2) a **private static** `instance`; (3) a **public static** `getInstance()` that creates on first use and returns the same object. It must be `static` so you can call it as `Singleton::getInstance()` before any object exists.



Singleton: private constructor, static instance, one global access point.

Initialisation strategies

Approach	How it works	Trade-off
Lazy (naive)	Create inside `getInstance()` when first requested.	**Not thread-safe:** two threads can both see `null` and create two objects.
Lazy + lock	Lock on every `getInstance()` call.	Safe but slow: locking on every call is expensive.
Double-checked locking	① check null (no lock) → ② lock → ③ check null again → create.	Best lazy option: lock only on first creation; the 2nd check stops the loser thread creating a duplicate.
Eager	Create at class-load time (before `main`).	Naturally thread-safe, no locks; wasteful if object is heavy and never used.

```
static Singleton* getInstance() {
  if (instance == nullptr) {                // check 1: no lock, fast path
    lock_guard<mutex> lock(mtx);
    if (instance == nullptr)                // check 2: inside lock
      instance = new Singleton();
  }
  return instance;
}
```

REAL-WORLD USE

Logger (no colliding file writes) · DB connection manager (connections are heavy) · Configuration manager (API keys, environment settings).

INTERVIEW TRAP: WHEN NOT TO USE IT

Entities needing **distinct state per instance** (a player in a game, a dark store's inventory manager). Downsides: global static state makes **unit testing and mocking hard**, and thread-safety adds complexity. Say it explicitly when you choose Singleton for a Manager.

CREATIONAL Factory family

▶ Video 9

Move object-creation logic (`new` + conditionals) out of business logic into a dedicated factory.

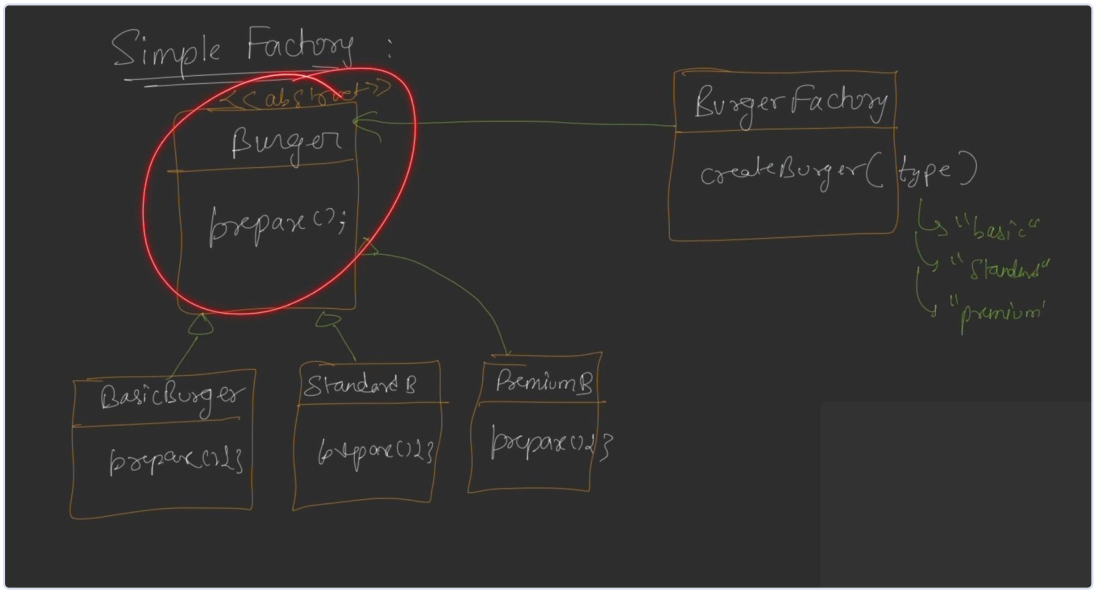
Recognise it when… a type/enum/string decides which concrete class to build ("create the right Order / Notification / Piece / Gateway"). Used in ~99% of LLD solutions.

Core problem: mixing business logic (send notification, process order) with creation logic (`if (type==SMS) new SmsSender…`) bloats code and couples it to concrete classes. The client should just ask the factory and receive an object.

1 · Simple Factory idiom, not GoF

One concrete factory with `createBurger(type)` and an `if/else` or `switch` returning `BasicBurger / StandardBurger / PremiumBurger` (all subclasses of abstract `Burger`).

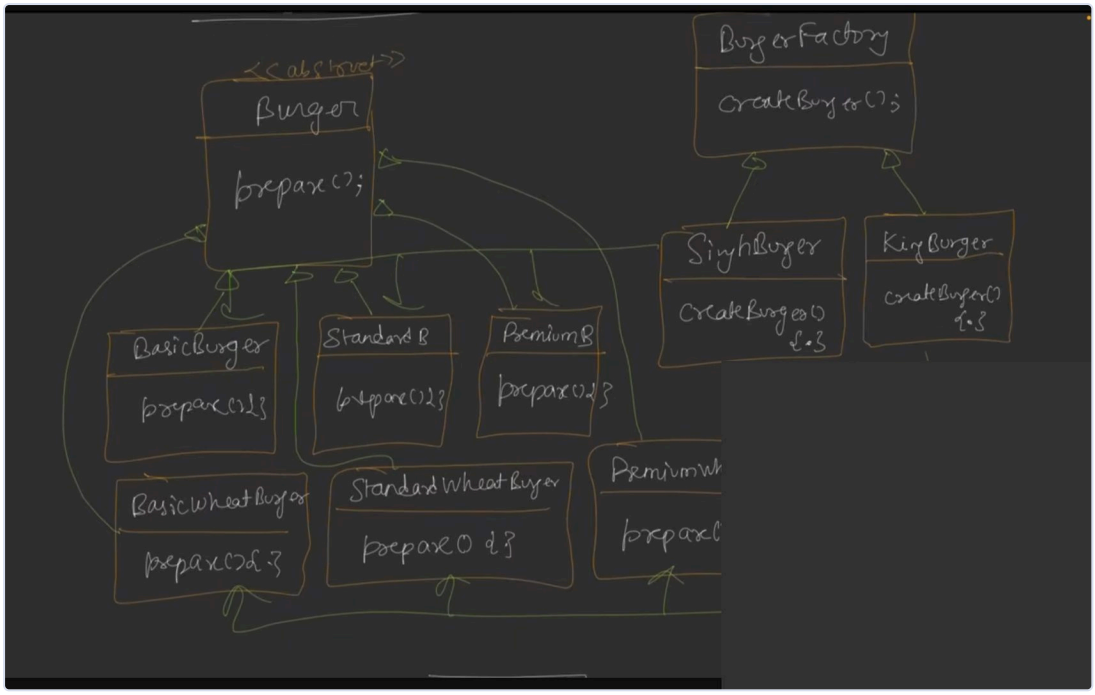
Limitation: adding a new product means editing the `if/else` , which violates **OCP**.



Simple Factory: BurgerFactory::createBurger(type) switches on "basic / standard / premium".

2 · Factory Method GoF

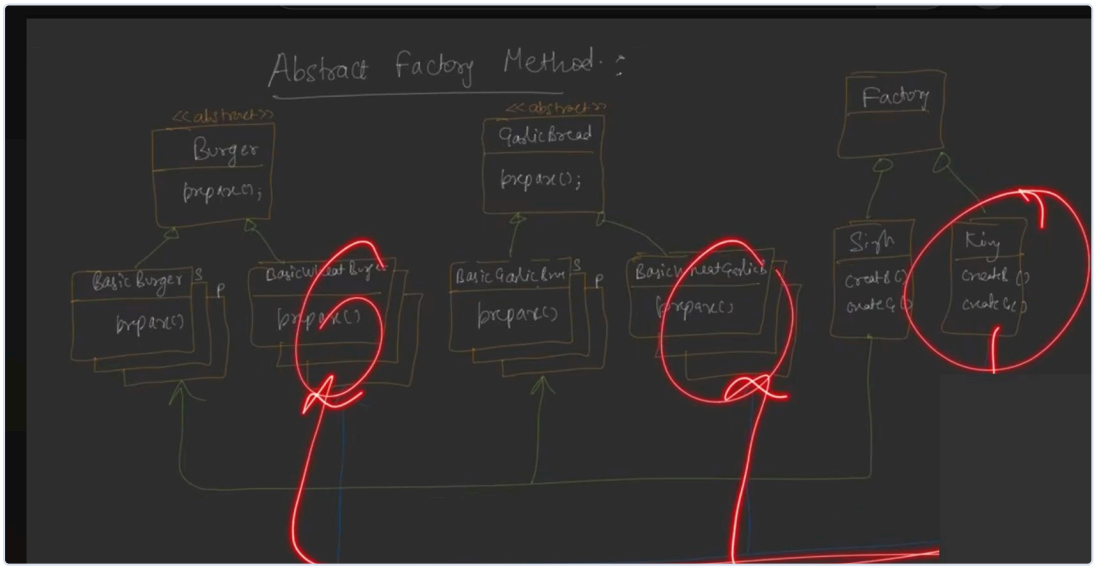
"Define an interface for creating an object, but let subclasses decide which class to instantiate." Abstraction now exists on **both** sides: abstract `BurgerFactory { createBurger() = 0 }` with concrete factories per franchise (`SinghBurger` , `KingBurger` producing wheat burgers). Two parallel hierarchies: **Product** and **Creator**. New franchise = new subclass; **OCP satisfied**.



Factory Method: parallel Burger (product) and BurgerFactory (creator) hierarchies.

3 · Abstract Factory GoF

*"An interface for creating **families of related objects** without naming concrete classes."* Use when a factory must make **several related products**: `MealFactory` defines `createBurger()` **and** `createGarlicBread()`; `KingBurger` returns `WheatBurger + WheatGarlicBread`. Guarantees **family consistency**: a client can't accidentally pair a normal burger with wheat bread.



Abstract Factory: each concrete factory builds a matching Burger and GarlicBread.

Variant	Creates	OCP?	Pick it when
Simple Factory	one product, switch on type	No (edit the switch)	Few stable types, quick solution in an interview
Factory Method	one product, subclass decides	Yes	Creation varies by "creator" subclass
Abstract Factory	a **family** of products	Yes	Products must be consistent as a set

PRAGMATIC NOTE

Most manager-level factories in the case studies (`DiscountStrategyManager` , `PieceFactory`) use an **enum + switch** on purpose, breaking OCP *locally* to shield all high-level code from instantiation details.

CREATIONAL Builder

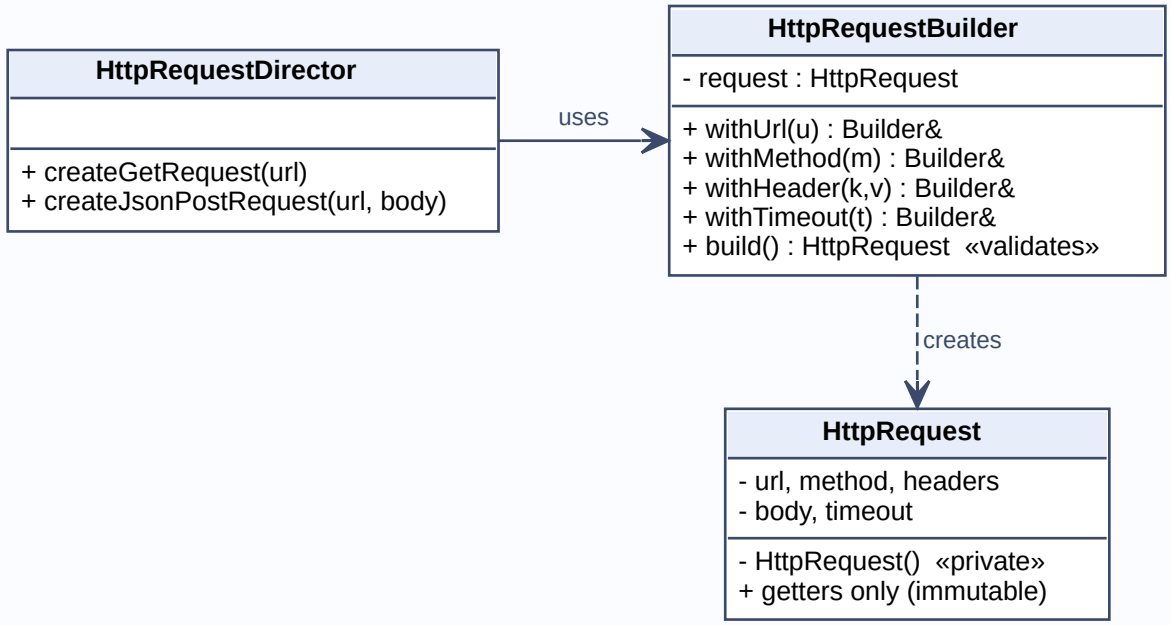
▶ Video 28

Construct a complex object step by step, separating construction from representation.

Recognise it when… an object has many optional parameters (HttpRequest: URL, method, headers, query, body, timeout) or must be **immutable and valid at creation**.

What raw constructors/setters get wrong:

1. **Constructor telescoping:** a constructor for every parameter combination.
2. **Mutability:** using setters to avoid long constructors makes fields changeable.
3. **Inconsistent state:** a client can call `execute()` on a half-built object → run-time error.
4. **Scattered validation** across callers.



Classic Builder with an optional Director. The target has a private constructor and only getters.

Classic Builder

Target constructor is `private` (builder is a friend/inner class), only getters (immutable). Builder methods `withUrl()` , `withMethod()` … set a field and `return this` to enable chaining; `build()` **validates** and returns the object.

```
HttpRequest r = HttpRequestBuilder()
    .withUrl("https://api.example.com").withMethod("POST")
    .withHeader("Content-Type","application/json").withTimeout(60)
    .build();                                  // validation happens here, once
```

Variants

Variant	Idea	Use when

Builder + Director	A `Director` exposes presets (`createGetRequest(url)` , `createJsonPostRequest(url, body)`) that call the builder's steps.	Standard configurations recur.
Step Builder	One interface per step: `UrlStep → MethodStep → HeaderStep → OptionalStep → build()` . The concrete builder implements **all** step interfaces; each method returns the *next* step type. Entry `start()` returns `UrlStep` .	You need **compile-time** enforcement of order and mandatory fields (IDE only shows the valid next call).

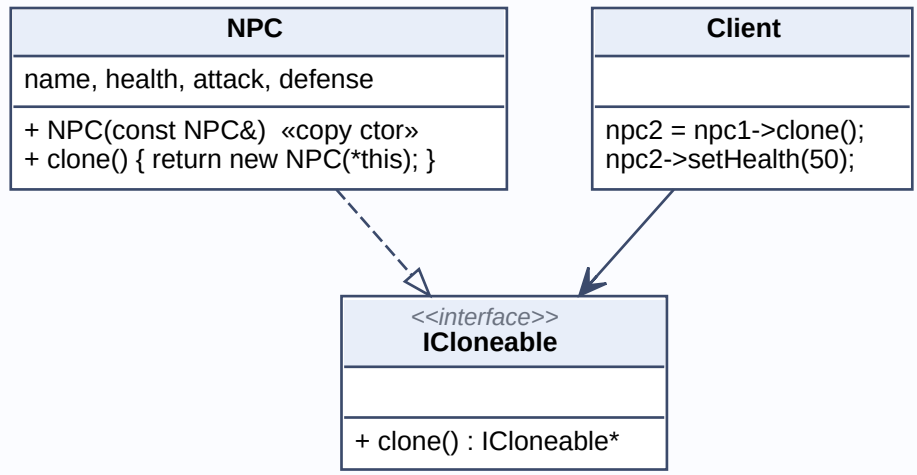
CREATIONAL Prototype

▶ Video 36

Create new objects by cloning an existing, already-initialised instance.

Recognise it when… object creation is expensive (DB calls, heavy computation, loading assets) and you need many similar copies (hundreds of NPCs, cached config snapshots).

Initialise **once** through the heavy pipeline, then `clone()` . The concrete class has a **copy constructor** that copies fields (no DB hit) and `clone()` returns `new NPC(*this)` . Tweak differences after cloning with setters.



Prototype: ICloneable marker interface; concrete class implements clone() via a copy constructor.

TRAP: SHALLOW VS DEEP COPY

Shallow copies pointers as raw addresses, so original and clone share the same heap object and mutate each other. **Deep** allocates new memory and recursively copies referenced objects (`s1 = new Student(*other.s1)`). Use deep copy whenever the object owns pointers.

8 Structural patterns

STRUCTURAL Adapter

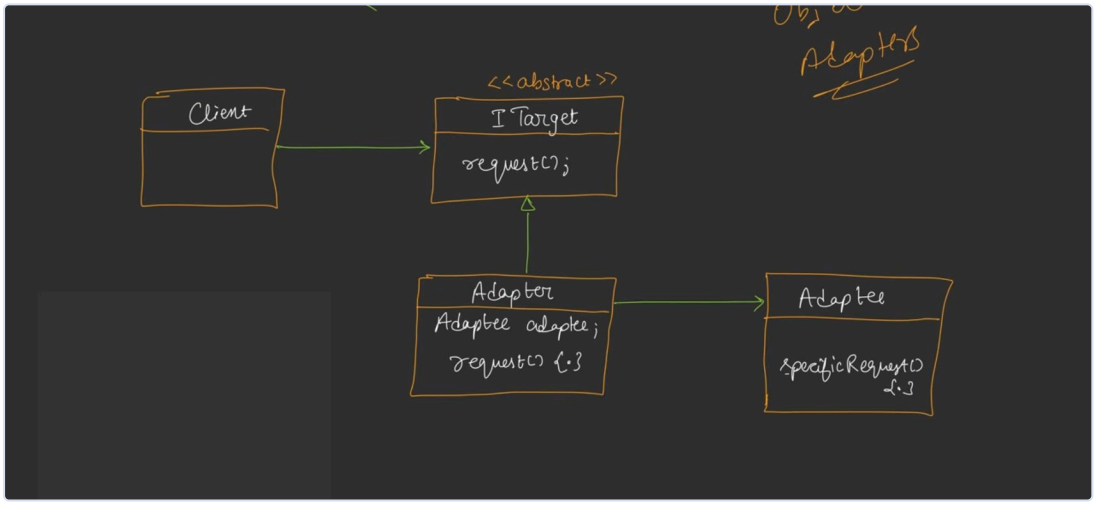
▶ Video 16

Convert one interface into another that the client expects, so incompatible classes can work together.

Recognise it when… integrating a **third-party or legacy** API whose methods don't match your interface (XML vs JSON; Razorpay/Stripe; Twilio/WhatsApp; Bluetooth vs wired speaker SDKs). "We may switch vendors later."

Why: calling a vendor API directly in client code couples you to it; switching vendors means refactoring everywhere (OCP violation). Think of two incompatible puzzle pieces and the adapter as the connecting piece (or a travel plug adapter).

- **Target** (`ITarget`): interface the client expects, e.g. `getJsonData()` .
- **Adaptee**: the incompatible class, e.g. `XmlDataProvider::getXmlData()` .
- **Adapter**: **Is-A** Target *and* **Has-A** Adaptee. `adapter->getJsonData()` calls `adaptee->getXmlData()` , converts, returns.



Adapter: Client uses ITarget; Adapter implements it and wraps the Adaptee.

	Object Adapter standard	Class Adapter
How	Composition (Has-A adaptee)	Multiple inheritance (Is-A Target and Is-A Adaptee)
Verdict	Flexible, composition over inheritance, works in every language incl. Java	Needs multiple inheritance (C++ yes, Java no); tightly coupled to adaptee internals

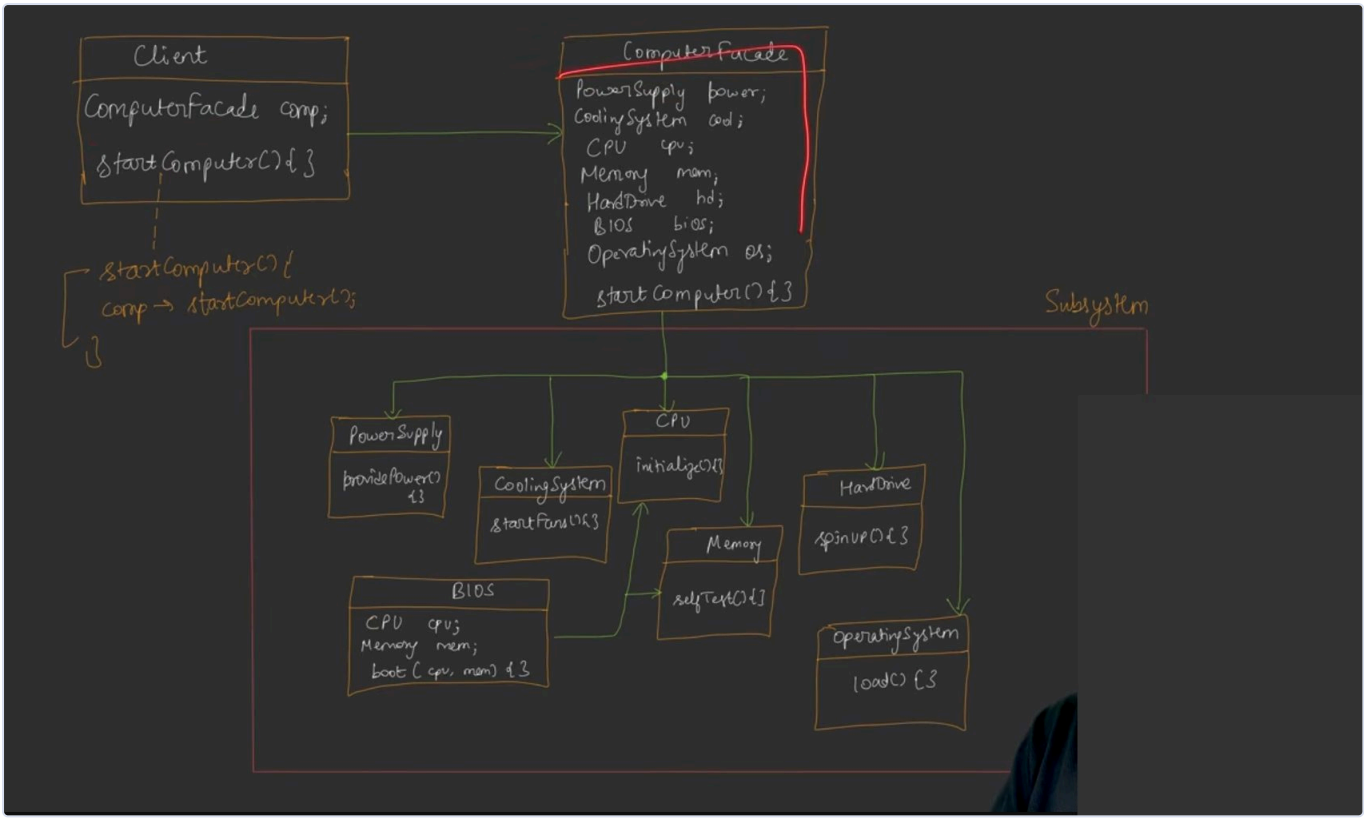
STRUCTURAL Facade

▶ Video 17

One simplified, unified entry point to a complex subsystem.

Recognise it when… a client would otherwise orchestrate many classes in a precise sequence (boot a PC, start a game engine, run a payment, control a music player). Every "Orchestrator / Manager / Controller / App" class in the case studies is a facade.

Example: starting a computer touches CPU (initialize), Memory (self-test), HardDrive (spin up), BIOS, OS, PowerSupply, Cooling in a fixed order. `ComputerFacade::startComputer()` hides all of that.



Client talks only to ComputerFacade; the facade drives the whole subsystem.

FACADE ENFORCES THE LAW OF DEMETER

It is the one immediate friend between client and subsystem, so the client never navigates `a.getB().getC()`.

INTERVIEW TRAP: FACADE VS ADAPTER

Intent: Facade *simplifies / hides* a large subsystem; Adapter *translates* mismatched interfaces. **Structure:** Adapter implements an *existing* Target interface and wraps one Adaptee; Facade defines a *brand-new* simple interface over many classes.

REAL-WORLD USE

Game engine `startGame()/loadGame()` (assets, memory, physics, rendering) · `makePayment()` (balance check, PIN, fraud, ledger, notification).

STRUCTURAL Decorator

▶ Video 13

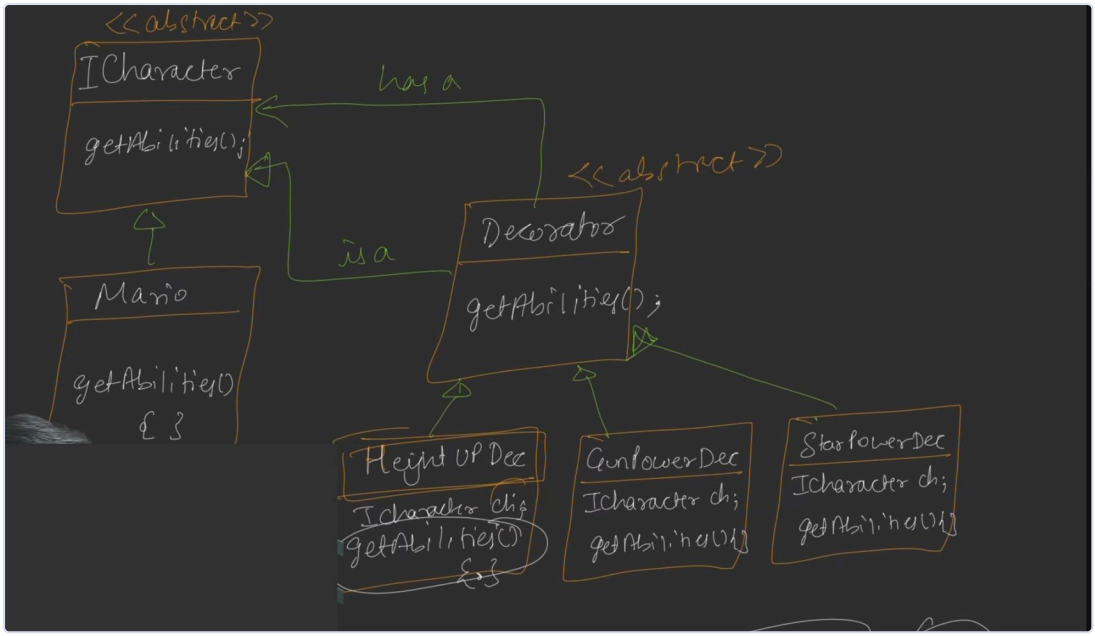
Attach extra responsibilities to an object **dynamically at run-time**, by wrapping it.

Recognise it when… features combine freely and you'd need a class per combination (Mario + height + gun + star; bold + italic + underline; timestamp + signature on a notification; validators stacked on a request).

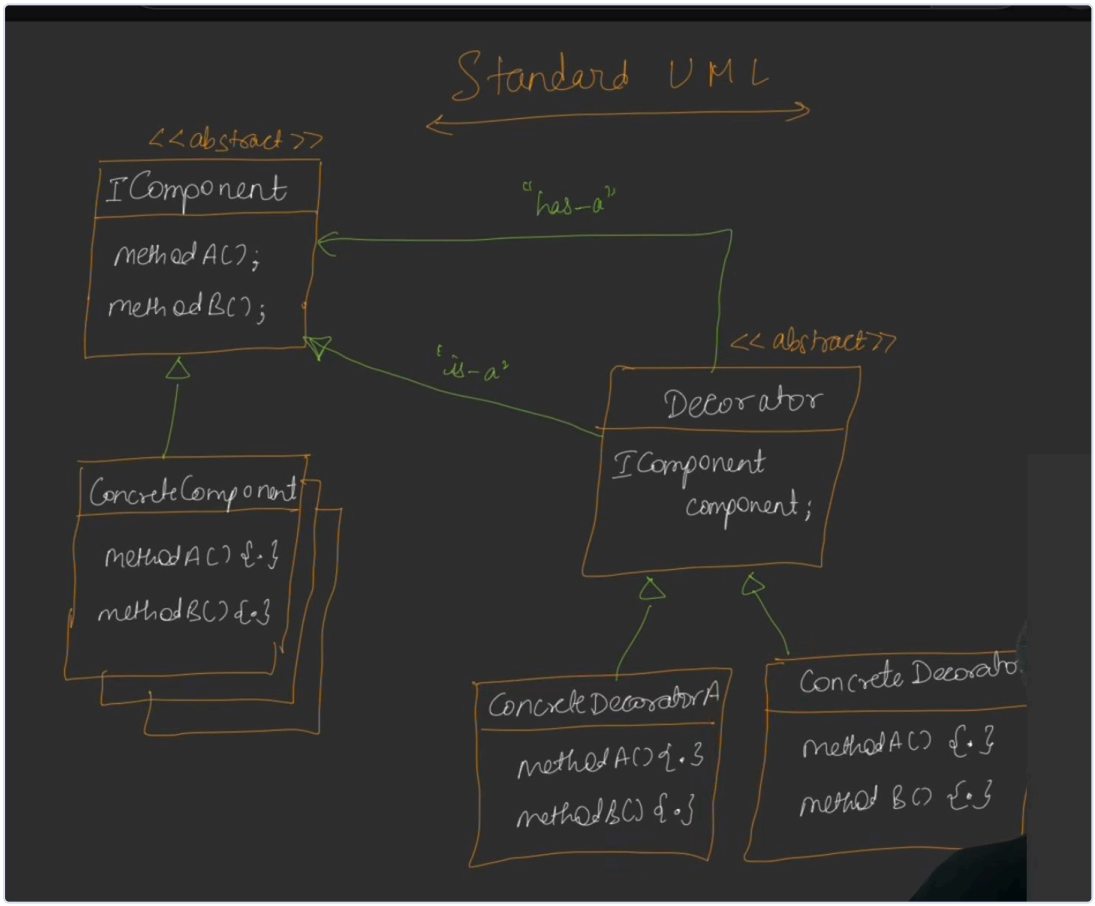
The anti-pattern: with inheritance, Mario powers explode into `MarioWithHeight`, `MarioWithGun`, `MarioWithHeightAndGun`, `MarioWithFlyHeightAndGun`…

THE UNIQUE HYBRID: IS-A AND HAS-A

Is-A: a Decorator *is a* Component, so it can be passed anywhere a component is expected and wrapped by other decorators. **Has-A:** it *holds* a Component and delegates to it before/after adding its own behaviour.



Mario example: ICharacter, Mario (concrete), abstract Decorator (is-a + has-a), concrete power-up decorators.



Standard Decorator UML: IComponent, ConcreteComponent, abstract Decorator, ConcreteDecorators.

Stacking is onion-like:

```
new StarPowerDecorator(new GunPowerDecorator(new HeightUpDecorator(new Mario())));
// getAbilities() recurses inward to Mario, then each layer appends on the way back:
//  "Mario" → "Mario + HeightUp" → "… + Gun" → "… + StarPower"
```

REAL-WORLD USE

Rich-text formatting (Bold/Italic/Underline) · request-validation pipelines (Email, SQL-injection, XSS checks) · Java I/O: `BufferedReader(new InputStreamReader(new FileInputStream(...)))` .

STRUCTURAL Proxy

▶ Video 21

A stand-in object that controls access to the real object, behind the same interface.

Recognise it when… you need lazy loading, an access/permission check, or a local handle for something remote; or want to add cross-cutting logic (retry, caching, logging) *without touching* the real class.

Both `Proxy` and `RealSubject` implement `ISubject` (**Is-A**, so the client can't tell them apart) and the proxy holds a reference to the real one (**Has-A**).

Virtual proxy

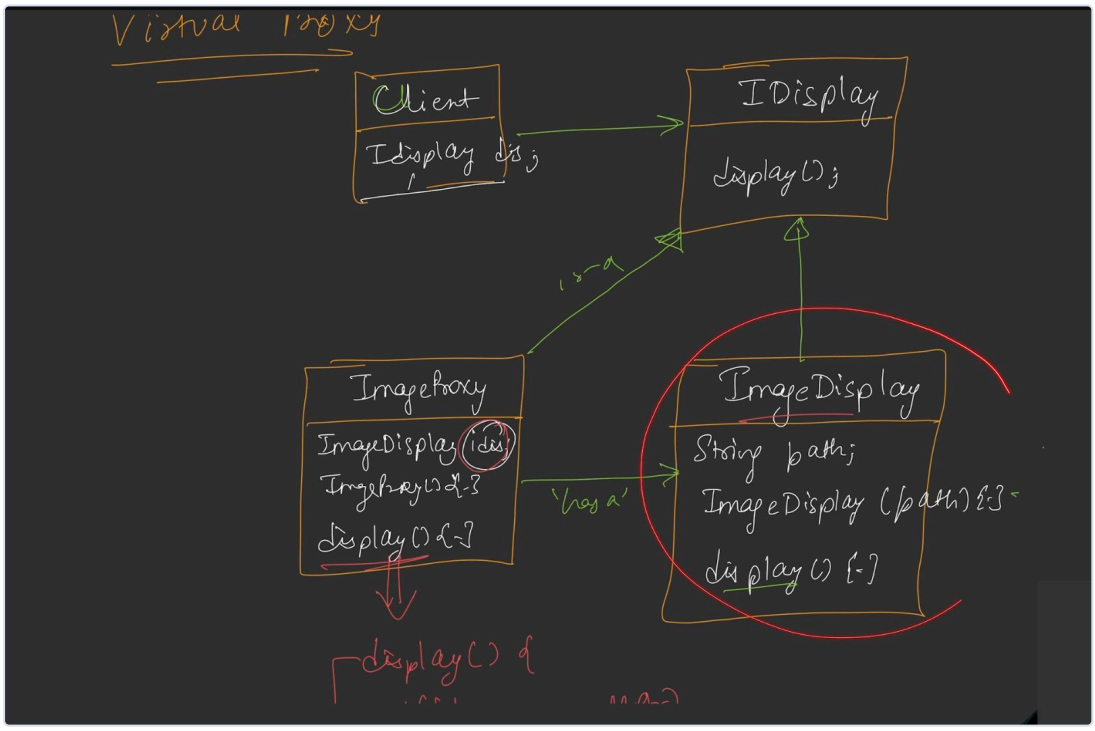
Lazy-loads an expensive resource: the image loads from disk only when `display()` is called.

Protection proxy

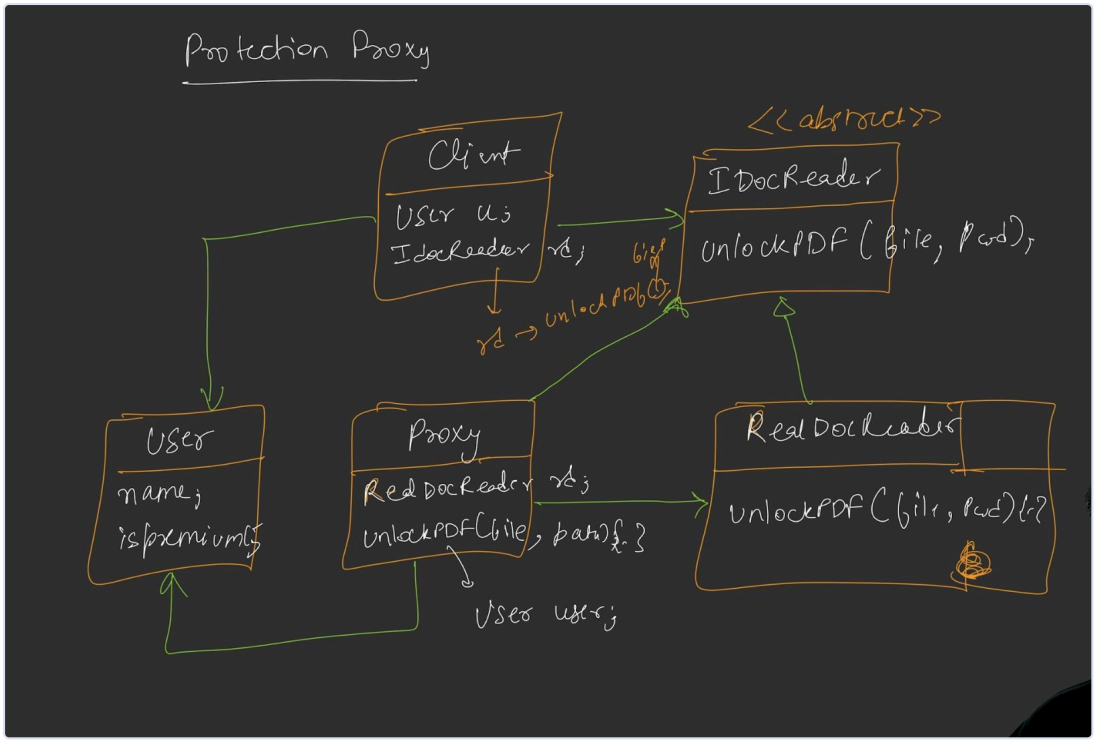
Checks credentials/permissions first: `unlockPDF()` works only if `user.isPremium` .

Remote proxy

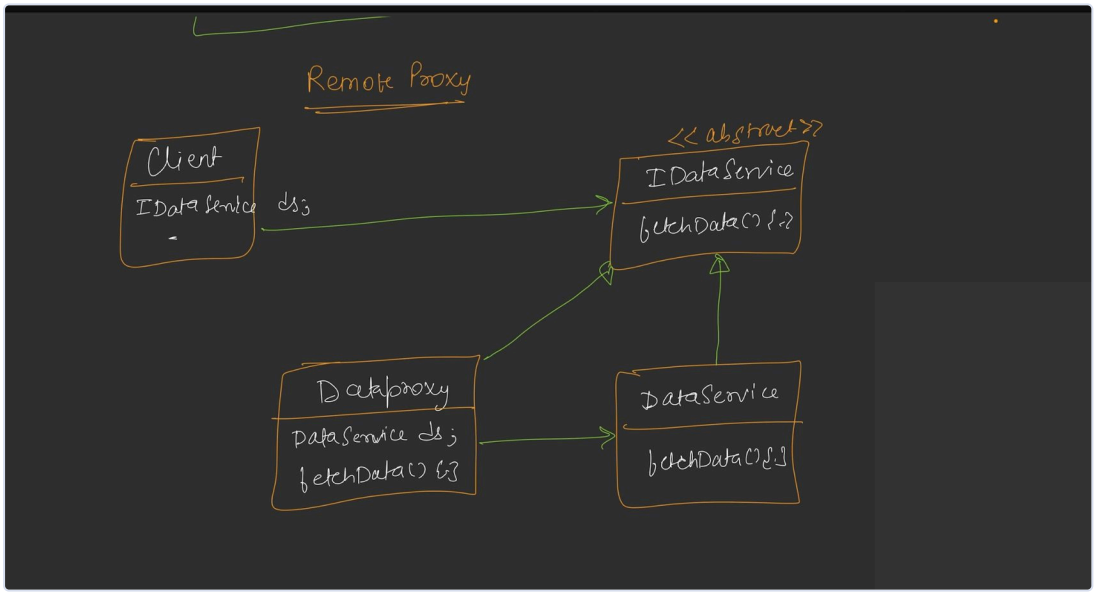
Local object representing one on another server; hides sockets/HTTP behind a simple method. `UserProxy` → `UserService` .



Virtual proxy: ImageProxy implements IDisplay and creates ImageDisplay only on first display().



Protection proxy: the proxy checks User.isPremium before delegating unlockPDF().



Remote proxy: DataProxy hides the network call to DataService.

> **REAL-WORLD USE**
> Microservice client stubs (CartService → UserProxy → UserService over HTTP) · lazy media loading · API gateways (auth, rate limiting) · the retry proxy in the Payment Gateway case study.

STRUCTURAL Composite

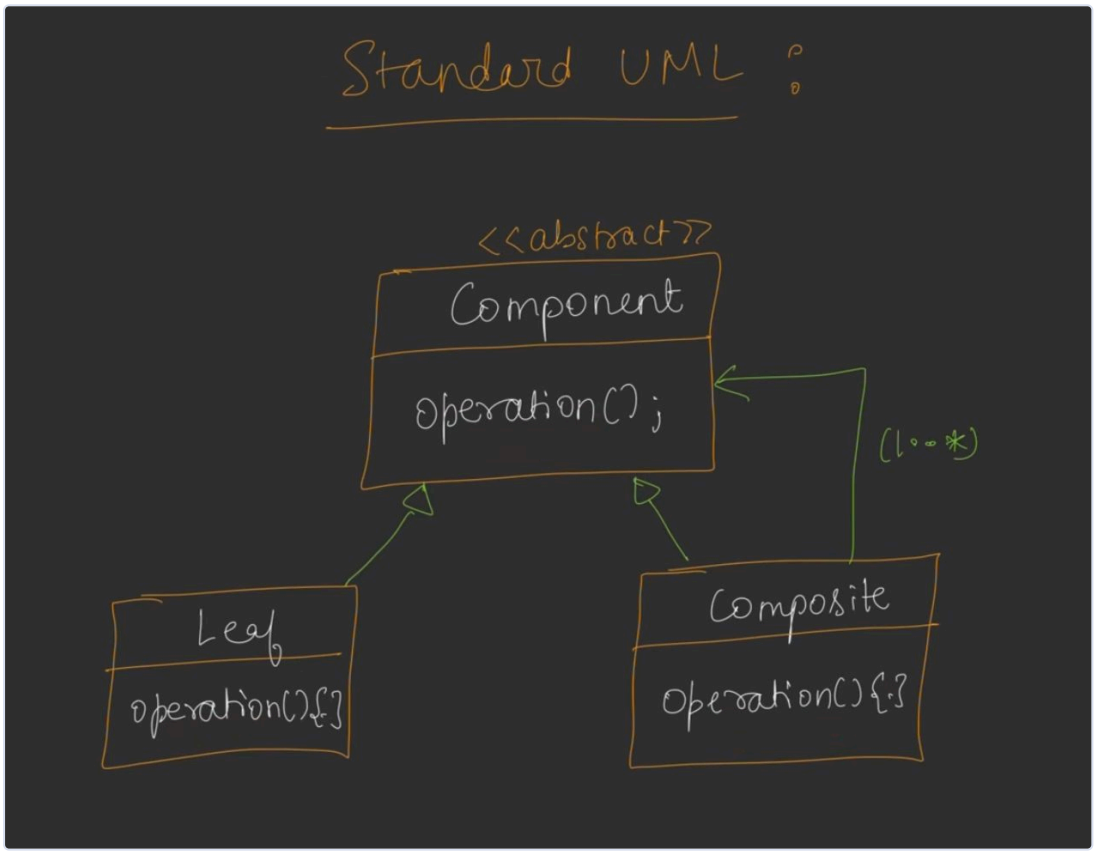
▶ Video 19

Treat individual objects and groups of objects uniformly through one interface, for tree-shaped part-whole structures.

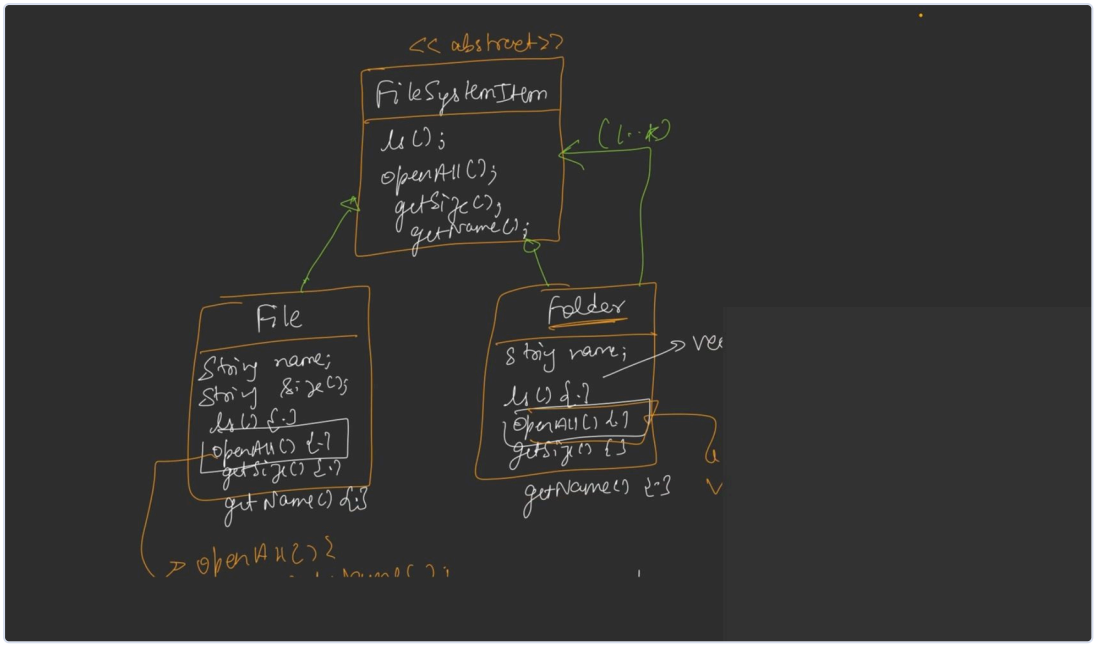
Recognise it when… the domain is a **tree** with leaves and containers: file system (File/Folder), menus with sub-menus, org charts (employee/manager), UI/DOM trees.

Without it the client keeps separate lists and writes `if (isFolder)` checks for every recursive operation (`ls`, `getSize`, `openAll`).

- **Component** (`FileSystemItem`): common abstract interface.
- **Leaf** (`File`): Is-A Component; does the terminal work. `getSize()` returns its size; `cd()` returns null (unsupported).
- **Composite** (`Folder`): Is-A Component **and** Has-A `vector<Component*>` (1-to-many). Operations **recurse** over children, not caring whether a child is a leaf or another composite.



Standard Composite UML: Component, Leaf, Composite holding 1.. Components.*



File-system example: Folder holds a vector of FileSystemItem and delegates ls/openAll/getSize.

STRUCTURAL Bridge

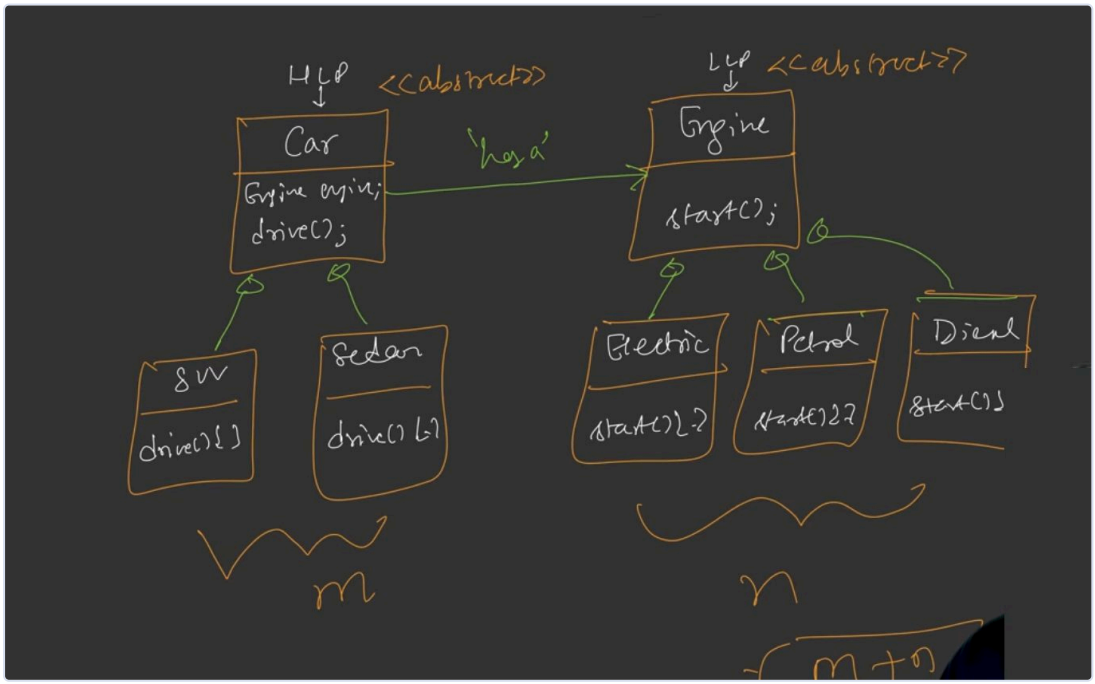
▶ Video 25

Decouple an abstraction from its implementation so the two hierarchies vary independently.

Recognise it when… two **independent dimensions** multiply (car type × engine type, remote × TV, UI widget × OS). "Adding one new X shouldn't require N new classes."

The m × n problem: *m* cars × *n* engines via inheritance = `SedanWithPetrol`, `SUVWithDiesel` …, **m × n** classes; one new engine (CNG) adds *m* more.

Solution: separate the high-level **Abstraction** (`Car` → Sedan, SUV) from the low-level **Implementor** (`Engine` → Petrol, Diesel, Electric). `Car` **Has-A** `Engine*` (the "bridge"); `car->drive()` calls `engine->start()`. Total classes: **m + n**.



Bridge: Car hierarchy (m) bridged to Engine hierarchy (n) = m + n classes.

INTERVIEW TRAP: BRIDGE VS STRATEGY

The UML is nearly identical (a class holding an interface). **Strategy** = *swap algorithms at run-time* on a live object. **Bridge** = *structural decoupling at design time* so both layers can grow with new classes; once a pair is wired (petrol in a sedan) you don't swap engines mid-operation.

REAL-WORLD USE

Remotes (Standard/Touch) × TVs (LCD/LED/OLED) · cross-platform GUI toolkits (TextBox/Dropdown) × OS renderers (Windows/macOS/Linux) · Zepto's InventoryManager × InventoryStore.

STRUCTURAL Flyweight

▶ Video 30

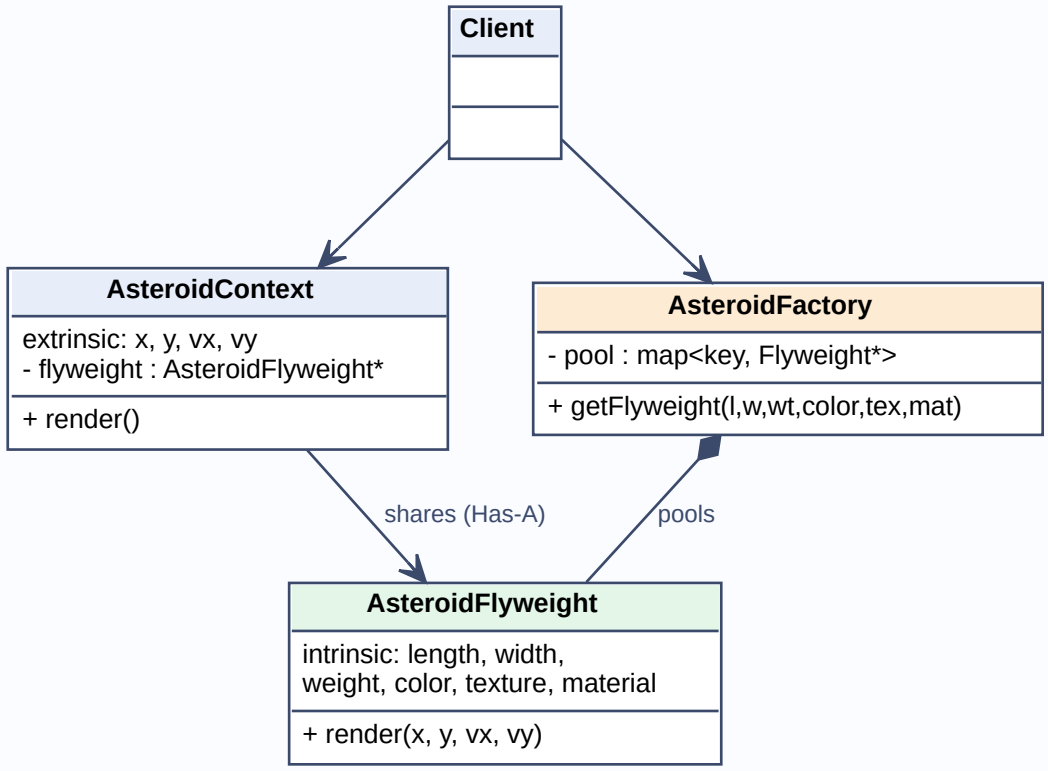
Save RAM by sharing the common part of millions of small objects.

Recognise it when… the scale is huge (1M asteroids, trees in an open world, characters in a document) and many objects are near-identical. Ask: *"Is RAM limited here? Should these be lightweight?"*

Split every object's state in two:

	Intrinsic state	Extrinsic state
Nature	Constant, **shared**, context-independent	Varies per instance, **not shareable**
Examples	Length, width, weight, colour, texture; glyphs 'A'–'Z'	Position (x,y), velocity (vx,vy); cursor position, font size
Lives in	Inside the **Flyweight**	A **Context** object, or passed as method arguments

FlyweightFactory is a "smart factory" with a pool (`unordered_map<string, Flyweight*>`). It builds a composite key from the intrinsic fields (`"10_10_1_Red_Hard"`); if present it returns the shared object, otherwise creates, stores and returns it.



Flyweight: many lightweight Context objects share a few pooled Flyweights.

IMPACT (SPACE SHOOTER, 1,000,000 ASTEROIDS)

Without: ~196 B each → **~186.9 MB**. With: 1M light contexts + **3 shared flyweights** → **~22.9 MB**, about **88% less RAM**.

TRAP: FLYWEIGHTS MUST BE IMMUTABLE

Hundreds of thousands of contexts point at one flyweight. Mutating its colour changes *every* referencing object. Pass extrinsic data as method parameters instead of storing it.

REAL-WORLD USE

AAA open-world games (shared meshes/textures/animations; per-instance coordinates and health) · text editors (shared glyphs; per-character font/colour/position).

9 Behavioral patterns

Strategy, the most-used behavioral pattern, is covered in Chapter 6.

BEHAVIORAL Observer

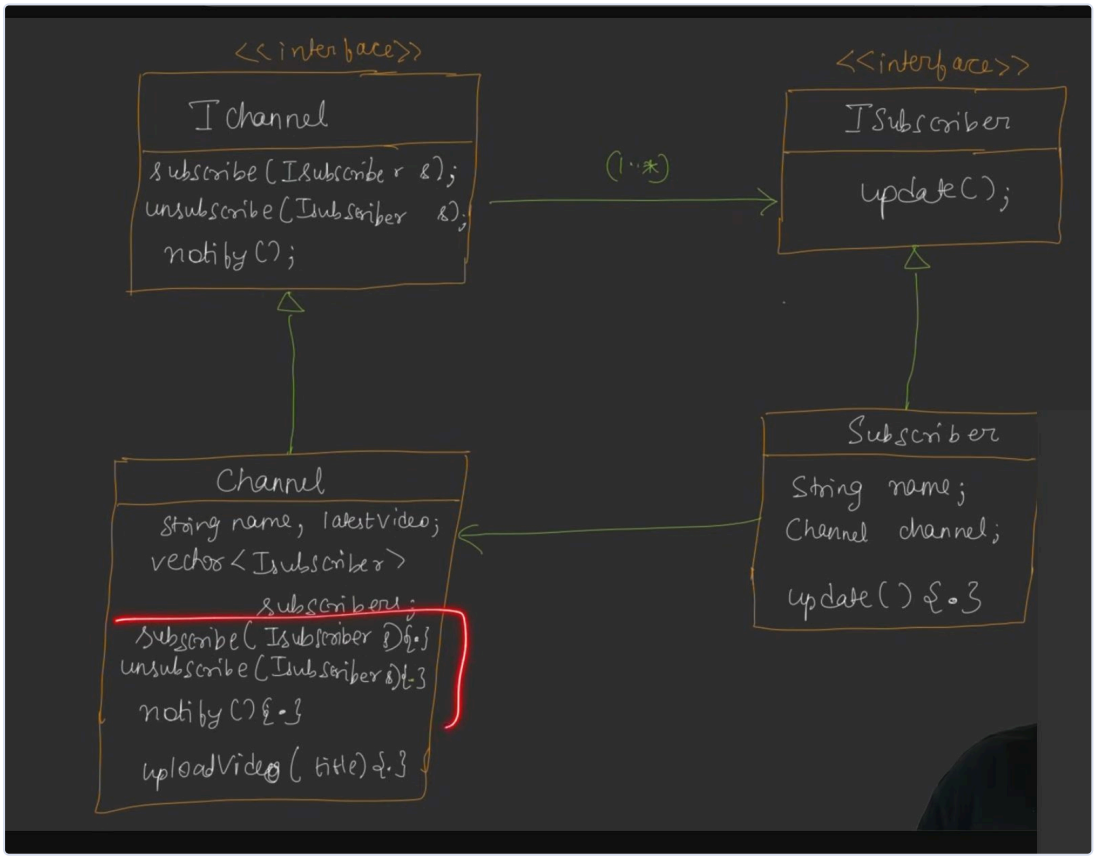
▶ Video 12

One-to-many dependency: when the subject changes state, all its observers are notified automatically.

Recognise it when… "notify / alert / subscribe / update when X happens": YouTube subscribers, order-status alerts, stock tickers, UI event listeners, group-expense notifications, game notifications.

Polling vs pushing: without Observer, observers repeatedly ask "has it changed?", wasting CPU and bandwidth on "no" answers. Observer **pushes**: the subject notifies registered observers itself. Also called **Publisher–Subscriber**.

Role	Responsibility
`IObservable` / `IChannel`	`subscribe(obs)` · `unsubscribe(obs)` · `notify()`
`IObserver` / `ISubscriber`	Single method `update()`
Concrete observable	Holds `vector<IObserver*>` and the real state (`latestVideoTitle`)
Concrete observer	Implements `update()` ; **Has-A** reference to the observable so it can pull data



Observer UML: IChannel (subscribe/unsubscribe/notify) ←→ ISubscriber (update); the subscriber holds the Channel to pull data.

Flow (push the signal, pull the data): ① observers `subscribe` → ② state changes (`uploadVideo()`) → ③ it calls `notify()` → ④ loop: `observer->update()` → ⑤ inside `update()` the observer calls `channel->getVideoData()` .

TRAP: OBSERVER BENDS SRP

The concrete channel manages observers *and* owns domain logic (`uploadVideo`). Strict fix: move the subscriber list and notify loop into the abstract base or a manager. Pragmatic answer: the notification logic is static, so keeping it in place avoids class bloat. Mention both.

VARIANTS USED IN THE CASE STUDIES

Targeted observer: a `map<userId, Observer*>` with `notifyUser(id, msg)` next to `notifyAll()` (Tinder). **Self-registration:** an observer's constructor fetches the singleton observable and calls `addObserver(this)` , so client code needs no wiring (Notification system, Tinder).

BEHAVIORAL Command

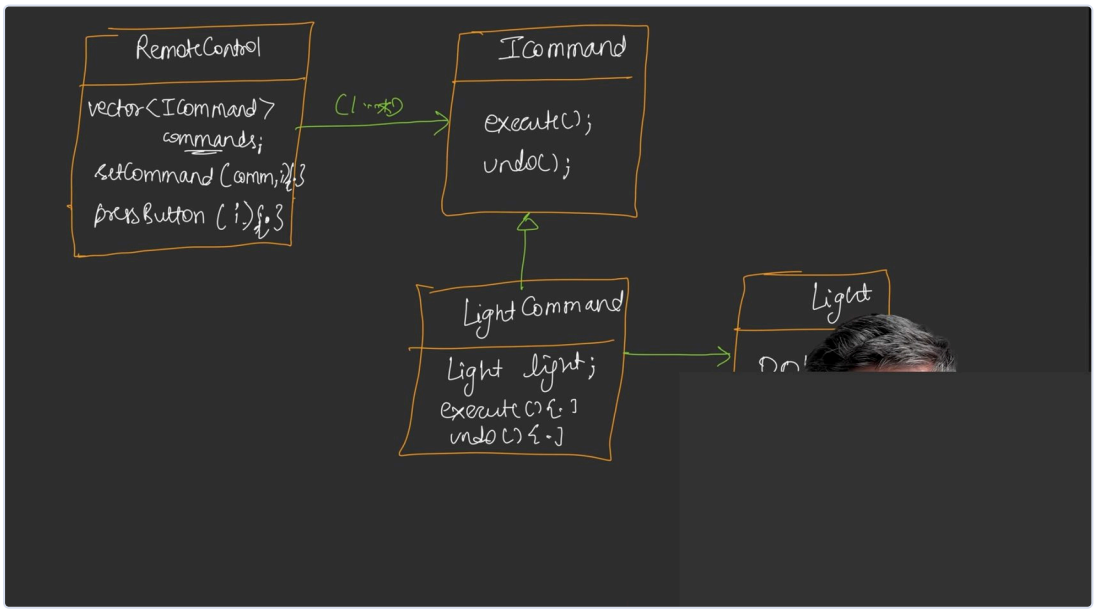
▶ Video 15

Turn a request into a standalone object, decoupling who triggers it from who performs it.

Recognise it when… you need **undo/redo**, request queues, scheduling, logging/replay, or a UI control whose action must be rebindable (remote-control buttons, keybindings, menu items).

Without it, receiver logic is hard-coded in the button or remote, so every new action edits the invoker (OCP violation).

Actor	Role
`ICommand`	Contract: `execute()` and `undo()` . Knows nothing about devices.
`ConcreteCommand`	(`LightCommand` , `FanCommand`) Has-A its **Receiver**; `execute()` → `light->on()` , `undo()` → `light->off()` .
Receiver	(`Light` , `Fan` , `AC`) Knows *how* to do the real work.
Invoker	(`RemoteController`) Holds `ICommand*` s; `pressButton(i)` → `commands[i]->execute()` . A "dumb launcher".
Client	Builds receivers, wraps them in commands, registers them with `setCommand(index, cmd)` .



Command: RemoteControl (invoker) → ICommand ← LightCommand → Light (receiver).

- **Toggle buttons:** keep a `buttonPressed[]` array: first press → `execute()` , next → `undo()` .

- **Global undo (Ctrl+Z):** push each executed command on a `stack<ICommand*>`; undo pops and calls `undo()`.
- Commands can be queued, delayed, serialised or logged for replay.

TRAPS

Why is Has-A on ConcreteCommand, not ICommand? The interface only defines *what* actions exist; only a concrete command knows *which* receiver it drives.

Why not one generic `Appliance` interface for all receivers? Devices have incompatible behaviours (on/off, setTemperature, setSpeed). A common base forces fake methods, violating **LSP/ISP**.

Downside: a class per action means many small classes.

REAL-WORLD USE

Undo/redo in Word, Docs, Photoshop · rebindable game controls · job queues and transactional systems with rollback.

BEHAVIORAL State

▶ Video 32

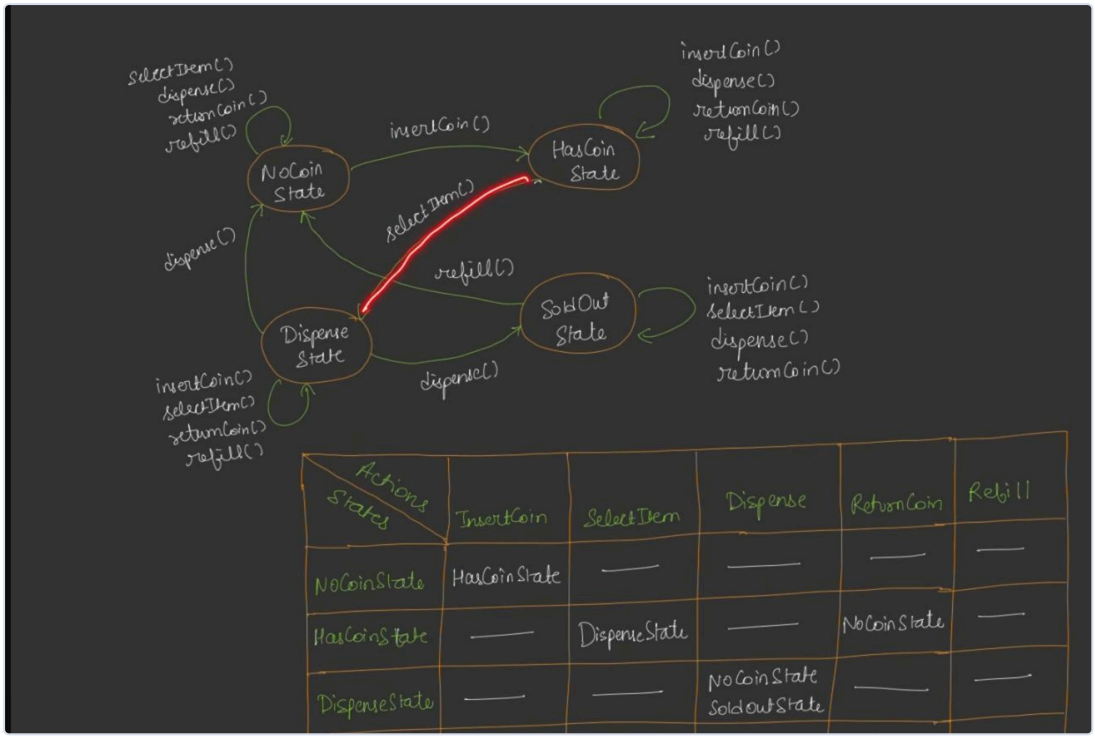
An object changes its behaviour when its internal state changes; it appears to change class.

Recognise it when… an object has a **finite set of states** and the same call means different things in each (vending machine, ATM, order lifecycle, document Draft→Review→Published). The tell: a growing `if (state == …)` / `switch` inside every method.

State Machine Diagram (from Theory of Computation): nodes = states, directed edges = operations. For any (state, operation) pair you either move deterministically to a new state or handle it in place.

Vending machine

State + operation	Result
`NoCoin` + `insertCoin`	→ `HasCoin`
`HasCoin` + `selectItem`	funds ok → `Dispense`
`HasCoin` + `returnCoin`	refund → `NoCoin`
`Dispense` + `dispense`	count-- ; if count > 0 → `NoCoin`, else → `SoldOut`
`SoldOut` + `refill`	restock → `NoCoin`
Invalid op (e.g. `selectItem` in `NoCoin`)	Print "Please insert coin first", **return the same state**

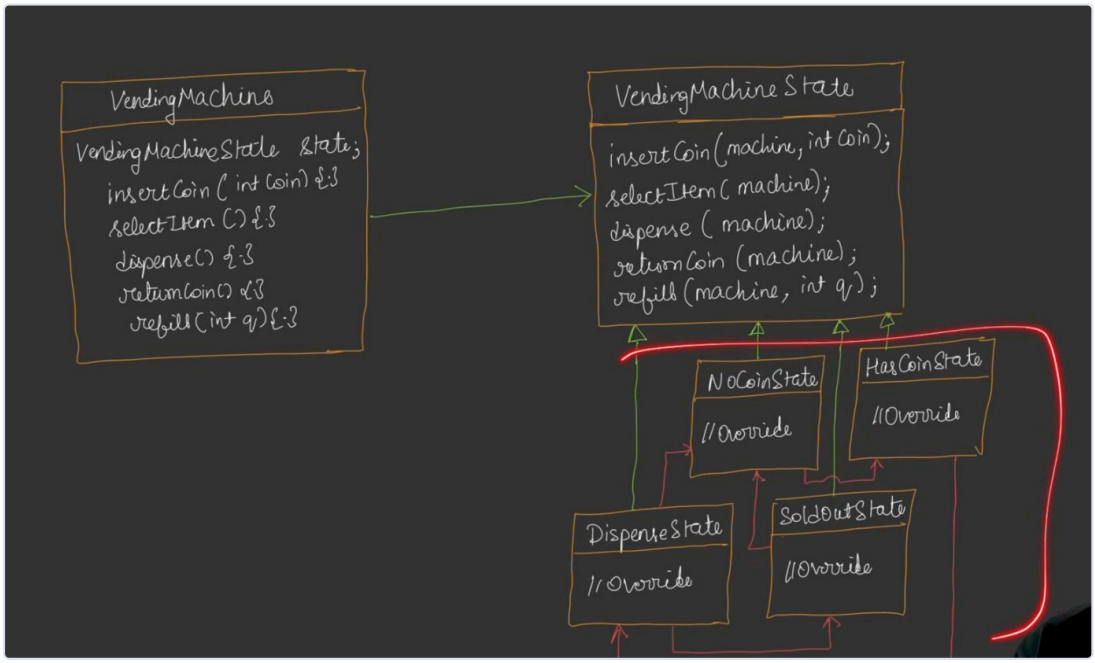


State machine diagram and the (state × operation) transition table for the vending machine.

- **Context** (`VendingMachine`): Has-A `currentState` ; contains **no conditional logic**, just delegates.
- **State interface**: one method per operation, each taking the context (`insertCoin(machine, coin)` …).
- **Concrete states**: encapsulate that state's rules and return the **next state**.

```
void VendingMachine::insertCoin(int coin) {
  currentState = currentState->insertCoin(this, coin);   // delegate + swap reference
}
```

Optimisation: the context pre-creates one instance of every state in its constructor and reuses them, avoiding object churn on each transition.



State pattern UML: VendingMachine → VendingMachineState ← NoCoin / HasCoin / Dispense / SoldOut.

WHY IT WINS

Kills `if/else` bloat · **SRP** (each state's rules in one class) · **OCP** (a new `MaintenanceState` touches no existing state).

REAL-WORLD USE

ATM (Idle → CardInserted → PINValidated → CashDispensing) · CMS document lifecycle · e-commerce order (Placed → Processing → Shipped → Delivered / Cancelled).

BEHAVIORAL Template Method

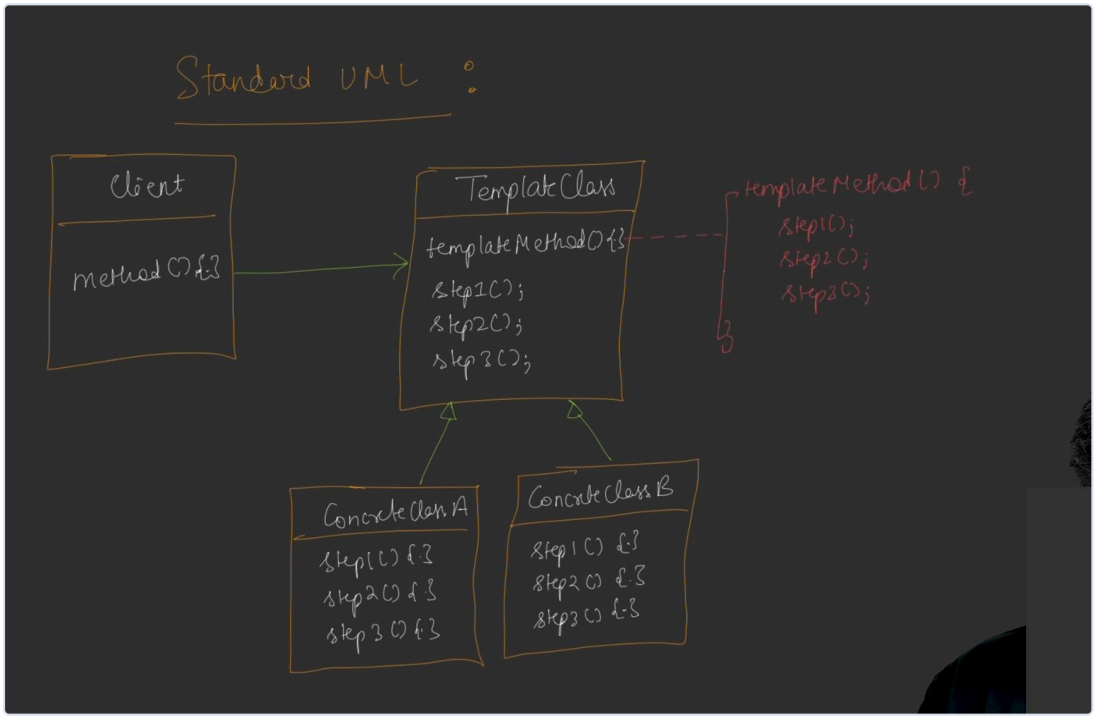
▶ Video 20

Fix the skeleton and order of an algorithm in a base class; let subclasses fill in the steps.

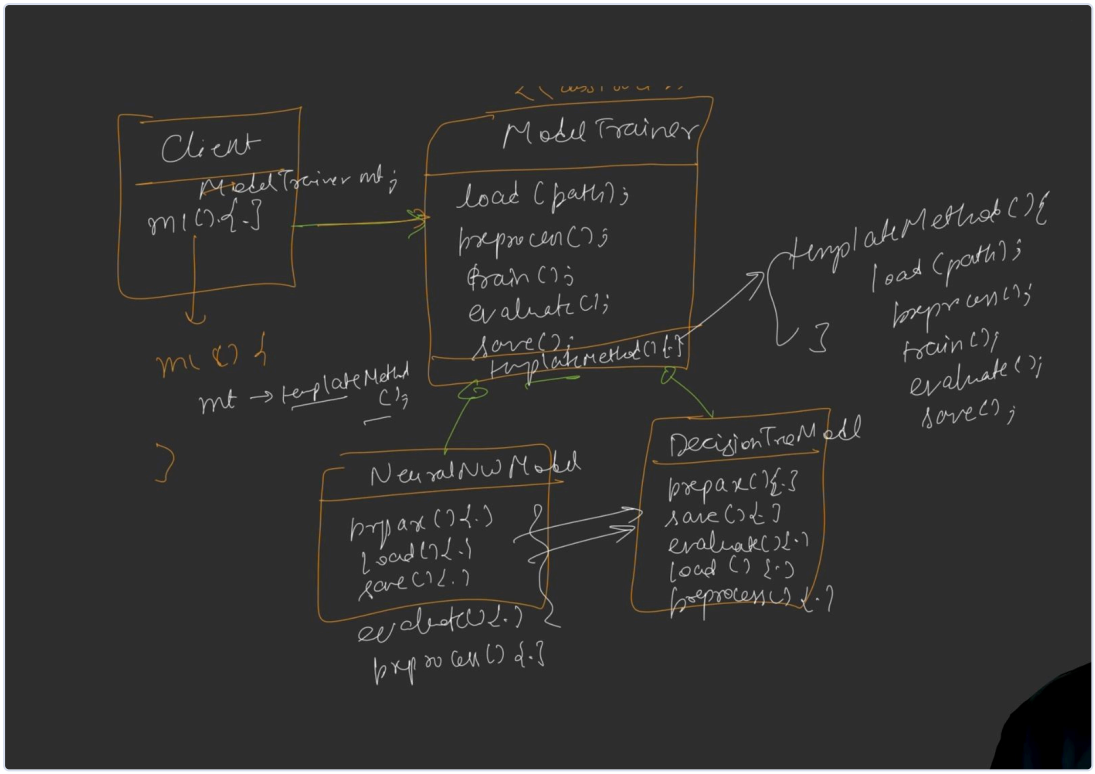
Recognise it when… several variants share the **same sequence of steps** that must never be reordered or skipped (ML training pipeline, payment flow, ETL, build lifecycle).

Problem: left alone, subclasses may reorder steps, skip mandatory ones, or train before pre-processing.

- **Template method** (`trainPipeline()`): in the abstract class, **non-overridable** (`final`); fixes the order `loadData → preprocessData → trainModel → evaluateModel → saveModel` .
- **Primitive (abstract) steps**: pure virtual; subclasses **must** implement (`trainModel` , `evaluateModel`).
- **Hook / default steps**: have a default body; subclasses may override (`loadData` , `saveModel`).



Standard Template Method UML: the template method calls step1/2/3; concrete classes override the steps.



ML pipeline: ModelTrainer.templateMethod(), with NeuralNetworkModel and DecisionTreeModel overriding steps.

REAL-WORLD USE

Payments: validate → debit → credit → verify · ETL: extract → transform → load · build tools: compile → test → package → deploy.

BEHAVIORAL Chain of Responsibility

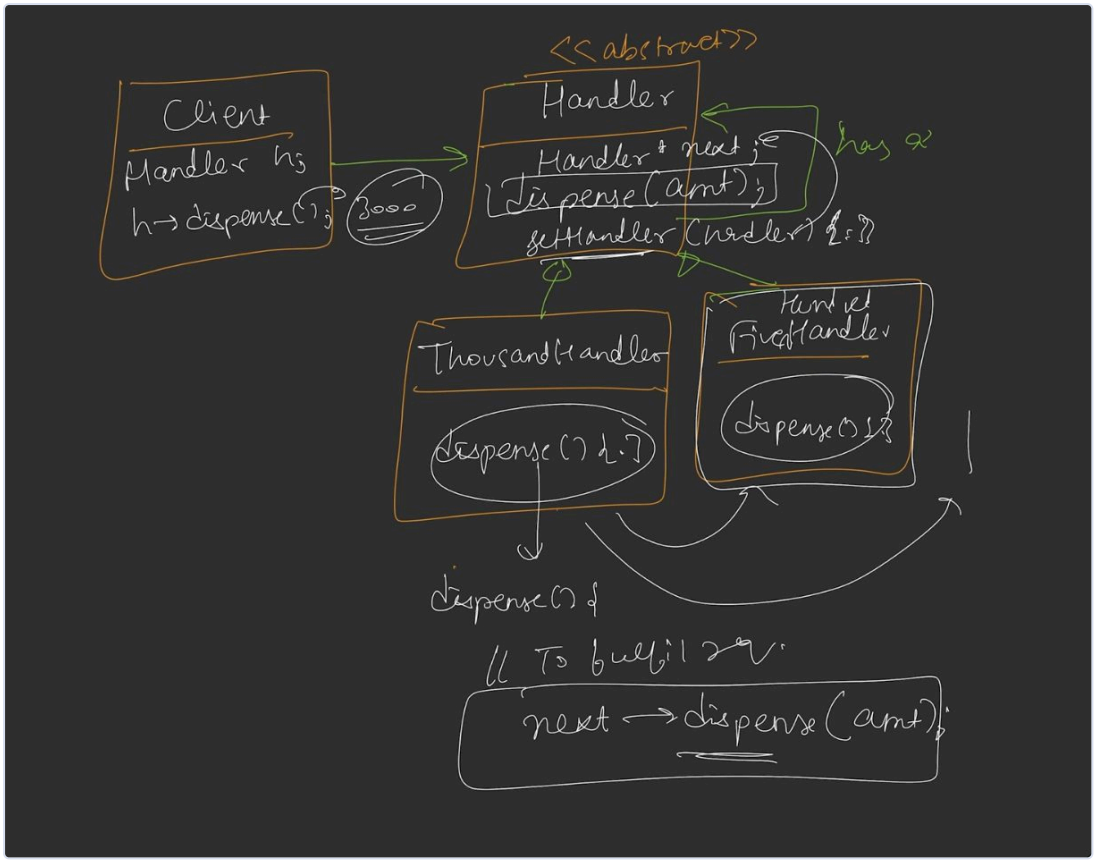
▶ Video 22

Pass a request along a chain of handlers; each decides to handle it, modify it, or forward it.

Recognise it when… a request goes through **sequential checks/handlers** that can **stop early** (ATM denominations, approval levels, log levels, middleware, discount stacking with "non-combinable" coupons, layered match scoring).

Think **linked list of handlers**: each holds a `next` pointer; all implement `IHandler { handleRequest(); setNext(); }`. The sender doesn't know which handler will act.

ATM example: handlers for ₹1000, ₹500, ₹200, ₹100. Request ₹4000 → the 1000-handler dispenses 3 notes then forwards ₹1000 to the 500-handler, and so on; if the chain ends unsatisfied → "insufficient funds".



ATM chain: ThousandHandler → FiveHundredHandler → …, each forwarding the remainder via next->dispense().

- **OCP:** add a ₹50 handler without touching others.
- **SRP:** each handler owns one denomination/rule.
- Wiring is through interface pointers, not hard-coded concrete links.

> **REAL-WORLD USE**
> Logger levels (Info → Debug → Error) · leave approval (Team Lead ≤2d → Manager ≤5d → Director) · web middleware (auth → rate-limit → parse).

BEHAVIORAL Iterator

▶ Video 29

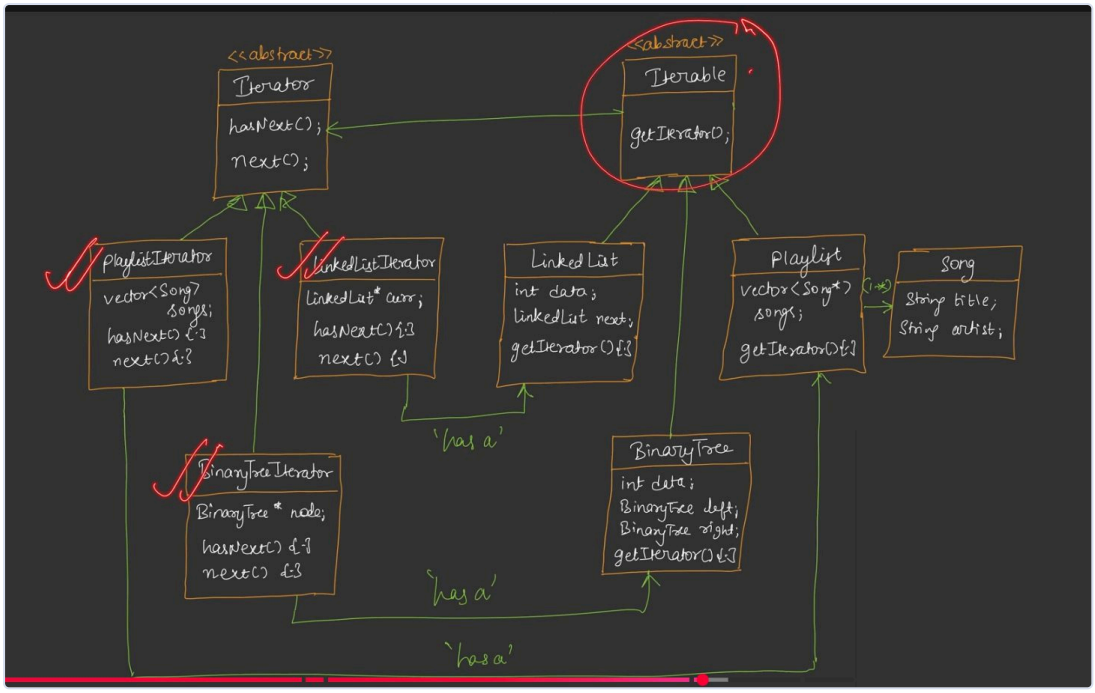
Traverse a collection sequentially without exposing its internal structure.

Recognise it when… a custom collection (playlist, linked list, tree, graph) needs traversal, or you might change its backing structure later.

Problem: a `Playlist` bundling storage logic *and* looping logic violates **SRP**; switching `vector` → `LinkedList` breaks every client loop (**OCP**).

Solution: move traversal state (index, node pointer, stack) into a separate **Iterator**. Clients only use `hasNext()` and `next()`.

- `IIterator<T>`: `hasNext()`, `next()`.
- `IIterable<T>`: `getIterator()` (a factory method returning an iterator; passes `this`).
- Concrete pairs: **LinkedListIterator** (`current` pointer; `hasNext: current != null`) · **BinaryTreeInOrderIterator** (a `stack<Node*>` + `current` turns recursion into step-by-step calls; `hasNext: !stack.empty() || current`) · **PlaylistIterator** (`index`; `hasNext: index < size`).



Iterator: IIterable.getIterator() creates a concrete iterator for Playlist, LinkedList, BinaryTree.

Swapping the structure now only means returning a different concrete iterator from `getIterator()`; client code is untouched. **Extension:** a *bidirectional* iterator adds `hasPrevious()` / `previous()`. C++ STL iterators, Java `Iterator` and C# `IEnumerator` are this pattern.

BEHAVIORAL Mediator

▶ Video 35

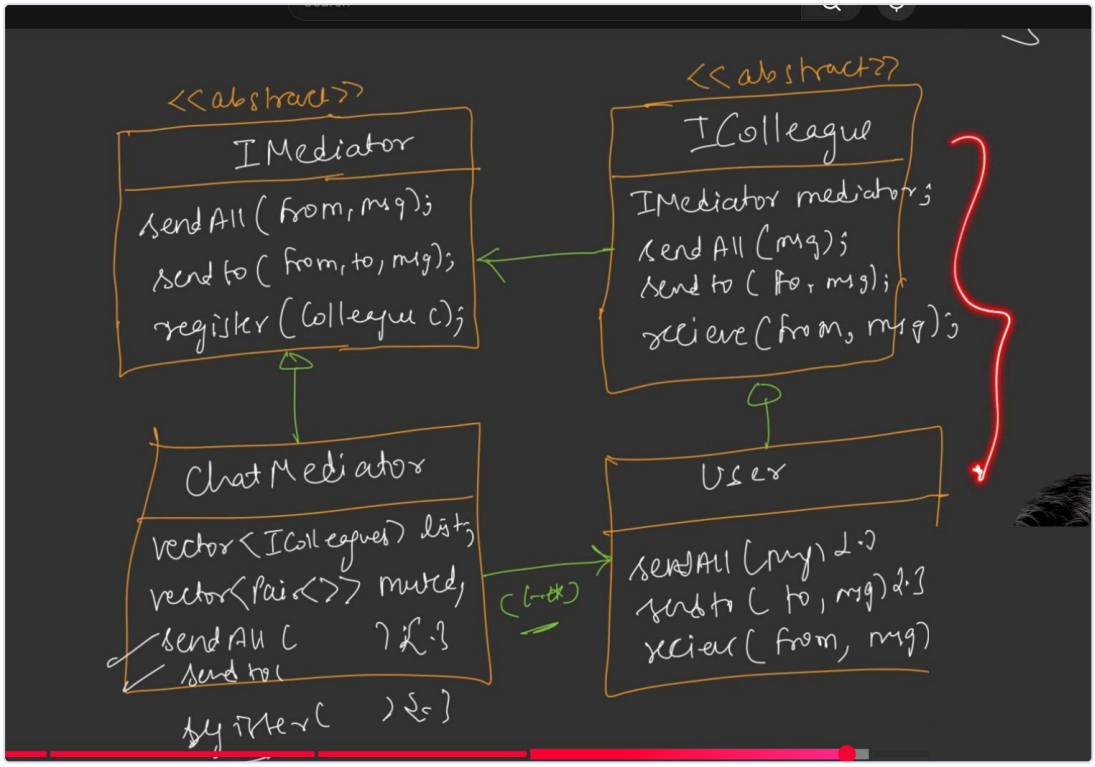
Centralise how a set of objects interact, so they never reference each other directly.

Recognise it when… many peers talk to each other (chat room, multiplayer lobby, air-traffic control) and you'd otherwise have an N×N web of references.

Problem: in a network of *N* objects where each stores *N−1* references you get **O(N²)** tight couplings; adding or changing one object means updating all the others (OCP).

Solution: a single **Mediator** hub. Each colleague holds **one** reference (to the mediator) and delegates: `mediator->sendAll(...)` / `sendPrivate(...)`. The mediator owns the registry and central rules (mute/block lists) and routes to `receive()`.

- `IMediator`: `registerColleague`, `sendAll`, `sendPrivate`. `ChatMediator` keeps the colleague list and blocked pairs.
- `IColleague`: Has-A `IMediator`; `sendAll/sendPrivate/receive`. `User` registers itself on creation.



Mediator: ChatMediator routes messages between Users who know only the mediator.

INTERVIEW TRAP: MEDIATOR VS OBSERVER

Both are one-to-many with registration. **Observer** = *one-way* broadcast of state changes from a subject to passive listeners. **Mediator** = *two-way* peer-to-peer interaction between equal colleagues, routed through a hub that can apply rules.

REAL-WORLD USE

Chat servers (group, private, mute/block) · multiplayer matchmaking and game lobbies · air-traffic control tower.

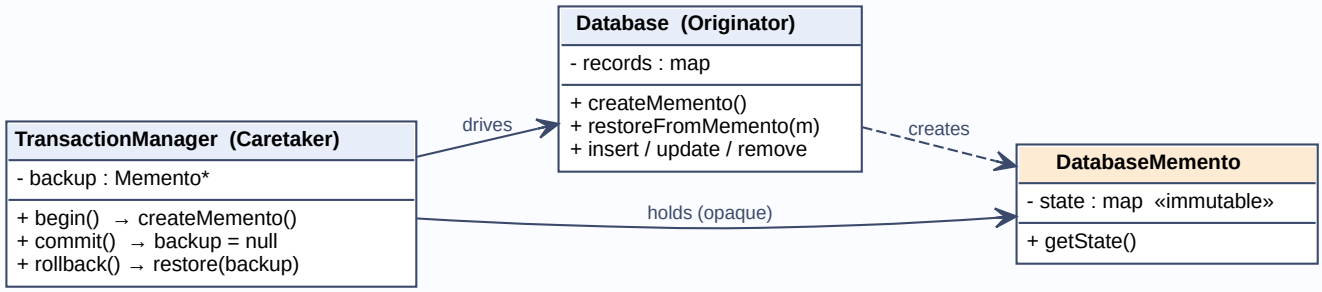
BEHAVIORAL Memento

▶ Video 39

Capture and store an object's state as a snapshot, so you can restore it later without breaking encapsulation.

Recognise it when… "undo", "rollback", "checkpoint", "restore previous version", "transaction commit/rollback".

Actor	Role
Originator (`Database`)	The object whose state changes; creates and restores from mementos.
Memento (`DatabaseMemento`)	Immutable snapshot of the state (e.g. the records map).
Caretaker (`TransactionManager`)	Stores mementos as **opaque** objects; never inspects the contents.



Memento: the caretaker keeps an opaque snapshot made by the originator and hands it back on rollback.

Transaction flow: begin → caretaker calls `db->createMemento()` and stores it as `backup` → CRUD mutates the DB → **commit** deletes the backup (`backup = nullptr`) → **rollback** calls `db->restoreFromMemento(backup)` .

> **REAL-WORLD USE**
> DB COMMIT / ROLLBACK · Git history (commit hashes, roll back) · editors and games with undo/redo buffers.

> **MEMENTO VS COMMAND FOR UNDO**
> Command undoes by running the *inverse operation*; Memento undoes by *restoring a saved state*. Use Memento when the inverse is hard to compute.

BEHAVIORAL Visitor

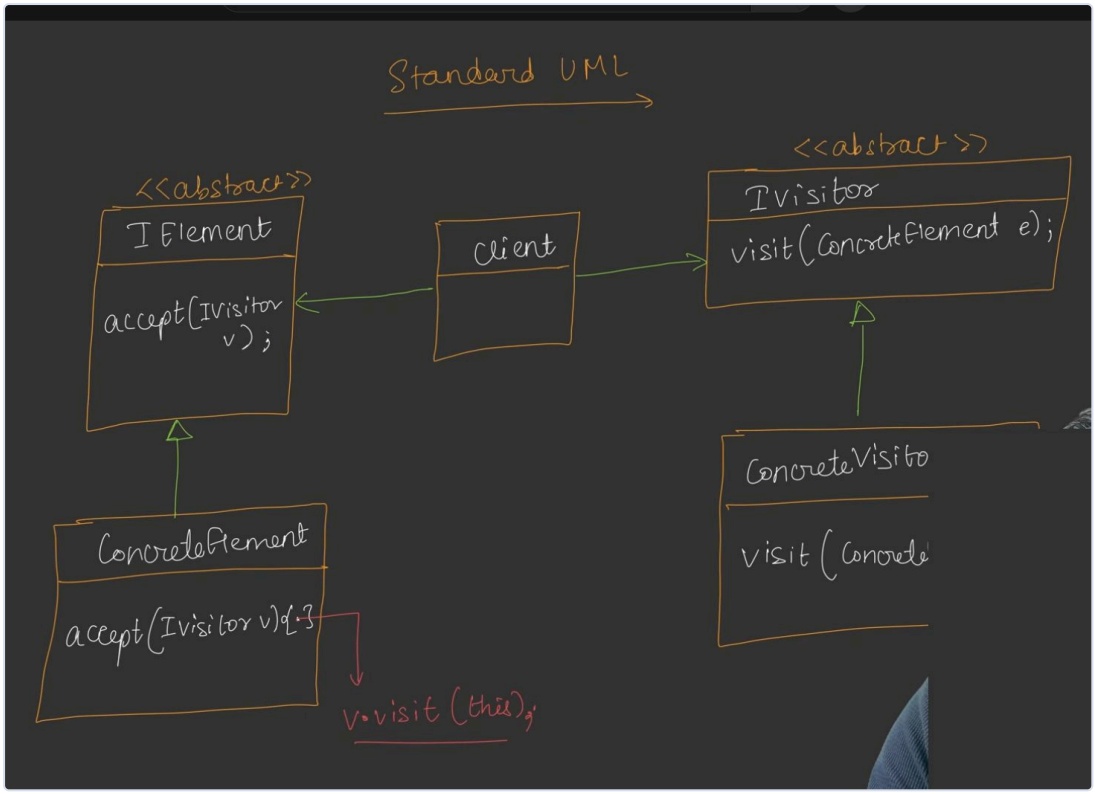
▶ Video 38

Add new operations to an existing class hierarchy without modifying the classes.

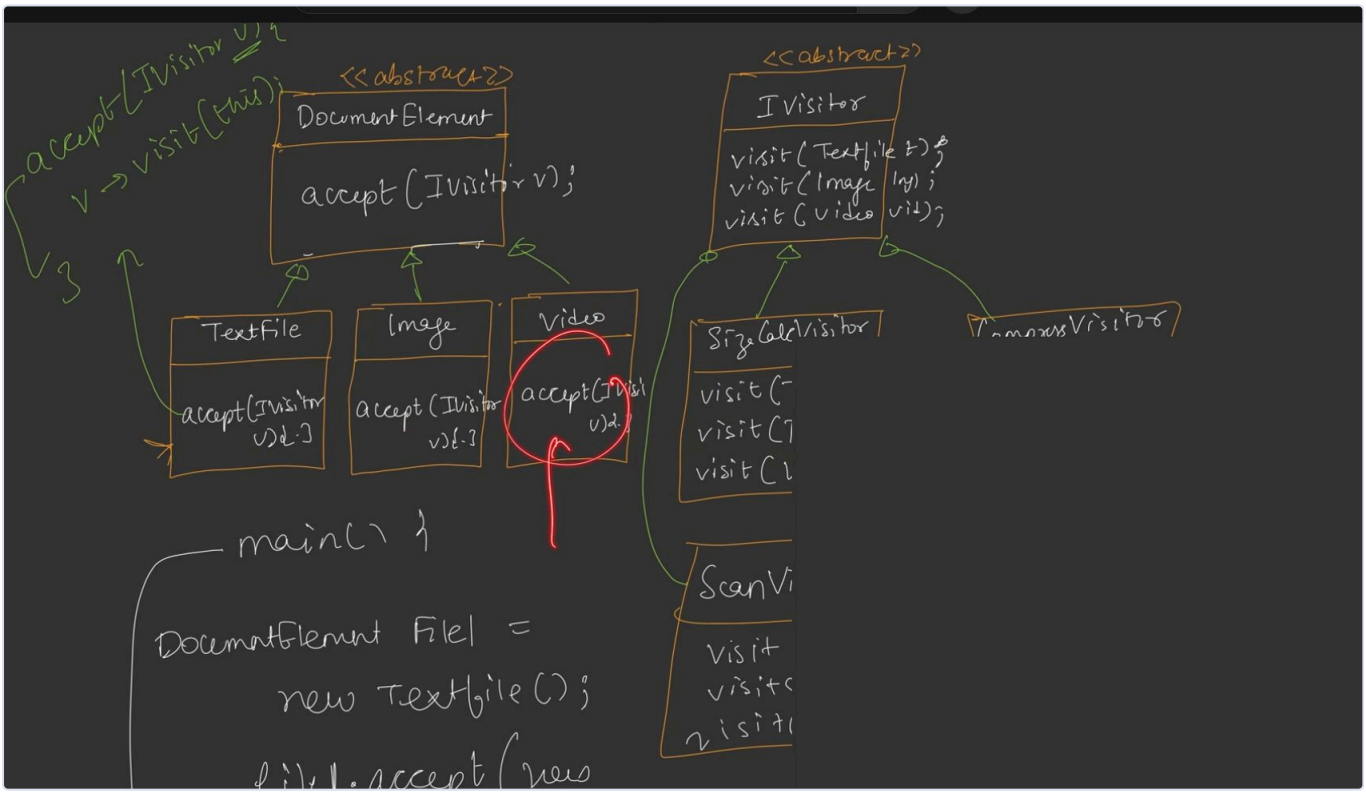
Recognise it when… the set of element types is **stable** (TextFile, ImageFile, VideoFile) but you keep adding **new operations** across all of them (size, compress, virus-scan, export). Compiler ASTs and game-object trees are classic cases.

Without it, every new operation is added into each element class: bloat, and OCP + SRP violations.

- **Element** (`DocumentElement`): `accept(IVisitor* v)` .
- **Concrete elements** implement `accept(v) { v->visit(this); }` .
- **IVisitor**: one overloaded `visit()` per element type (`visit(TextFile*)` , `visit(ImageFile*)` , `visit(VideoFile*)`).
- **Concrete visitors** (`SizeCalculationVisitor` , `CompressionVisitor` , `VirusScanningVisitor`) hold *all* the logic of one operation.



Standard Visitor UML: IElement.accept(IVisitor) ← → IVisitor.visit(ConcreteElement).



File example: each file type's accept() calls v->visit(this), choosing the right overload.

DOUBLE DISPATCH

1st dispatch: `element->accept(visitor)` picks the concrete element's `accept`. **2nd dispatch:** inside it, `v->visit(this)` picks the visitor's overload using the element's concrete type. Two polymorphic decisions: (element type × visitor type).

	Strategy	Visitor
Varies	*How* one behaviour is done	The *number of operations* over a fixed set of elements
Stable part	The operation	The element types
Example	`FlyWithWings` vs `FlyWithJet`	Size / Compress / VirusScan over files

DOWNSIDE TO MENTION

Adding a *new element type* forces a new `visit()` in every visitor, so Visitor only pays off when elements rarely change.

10 Look-alike patterns: choosing between them

Interviewers love "why X and not Y?". The structures look alike; the **intent** differs.

Pair	Left	Right
Strategy vs Factory	**Strategy**: swap *behaviour* at run-time; instances already exist or are injected.	**Factory**: *isolate creation* logic from business logic. They often combine (`PaymentStrategyFactory`).
Strategy vs Bridge	**Strategy**: interchangeable algorithms on a live object.	**Bridge**: structural split so abstraction and implementation grow independently (m+n).
Strategy vs Visitor	**Strategy**: vary *how* one operation works.	**Visitor**: add *many operations* to stable element types.
Strategy vs State	**Strategy**: client chooses the algorithm; strategies don't know each other.	**State**: the object *itself* moves between states; states trigger transitions.
Adapter vs Facade	**Adapter**: translate to an *existing* interface (1 adaptee).	**Facade**: *new simple* interface over many classes.
Observer vs Mediator	**Observer**: one-way broadcast of state change.	**Mediator**: two-way peer routing through a hub with rules.
CoR vs Decorator	**CoR**: handlers can **stop** the chain early or branch on conditions.	**Decorator**: every layer always runs; it *adds* behaviour around a core.
Decorator vs Proxy	**Decorator**: adds features; client composes the layers.	**Proxy**: controls *access* to one object (lazy / auth / remote / retry).
Command vs Memento	**Command**: undo via inverse operation.	**Memento**: undo via saved snapshot.
Simple Factory vs Factory Method	One class with `switch` ; breaks OCP.	Subclass-per-variant creators; OCP-safe.

11 Null Object and anti-patterns

▶ Video 40 of the playlist

IDIOM Null Object

▶ Video 40

Replace `null` with a neutral "do-nothing" object that implements the same interface.

Instead of `if (obj != nullptr)` scattered through client code, give clients an object whose methods are empty or return defaults (`0`, `""`). Polymorphism replaces branching, there are no null-pointer crashes, and it respects **LSP**.

Where it plugs in: Strategy → `NoFly` / `NoTalk` for non-flying/silent robots · Command → `NoCommand` in unmapped remote buttons.

Anti-patterns to avoid (and to call out in reviews)

Anti-pattern	What it is / fix
God Object	One monolith hoarding responsibilities. *Note:* a pure orchestrator/facade that only **delegates** is **not** a god object.
Spaghetti code	No clear entry/exit, cyclic dependencies, tight coupling.
Hard-coding	Magic numbers, strings, credentials in business logic. Use constants, config, injection.
Gold plating / over-engineering	Patterns and abstractions for cases that will never occur. Design for *stated* requirements.
Violating DRY	Copy-pasted logic. Extract helpers/utilities.
Constructor telescoping	Endless overloaded constructors. Fix: **Builder**.
Getter/setter overuse	Auto-generating them for every private field exposes internals and defeats encapsulation. Expose behaviours, not fields.
Premature optimisation	"Make it work, then make it fast."
Inheritance overuse	Deep, fragile hierarchies. Prefer **composition**.

PART 3

Part 3 · Case Studies

Eleven solved LLD problems. Each one is a template you can remix: the same four or five patterns appear again and again.

12 Reading the case studies

Every case follows the same shape so you can compare them quickly: **scope → entities → patterns → flow/algorithm → what the interviewer will challenge**.

Which pattern shows up where

Case	Strategy	Factory	Singleton	Observer	Facade / Orchestrator	Others
Zomato	Payment	Order (Now/Schedule)	Managers		Tomato	
Notification	Channels		Service	Logger + Engine	Service	Decorator
Spotify	Play modes	Device	Managers		MusicPlayerFacade	Adapter
Payment Gateway		Gateway	Factory, Service, Controller		Controller	Template, Proxy, Adapter
Discount	Calculation	Strategy mgr	Managers		CouponManager	CoR, Template
Zepto	Replenish	Product	Managers		Zepto	Bridge
Tinder	Location	Matcher	Several	Targeted	DatingApp	CoR
Splitwise	Split	Split	SplitWise	Group → User	SplitWise	Greedy algo
Tic-Tac-Toe	Rules	Game		Game	Game	
Snake & Ladder	Setup + Rules	Game		Game	Game	
Chess	Matching + Rules	Piece	GameManager		GameManager	Mediator

OBSERVATIONS WORTH SAYING ALOUD

- **Strategy** and **Factory** appear in almost every problem: Strategy for the "many ways to do X" and Factory to build the right one.
- **Singleton managers + a Facade/Orchestrator** form the usual skeleton for the entry point.
- Games use **top-down** modelling; commerce/service apps use **bottom-up**.
- A **Bridge/Adapter** shows up whenever a swappable storage or third-party layer sits underneath.

13 Zomato: food delivery (user flow)

▶ Video 11 of the playlist

Scope

Functional: search restaurants by location → view menu → add to cart → place order (now or scheduled) → choose payment → get notification. **Non-functional:** scalability, concurrency, reliability.

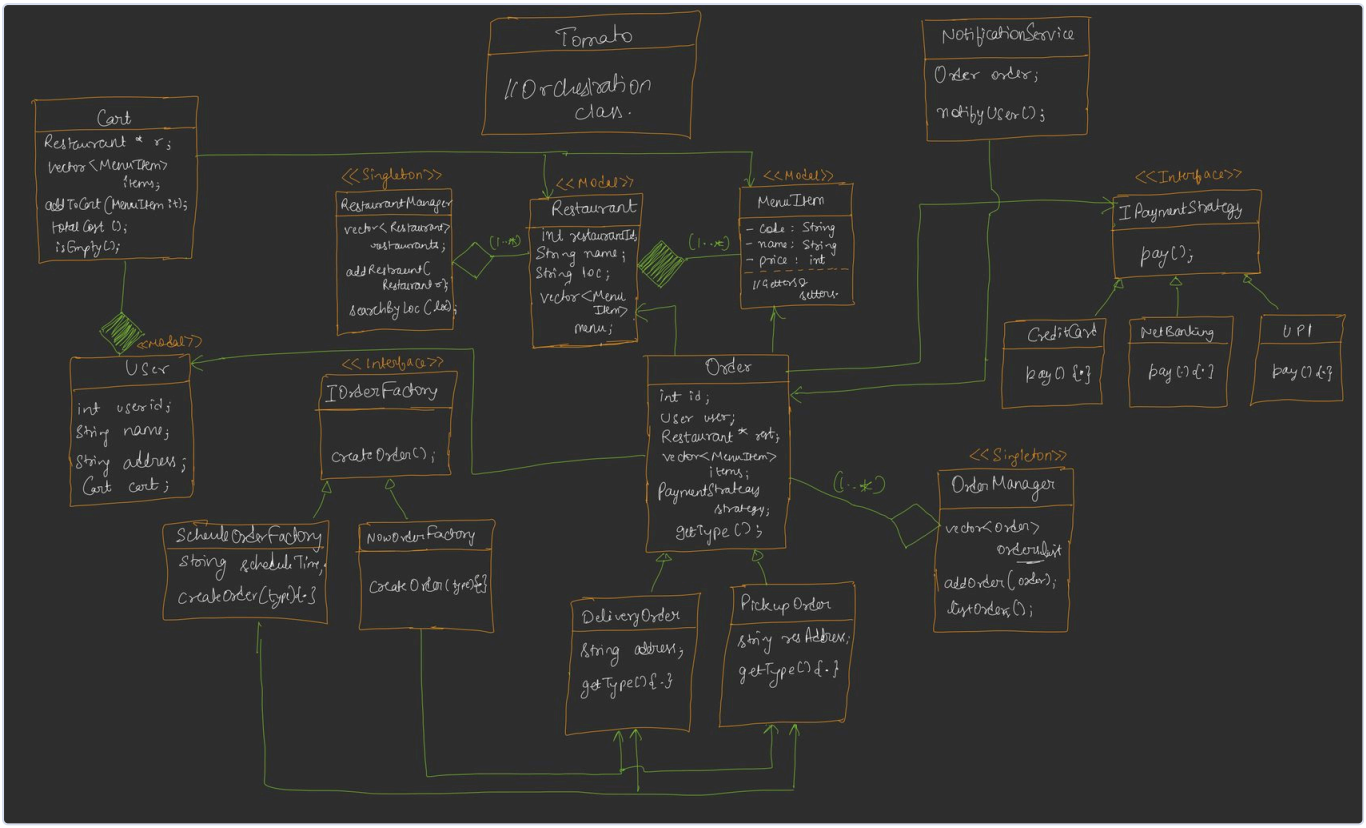
Don't build a production app in an hour: focus on the **user-centric flow**, treat **Payment Gateway** and **Notification** as third-party interfaces, and explicitly **exclude the delivery-agent flow**.

Entities (bottom-up)

Class	Notes
`MenuItem`	code, name, price. **Composition** under Restaurant (cannot exist without it in this scope).
`Restaurant`	id, name, address, `vector<MenuItem*> menu`.
`RestaurantManager`	**Singleton**; `list<Restaurant*>` (**aggregation**: restaurants outlive it); `searchByLocation()`.
`User` / `Cart`	User owns Cart (**composition**, 1:1). Cart: restaurant, items, total; `addToCart / clear / isEmpty / getTotalCost`.
`Order` (abstract)	id, user, restaurant, items, payment strategy, total, scheduledTime; pure virtual `getType()`. Subclasses `DeliveryOrder` (user address) and `PickupOrder` (restaurant address).
`OrderManager`	**Singleton**; `list<Order*>`.

Patterns

Pattern	Where and why
Strategy	`PaymentStrategy::pay(amount)` with `UPIPayment`, `CreditCardPayment`, `NetBankingPayment`. New mode, no change to Order or Tomato.
Factory Method	`OrderFactory` → `NowOrderFactory` (stamps `TimeUtils::getCurrentTime()`) / `ScheduleOrderFactory` (future time); both build Delivery or Pickup by `orderType`.
Singleton	`RestaurantManager`, `OrderManager`: single source of truth.
Facade	`Tomato` orchestrator: `searchRestaurant, addToCart, checkoutNow, payForOrder`.



Zomato ("Tomato") UML: managers, models, order hierarchy, factories and payment strategies.

WHAT THE INTERVIEWER WILL CHALLENGE

`Tomato` knows every entity, factory and manager (**SRP** violation) and walks deep chains like `user->getCart()->getRestaurant()` (**Law of Demeter**).

Answer: an orchestrator gives the client a single clean interface for the interview; in production you'd split it into **controllers (APIs)** and **services** (Spring Boot / Django style).

EASY EXTENSIONS TO OFFER

`PaymentStrategyFactory` (Factory + Strategy) instead of `new UPIPayment` in `main` · `NotificationService` as an interface with Email/SMS/Push · auto-increment IDs via `private static int nextId` in constructors.

14 Notification system

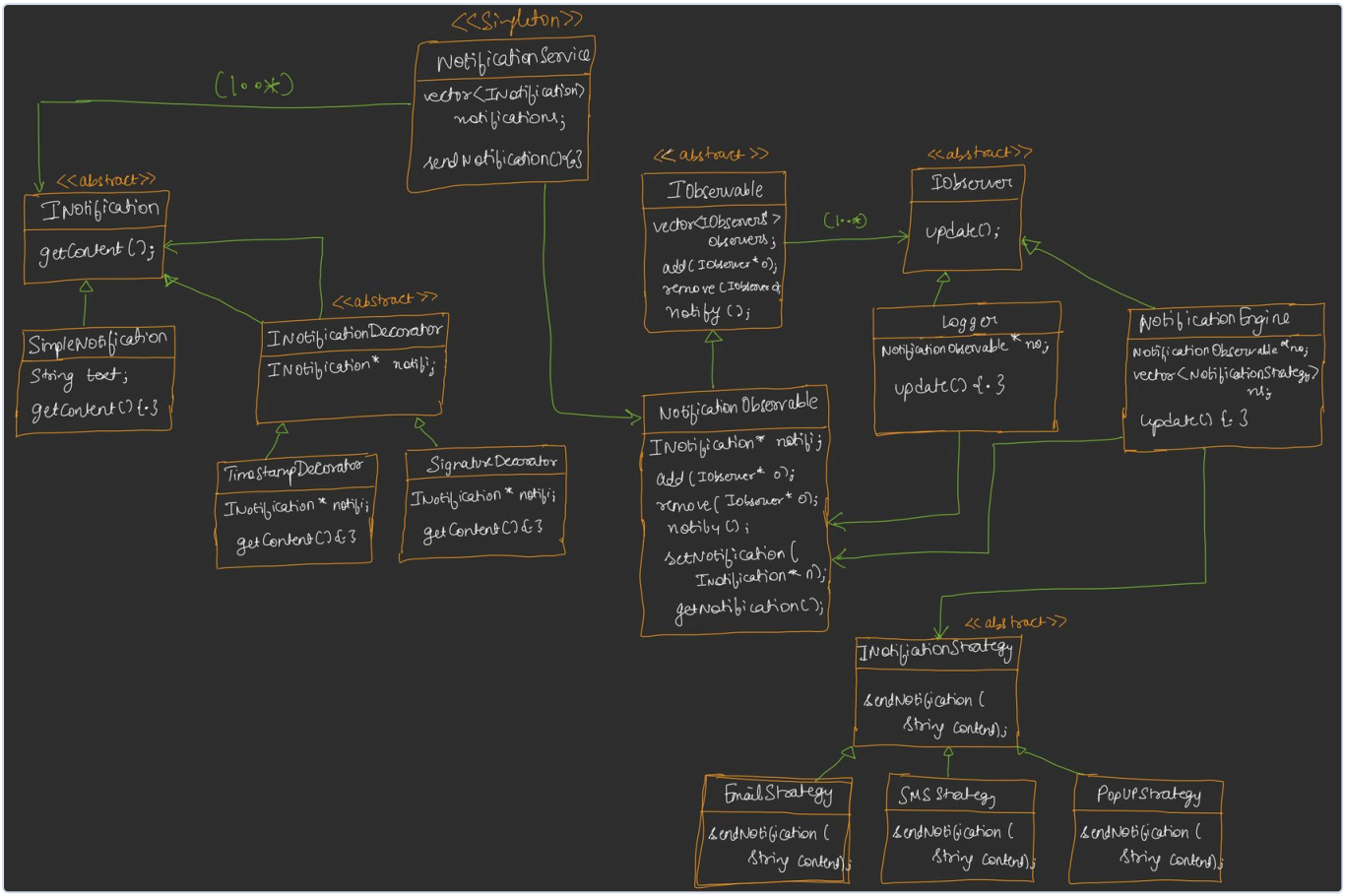
▶ Video 14 of the playlist

Requirements

- **Plug-and-play** library: integrates into any client with minimal changes.
- **Extensible channels** (SMS, Email, Pop-up, WhatsApp) without editing core logic (OCP).
- Notifications can be **decorated at run-time** (timestamp, signature, header/footer).
- Every sent notification is **logged** and kept in **history**.

Design: four patterns, one pipeline

Stage	Pattern and design
1. Build content	**Decorator.** `INotification { getContent() }`; `SimpleTextNotification` (extendable to HTML/rich). `INotificationDecorator` Is-A + Has-A `INotification`; `TimeStampDecorator`, `SignatureDecorator`. A double-decorated call prepends the timestamp, takes the text, appends the signature.
2. Fire event	**Observer.** `NotificationObservable` keeps `vector<IObserver*>`; `setNotification()` automatically triggers `notifyObservers()`. Observers: `Logger` (prints/logs) and `NotificationEngine` (dispatches).
3. Deliver	**Strategy, adapted to 1-to-many.** `INotificationStrategy::sendNotification(content)`: `EmailStrategy` (email id), `SmsStrategy` (phone), `PopupStrategy`. The engine holds a **vector** of strategies and loops: one event fans out across all channels.
4. Entry point	**Singleton + Facade.** `NotificationService::getInstance()` keeps the history list; `sendNotification(n)` = append to history, then `observable->setNotification(n)`.



Notification system UML: Decorator chain, Observable with Logger and Engine observers, channel strategies, Singleton service.

SELF-REGISTRATION: THE PLUG-AND-PLAY TRICK

Manually passing the observable and calling `addObserver()` in client code leaks internal wiring. Instead, each observer's **default constructor** does `NotificationService::getInstance()->getObservable()->addObserver(this)`. The client just creates observers, attaches strategies, wraps decorators and calls `sendNotification()`.

15 Spotify: music player

▶ Video 18 of the playlist

Requirements

Play/pause songs · create playlists and add songs · **play modes: Sequential, Random, Custom Queue** · **output devices**: Bluetooth speaker, wired speaker, headphones. Non-functional: extensible (OCP), loosely coupled, new devices/algorithms without breaking anything.

Entities (bottom-up)

- `Song` (title, artist, filePath). `Playlist` (name, `vector<Song*>`), simple **association** (a song outlives any playlist).
- `PlaylistManager`: **Singleton**, `map<string, Playlist*>` for **O(1)** lookup, **composition** with Playlist.

Two subsystems

Audio devices: Adapter + Factory

Third-party SDKs (`BluetoothSpeakerAPI` , `WiredSpeakerAPI` , `HeadphonesAPI`) have incompatible methods. Target `IAudioOutputDevice::playAudio(Song*)` ; `BluetoothSpeakerAdapter` etc. implement it and wrap their SDK. `DeviceFactory` builds the adapter from a `DeviceType` enum; `DeviceManager` (Singleton) holds the active device. `AudioEngine` (currentSong, isPaused) plays via `activeDevice->playAudio` .

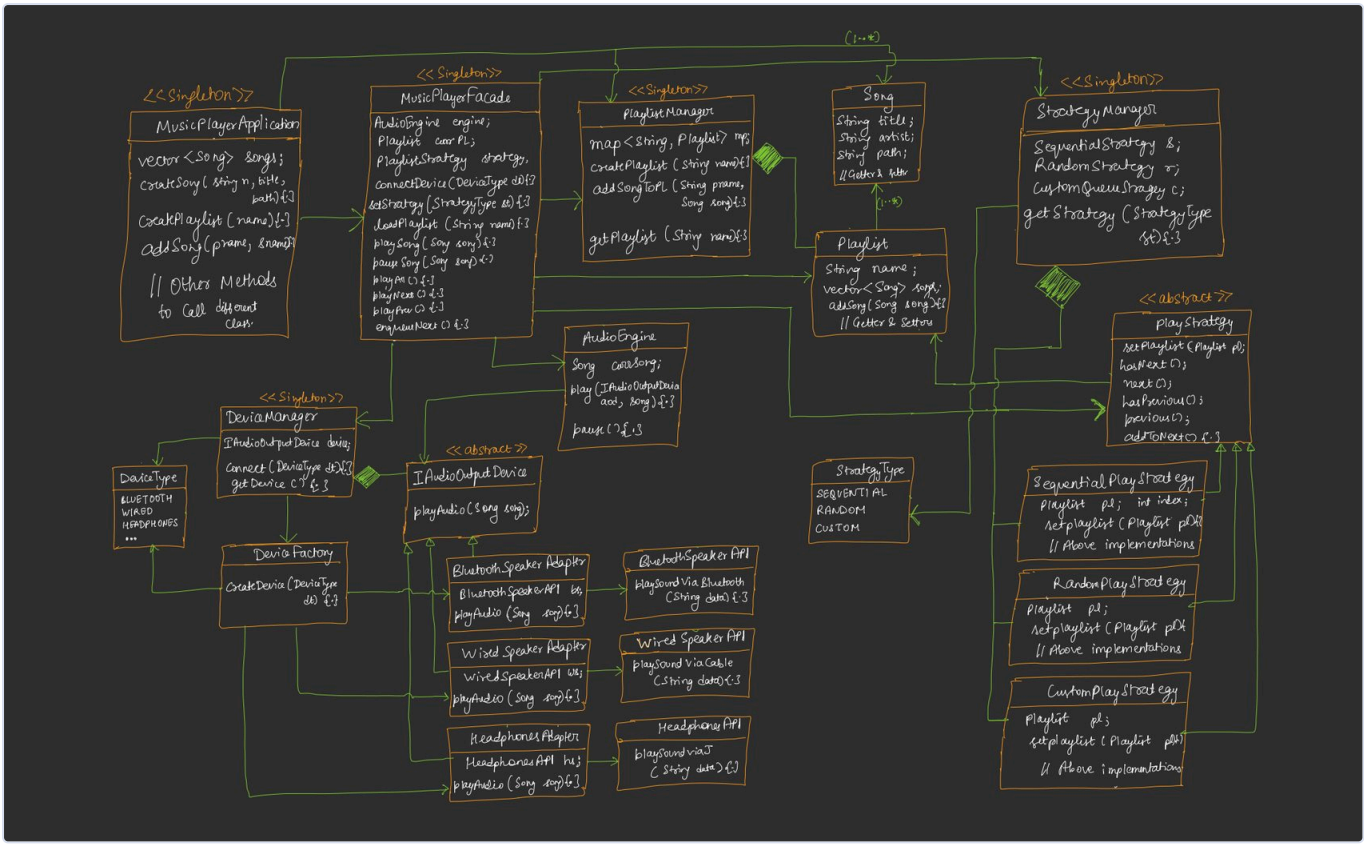
Play modes: Strategy

`IPlayStrategy` : `setPlaylist, next, previous, hasNext, hasPrevious, addToNext` . `Sequential` (`currentIndex`); `Random` (random unplayed from a pool + **stack** history for `previous()`); `CustomQueue` (internal **queue** + stack). `PlayingStrategyManager` (Singleton) returns one by `StrategyType` . `addToNext()` has an **empty default body** so non-queue strategies need no boilerplate (a deliberate ISP trade-off).

Orchestration

`MusicPlayerFacade` (Singleton) fronts `DeviceManager` , `PlayingStrategyManager` , `AudioEngine` , `PlaylistManager` with `connectDevice, setStrategy, loadPlaylist, playSong, pauseSong, playAllTracks, playNextTrack, playPreviousTrack` . `MusicPlayerApp` is the client-facing entry holding the global song library.

`playAllTracks` flow: get current playlist → `while (strategy->hasNext())` → `nextSong` → get active device → `AudioEngine::play(device, nextSong)` .



Spotify UML: Adapter (devices), Factory, Strategy (play modes), Singleton managers and the Facade.

Patterns: **Adapter** **Factory** **Singleton** **Strategy** **Facade**

16 Payment gateway

▶ Video 23 of the playlist

Requirements

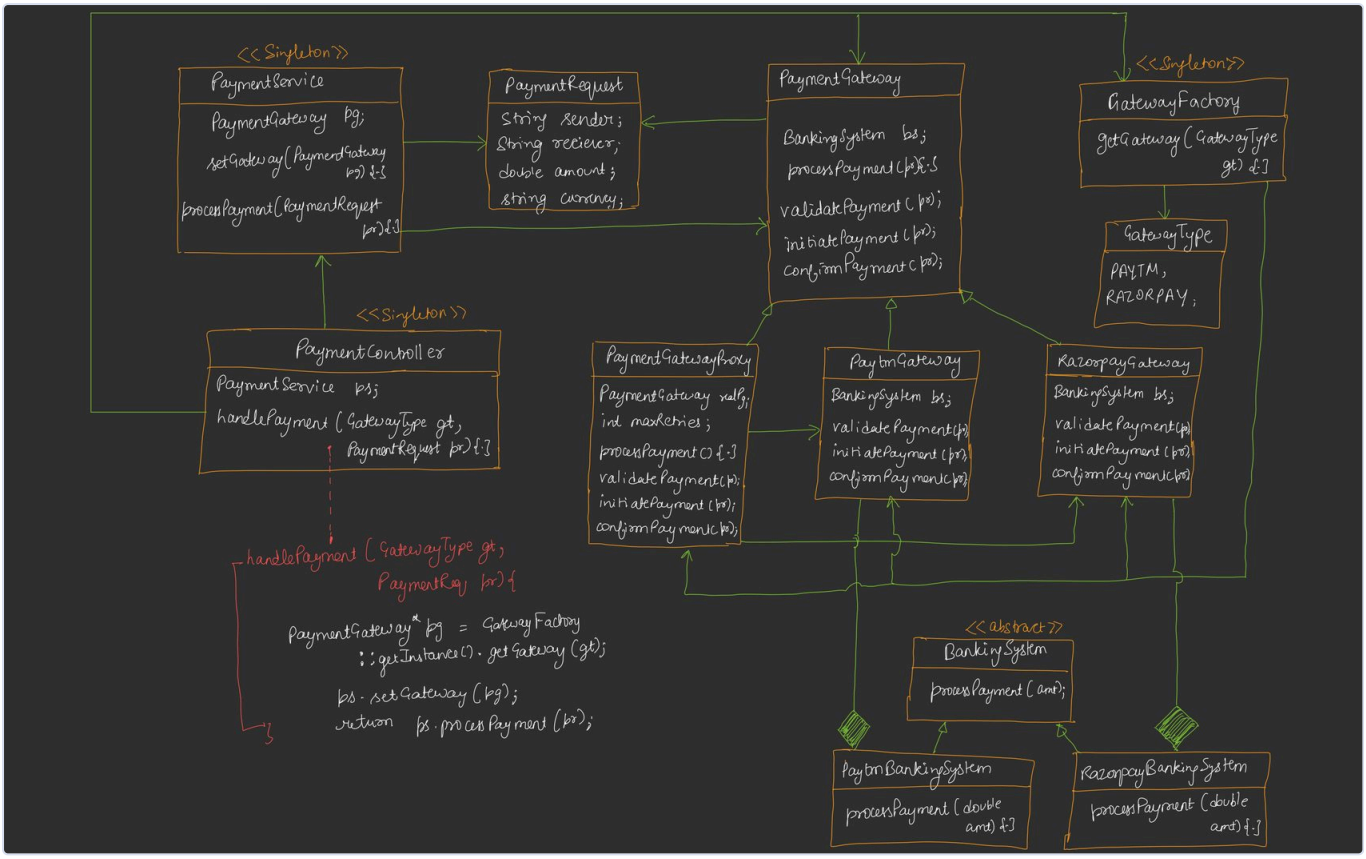
- A **plug-and-play gateway** that apps (Amazon, Zomato) integrate. It is a **mediator to external providers over HTTP**, not a bank ledger.
- Multiple providers (Paytm, Razorpay), new ones addable without edits.
- Mandatory lifecycle: **Validate** → **Initiate** → **Confirm**.
- **Retry** failed transactions up to a threshold, then notify the client app.
- Multi-currency (INR, USD, EUR) carried in a `PaymentRequest` (sender, receiver, amount, currency).

Patterns, each for one job

Concern	Pattern and design
Fixed payment flow	**Template Method.** `PaymentGateway::processPayment()` is non-overridable and calls `validatePayment → initiatePayment → confirmPayment`. `PaytmGateway` / `RazorpayGateway` override the steps with vendor-specific validation (amount > 0, currency).
Talk to bank APIs	**Adapter-style subsystem.** `IBankingSystem::processPayment(amount)` with `PaytmBankingSystem`, `RazorpayBankingSystem`. **Composition** (filled diamond): gateway owns its banking object.
Retry logic	**Proxy.** `PaymentGatewayProxy` Is-A `PaymentGateway`, Has-A `realGateway`; loops up to `maxRetries` (e.g. 3), stops on first `true`, throws if all fail. Keeps retries **out of** gateways and the client (SRP).
Create gateways	**Factory (Singleton).** `GatewayFactory::getGateway(GatewayType)` builds the concrete gateway and **wraps it in the proxy** before returning.
Entry point	**Facade, layered.** `PaymentController::handlePayment(type, request)` (the only thing clients call) → `PaymentService` (holds the active gateway via `setGateway()`, delegates `processPayment()`). Both Singletons.

End-to-end flow

1. Client passes `GatewayType` + `PaymentRequest` to the controller.
2. `GatewayFactory` creates e.g. `PaytmGateway`, wraps it in `PaymentGatewayProxy`.
3. Controller injects the proxy into `PaymentService` and calls `processPayment()`.
4. Proxy runs the **retry loop**, calling `realGateway->processPayment()`.
5. Template runs **validate** → **initiate** → **confirm**; `initiatePayment()` calls `IBankingSystem::processPayment()`.
6. Success/failure returns through the proxy to the client.



Payment gateway UML: Controller, Service, Factory, Template-method gateways, Proxy, and banking systems.

EXTENSIONS

Exponential backoff inside the proxy to reduce load on failing bank servers · **recurring/subscription** payments (Netflix-style scheduled charges) by extending the template flow.

17 Discount / coupon system

▶ Video 24 of the playlist

Requirements

- Plug-and-play library for e-commerce / food / grocery apps.
- **Card-level** discounts (whole cart) and **product-level** discounts (a category, e.g. Electronics).
- Coupon types: Seasonal, Loyalty, Banking, Bulk purchase.
- Calculations: **Flat**, **Percentage**, **Percentage with cap** (60% off up to ₹120).
- **Stacking** and **exclusivity**: combinable coupons apply in sequence; a non-combinable coupon blocks further ones.
- **Thread-safe**: limited coupons for the first *N* users, using a mutex.

Entities

`Product` (name, category, price) · `CartItem` (Product + quantity; `getItemTotal()`; **fully hidden**: clients call `Cart::addProduct(product, qty)` and never touch CartItem) · `Cart` (items, `isLoyaltyMember`, `paymentBank`, `originalTotal`, `currentTotal`, `applyDiscount(amount)`).

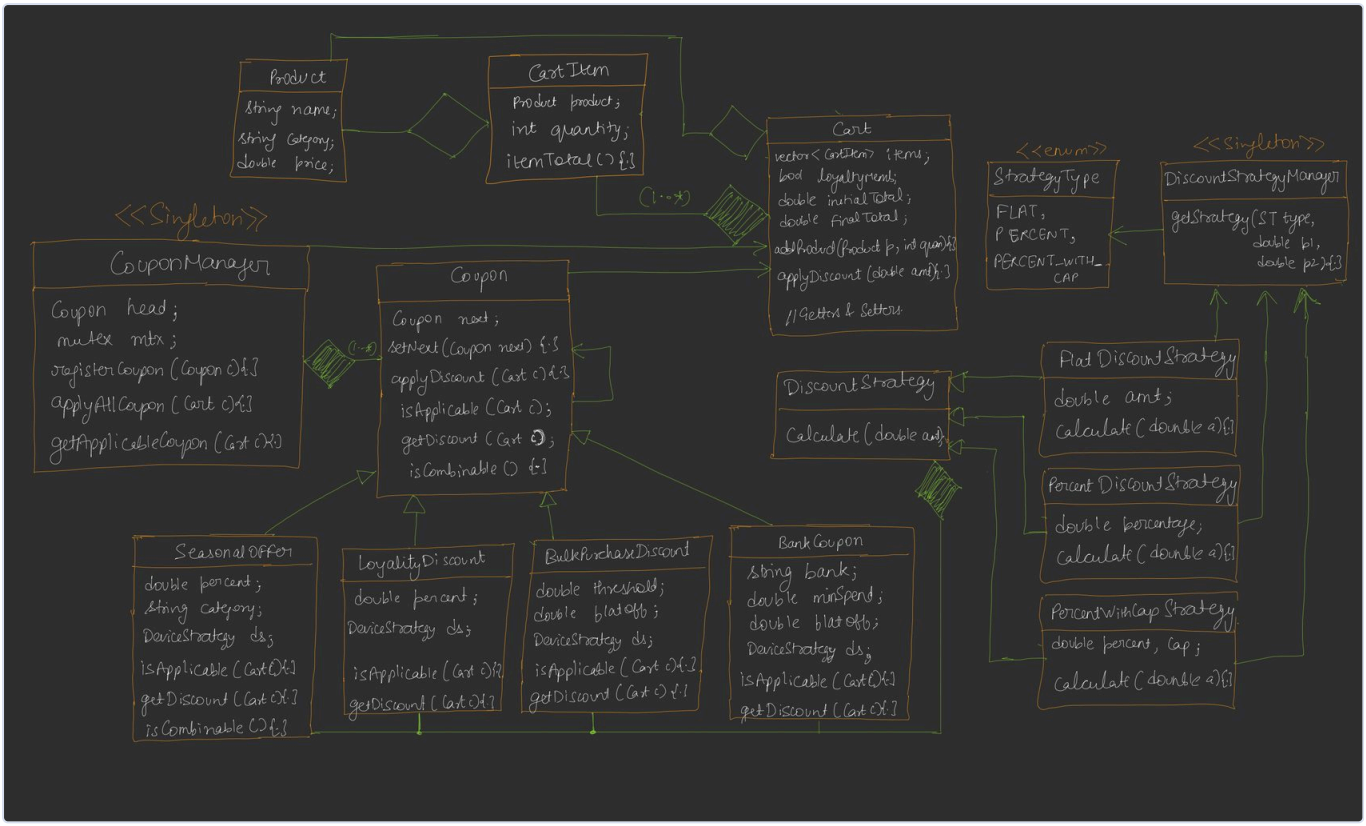
Patterns

Pattern	Role
Strategy	`IDiscountStrategy::calculate(base)`: `FlatDiscountStrategy`, `PercentageDiscountStrategy`, `PercentageWithCapDiscountStrategy` (`min(pct, cap)`). Created by `DiscountStrategyManager` (Factory + Singleton, enum-switch).
Chain of Responsibility	`Coupon` base handler has `next` (`setNext`) and abstract `isApplicable(cart)`, `isCombinable()`, `getDiscount(cart)`.
Template Method	Inside `Coupon::applyDiscount(cart)` fixed pipeline: `if isApplicable` → `getDiscount` (uses its strategy) → `cart.applyDiscount` → `if isCombinable && next` → `next->applyDiscount(cart)`.
Singleton	`CouponManager`, `DiscountStrategyManager`.

Concrete coupons

Coupon	Level	Rule
`SeasonalOfferCoupon`	Product	e.g. 10% off Clothing: scans cart items for the category and discounts only matching subtotals.
`LoyaltyDiscountCoupon`	Cart	Applies if `cart.isLoyaltyMember()`.
`BulkPurchaseCoupon`	Cart	₹100 off when `originalTotal ≥ 1000`.
`BankingCoupon`	Card	Min spend + matching bank (15% up to ₹500 on ABC Bank).

`CouponManager` (Facade) holds the chain `head` and a `std::mutex`: `registerCoupon()` appends to the tail (locked) · `getApplicableCoupons(cart)` walks the chain · `applyAll(cart)` runs `head->applyDiscount(cart)` under lock and returns the final total.



Discount UML: Coupon chain with Strategy-based calculation, Cart/CartItem model and CouponManager.

WHY COR AND NOT DECORATOR?

Both can stack discounts. CoR is the better fit because the chain must **stop** when it hits a non-combinable coupon (`isCombinable() == false`), which flows naturally in a linked chain. A decorator would always run every layer.

Local OCP break: `DiscountStrategyManager` uses `if/switch` on the type, so you break OCP in one small place to shield all business logic from instantiation details.

18 Zepto: quick commerce

▶ Video 26 of the playlist

The twist vs. normal e-commerce

10-minute delivery relies on **dark stores** (local mini-warehouses). The nearest store may lack some items, so the system must **scan nearby stores, fulfil from several, and split one order across multiple delivery partners**. Focus on **inventory management and order fulfilment**, not profiles or payments.

Requirements

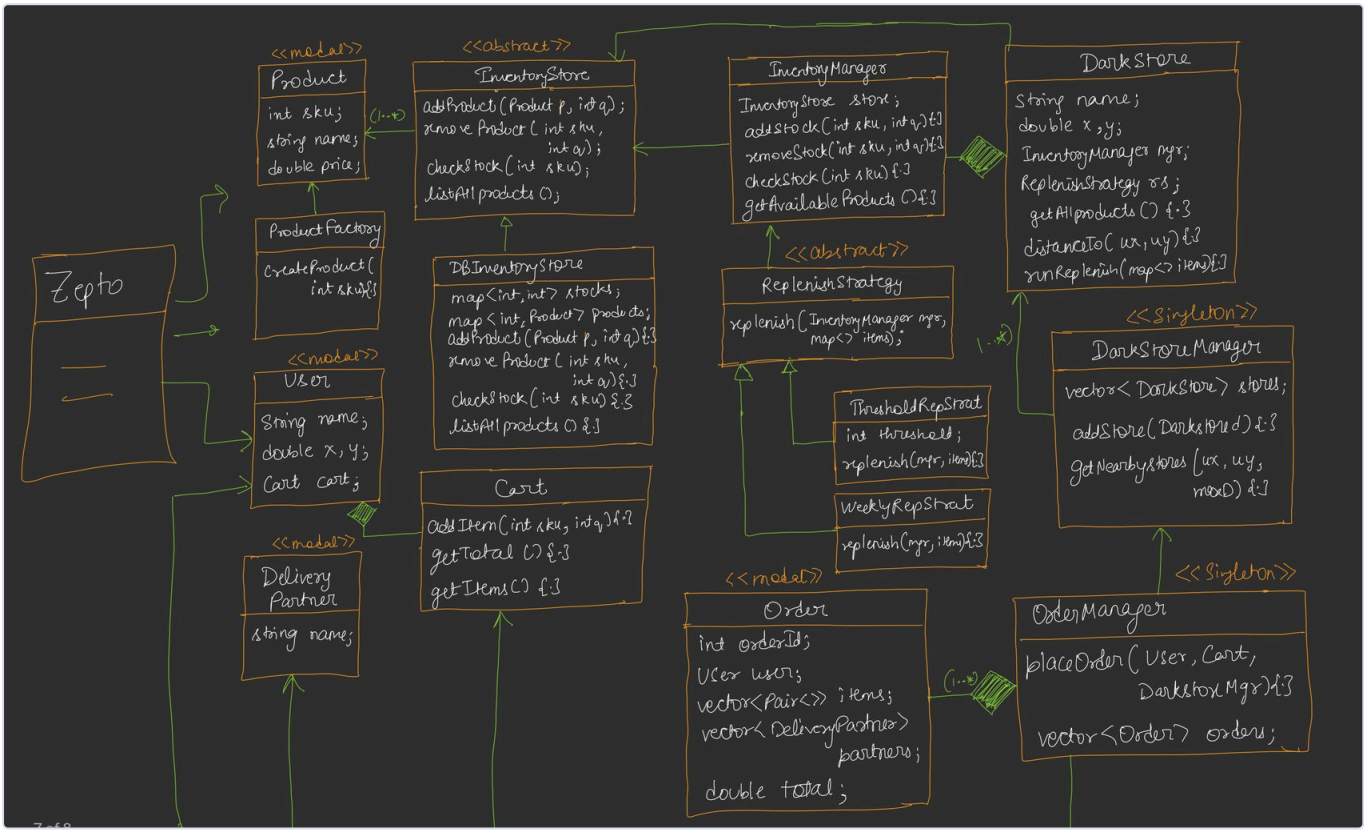
1. Add/remove/update/check stock across categories.
2. **Pluggable replenishment** (threshold-based, weekly).
3. **Pluggable inventory storage** (DB store, file store).
4. Show only products within a radius (e.g. 5 km).
5. Split orders across stores with a delivery partner per store.

Entities and patterns

Element	Design
`Product` + `ProductFactory`	SKU, name, price; the factory creates a product from SKU.
`InventoryStore` (abstract)	`addProduct, removeProduct, checkStock, listAvailableProducts`. `DBInventoryStore` uses **two hash maps**: `SKU → quantity` and `SKU → Product*` for **O(1)** everything (vs O(N) list scans during frequent restocking). Persists via a `Persistence` layer (`SqlPersistence` , `MongoPersistence`).
`InventoryManager`	Has-A `InventoryStore` , delegates. **Not a Singleton** (see below). With the store this is a **Bridge** (abstraction × storage implementation).
`DarkStore`	name, (x,y), `InventoryManager*` (**composition**), `ReplenishStrategy*` ; `distanceTo(ux,uy)` (Euclidean) and delegating methods `addStock / checkStock` (Law of Demeter).
`ReplenishStrategy`	**Strategy.** `ThresholdReplenishStrategy` (restock when below a threshold), `WeeklyReplenishStrategy` .
`DarkStoreManager`	**Singleton**; `vector<DarkStore*>` ; `getNearbyDarkStores(ux,uy,maxDist)` returns stores ≤ 5 km sorted by distance.
`User` / `Cart` / `Order` / `OrderManager`	User owns Cart (composition); Order has items, total and `vector<DeliveryPartner*>` ; `OrderManager` is a Singleton and orchestrates placement. `Zepto` is the facade.

Order-splitting algorithm: `placeOrder(User*, Cart*)`

```
1. stores = DarkStoreManager.getNearbyDarkStores(user, 5km)      // sorted by proximity
2. items  = cart items and quantities
3. if nearest store can fulfil 100%  →  deduct, assign 1 DeliveryPartner, done
4. else for each store in order:
       take whatever it has of the still-missing items, update its stock,
       record the remaining deficit,
       assign a NEW DeliveryPartner for this contributing store
5. recompute total on what was actually fulfilled; log unfulfilled items; persist order
```



Zepto UML: DarkStore (composition with InventoryManager), strategies, managers, order and delivery partner.

> **TRAP: WHY IS INVENTORYMANAGER NOT A SINGLETON?**
> Every dark store owns its **own** inventory. A singleton would force 50 stores to share one manager and break localised stock. **Singleton only where there is truly one global instance** (DarkStoreManager, OrderManager).

19 Tinder: dating app

▶ Video 27 of the playlist

Requirements

- Swipe LEFT (pass) / RIGHT (like).
- Profile (name, age, gender, bio, photos, interests) and **preferences** (age range, max distance, genders, hobbies).
- **Location-based discovery** (e.g. 5 km) with **pluggable strategies**.
- **Multi-factor matching score** (location, interests, basic details).
- Mutual right-swipe auto-creates a **chat room**; **notifications** for matches and messages.

> **WHY TOP-DOWN HERE**
> ~80-90% of LLD problems suit bottom-up, but Tinder has many small objects (UserProfile, Preferences, Interest, Gender, Location, SwipeHistory, ChatRoom, Message). Starting from `User` and `DatingApp` avoids fragmentation and shows how the small pieces attach.

Patterns

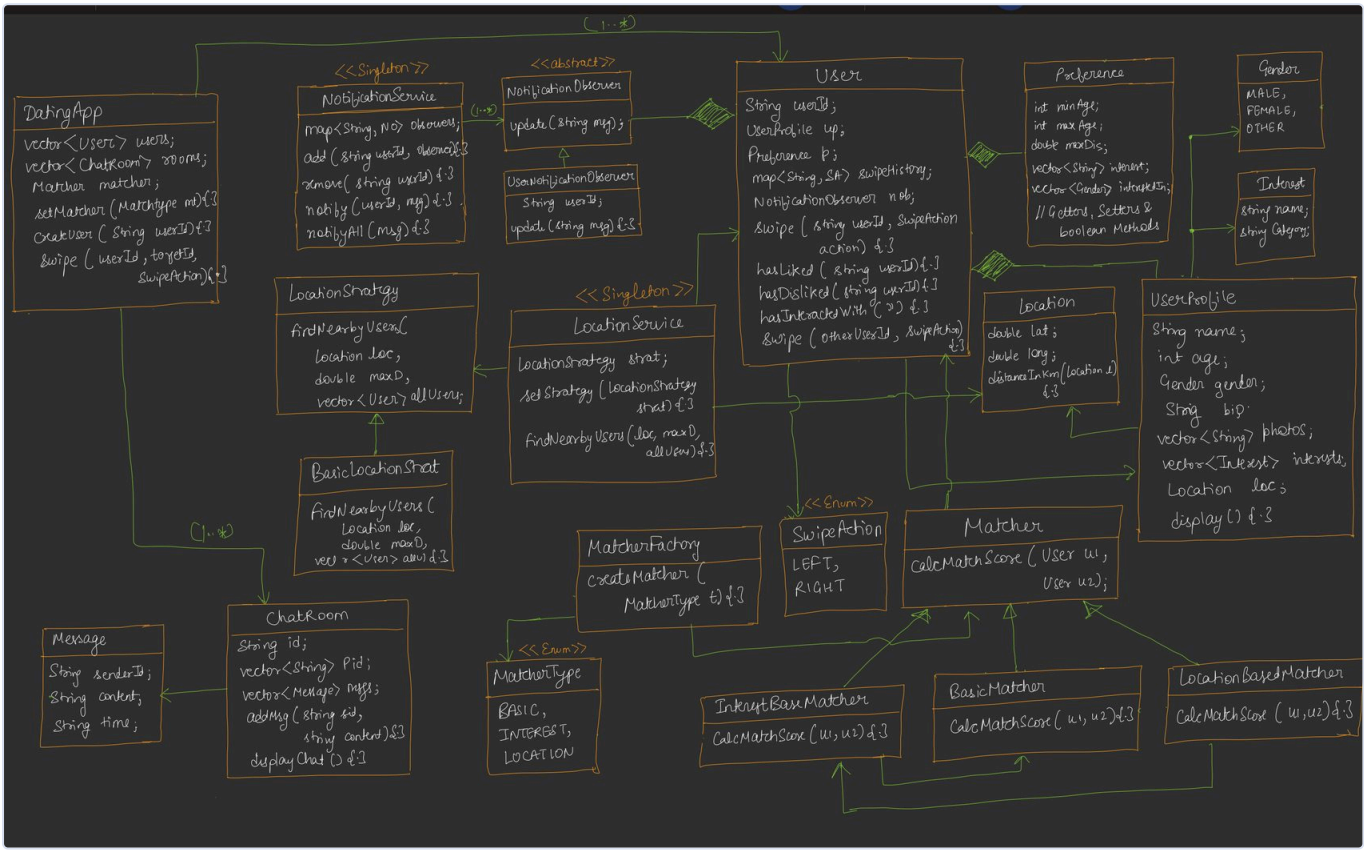
Pattern	Design
Observer (targeted)	`NotificationService` keeps `map<userId, Observer*>` and exposes `notifyUser(id, msg)` plus `notifyAll()`. The `User` constructor **self-registers** its observer.
Chain of Responsibility	Matchers `LocationBased → InterestBased → BasicDetails`. Each calls the next layer; if a lower layer vetoes (gender/age mismatch) it **short-circuits and returns 0.0**; otherwise bonuses accumulate (shared-interest overlap, proximity via Haversine) into a composite score.
Strategy	`LocationStrategy::findNearbyUsers(location, maxDist, allUsers)`; `BasicLocationStrategy` filters by spherical distance. Blacklist/activity-based variants plug in later.
Factory Method	`MatchFactory` builds a matcher chain from `MatcherType` (LOCATION / INTEREST / BASIC).
Singleton	NotificationService, LocationService, MatchFactory, DatingApp.

Models

`Location` (lat, lng; `distanceInKilometers()` via **Haversine**, R ≈ 6371 km) · `UserProfile` · `UserPreferences` · `User` (composition with profile and preferences; `map<string, SwipeAction>` history; `displayProfile()` delegates for LoD) · `Message` (sender, content, timestamp) · `ChatRoom` (id `"userId1_userId2"`, participants, messages).

Swipe & match flow

```
A.swipe(B, RIGHT)
  → record in A's swipe map
  → if action != RIGHT: stop
  → if B already swiped RIGHT on A:  MATCH
        1. chatRoomId = "A_B"
        2. create ChatRoom, register in DatingApp
        3. notifyUser(A, "New match with B!") ; notifyUser(B, "New match with A!")
  → else: wait for B's later swipe
```



Tinder UML: User with profile/preferences, matcher chain, location strategy, notification observer and chat models.

TRAPS

CoR vs Strategy for matching: Strategy picks *one* algorithm; matching is **cumulative** (veto + interests + proximity), which is exactly a chain.

Haversine vs Euclidean: Euclidean ($\sqrt{(\Delta x^2+\Delta y^2)}$) ignores Earth's curvature; Haversine gives true kilometres.

Law of Demeter: call `user->displayProfile()`, not `user->getProfile()->display()`.

20 Splitwise: expense sharing

▶ Video 31 of the playlist

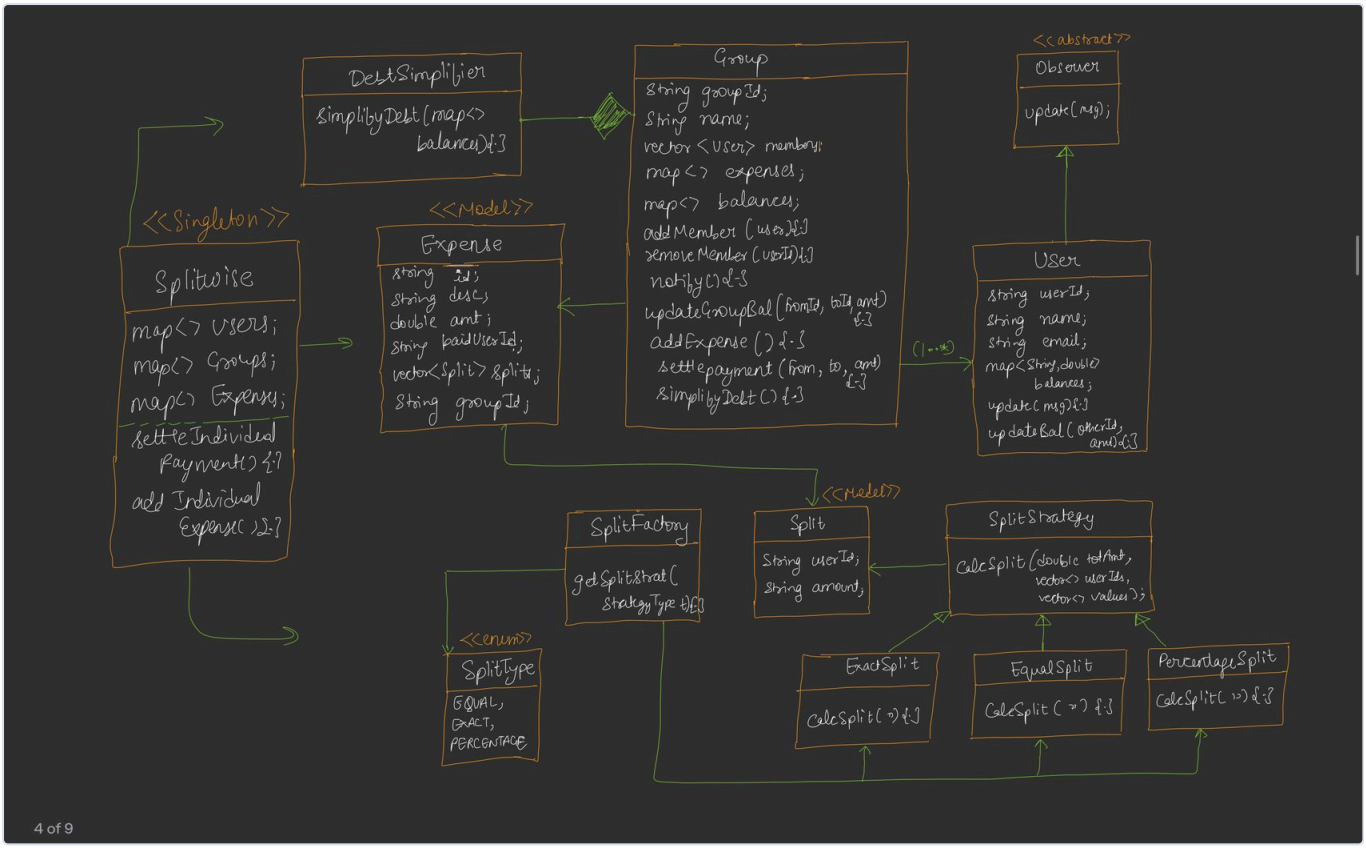
Requirements

- Group **and** 1-on-1 expenses. Create/join/leave groups.
- **Hard rule:** a user **cannot leave a group until all balances are settled**.
- Splits: **Equal, Exact, Percentage**.
- Notify members when an expense is added or settled.
- **Debt simplification** with a greedy algorithm.

Entities and patterns

Element	Design
`User` **concrete observer**	id, name, email, `map<otherUserId, double> balances` (+ve = they owe me, −ve = I owe them); `update(msg)` .
`Split` / `Expense`	`Split` = (userId, amount): a small value class instead of a raw `std::pair` (readability, no confusing `.first/.second`). `Expense` : id, description, total, paidBy, `vector<Split*>` , groupId (empty for 1-on-1).
`Group` **concrete observable**	members (observers), `map<id, Expense*>` history, and `map<string, map<string,double>> groupBalances` (2-D, O(1) pairwise lookup). Methods: add/removeMember (checks `canUserLeaveGroup`), `addExpense, settlePayment, notifyMembers, simplifyGroupDebts` .
Observer	`IObserver::update(msg)` ; `Group` calls `user->update()` on add/settle.
Strategy + Factory	`SplitStrategy::calculateSplit(total, userIds, values)` : Equal (`total/n`), Exact (given amounts), Percentage (`total*pct/100`). `SplitFactory` picks by `SplitType` .
Facade + Singleton	`SplitWise` holds global user/group/expense maps, delegates group requests, handles 1-on-1 directly.

Leave rule: `canUserLeaveGroup(id)` scans that member's row in `groupBalances` ; if any |balance| > 0.01 (float tolerance) it rejects.



Splitwise UML: SplitWise facade, Group (observable) with balances, User (observer), Expense/Split, SplitStrategy + factory.

Greedy debt simplification (≤ N−1 transfers)

A raw log can form a cycle (A owes B 200, B owes C 200, C owes A 200) needing many payments. Reduce it:

1. Compute each member's **net** = (owed to them) − (they owe).
2. Split into **debtors** (net < 0) and **creditors** (net > 0).
3. Sort both by absolute value, **descending**.
4. Repeatedly match the top debtor with the top creditor: `transfer = min(|debtor|, creditor)` ; update both; drop anyone at 0.
5. Overwrite `groupBalances` with the simplified transfers.

> **RESULT**
> Transactions drop from O(N²) to **at most N−1**.

21 Tic-Tac-Toe

▶ Video 33 of the playlist

Requirements

n × n board (not hard-coded 3×3) · **extensible rules** (disable diagonals, 4-in-a-row…) · in-app **notifications** for moves, wins and draws.

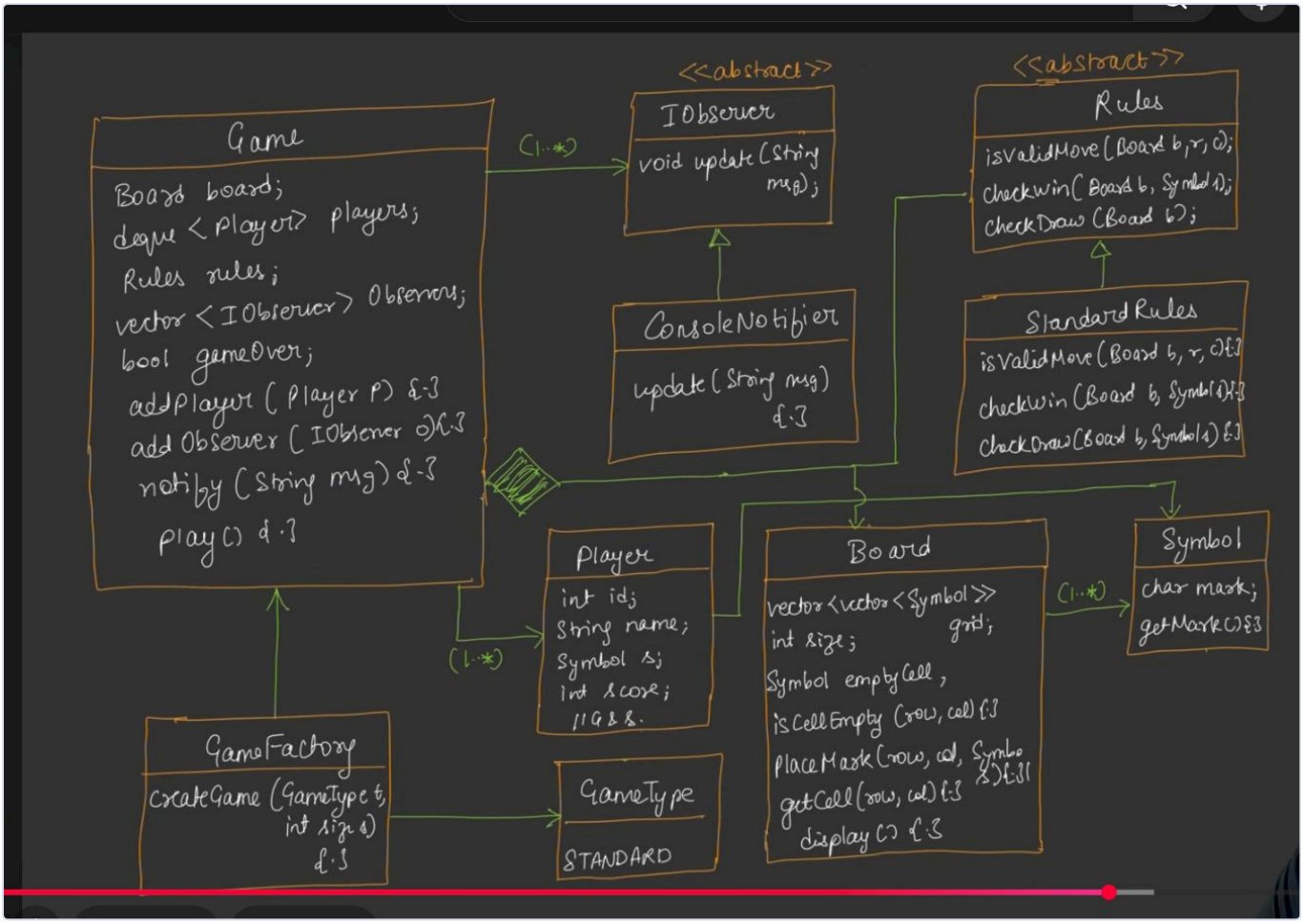
> **TOP-DOWN FOR BOARD GAMES**
> Start with `Game` and work down to Board, Player, Rules: it clarifies ownership and orchestration earlier (same for Snake & Ladder and Chess).

Entities

- `Symbol` (`char mark`) · `Player` (id, name, Symbol, score).
- `Board` : `vector<vector<Symbol*>> grid` , size, a default `emptySymbol '_'` (avoids null pointers). Only grid operations (`isCellEmpty, placeMark, getCell, display`); **knows nothing about winning** (SRP).
- `Game` : **orchestrator + observable**; holds Board, `deque<Player*>` , rules, observers, `isGameOver` ; runs `play()` .

Patterns

Pattern	Design
Strategy	`TicTacToeRules` : `isValidMove, checkWinCondition, checkDrawCondition` . `StandardTicTacToeRules` scans rows, columns, both diagonals. Swap in `CustomRules` without touching Game.
Observer	`Game` is observable; `IObserver::update(msg)` ; `ConsoleNotifier` prints.
Factory	`TicTacToeGameFactory` builds a configured Game from `GameType` + size.



Tic-Tac-Toe UML: Game orchestrator, Board/Symbol, Rules strategy, Observer notifier and GameFactory.

Round-robin turns with a `deque`

Take `players.front()`, let them play, then `pop_front()` and `push_back()`. Works for 2 or *n* players with no index arithmetic.

```
play(): require players >= 2
  while (!isGameOver):
     p = players.front()
     read move; if !rules.isValidMove → retry
     board.placeMark
     if rules.checkWin  → notify win, end
     if rules.checkDraw → notify draw, end
     rotate deque
```

22 Snake & Ladder

▶ Video 34 of the playlist

Requirements

Scalable board (default 100 cells) · **pluggable setup strategies**: Standard (preset), Random (by difficulty), Custom (counts or exact positions) · extensible rules (movement, bounds, win) · console notifications for dice rolls, snakes, ladders, wins.

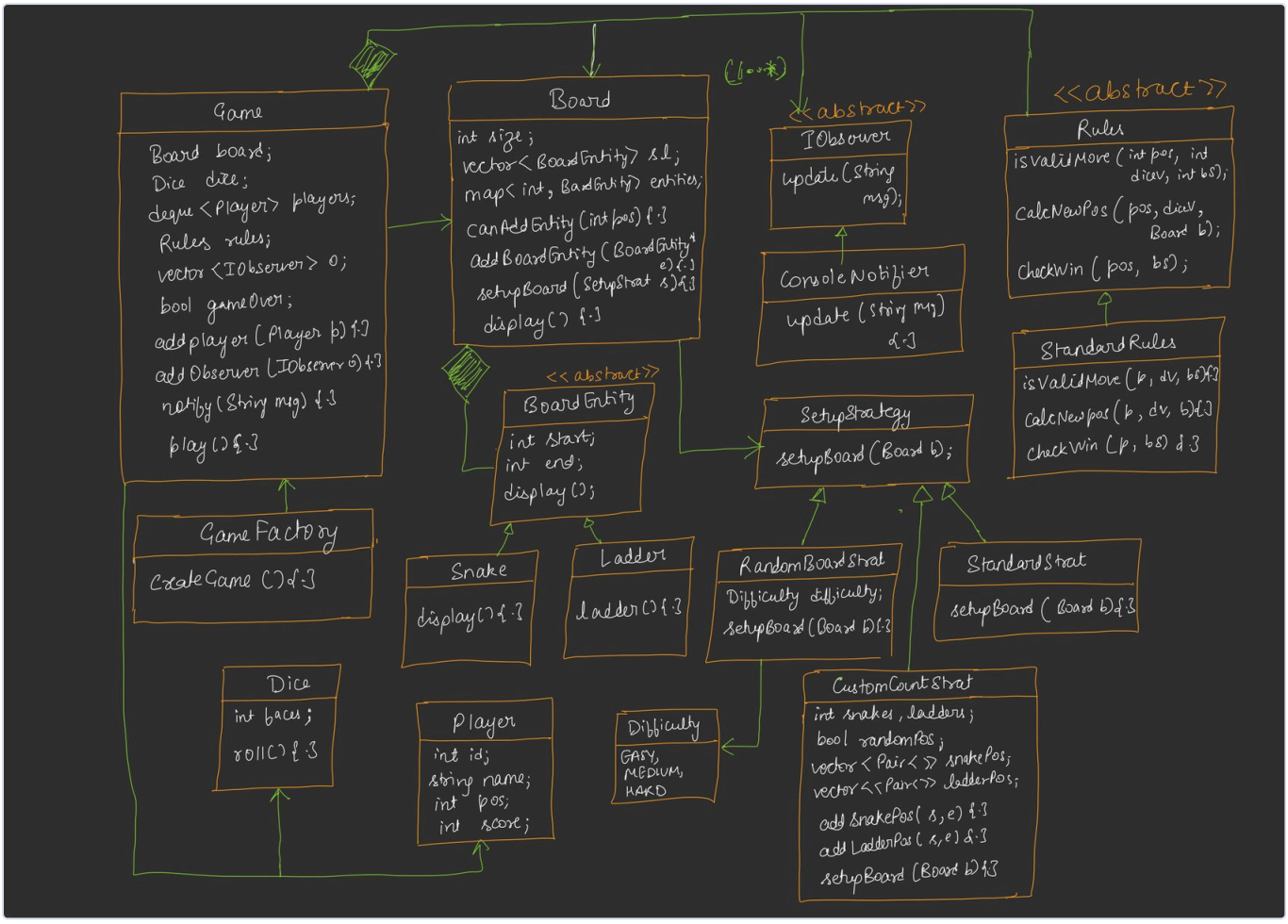
Smart board representation

Cell numbers are implicit (1…N), so **don't allocate an N×N matrix or even an N-array**. Store **sparse**: `map<int, BoardEntity*>` (cell → snake/ladder) for **O(1)** lookup on every move, plus a `vector<BoardEntity*>` of all entities.

Class	Design
`BoardEntity` (abstract)	`start`, `end`, `display()`. `Snake`: start = head, end = tail (`start > end`). `Ladder`: start = bottom, end = top (`start < end`). One map handles both polymorphically.
`Board`	size, entities, map; `addBoardEntity, canAddEntity, getEntity, setupBoard(strategy)`. **Owns** its entities (composition).
`Dice` / `Player`	`Dice(faces=6).roll()` ∈ [1, faces]; Player: id, name, position, score.
`Game`	Board, Dice, `deque<Player*>`, rules, observers, `isGameOver`. Owns Rules and Board.

Patterns

- **Strategy: board setup** `IBoardSetupStrategy::setupBoard(Board*)`. `Standard` (hard-coded 10×10 layout), `Random(Difficulty)` (EASY: fewer snakes/more ladders; MEDIUM: equal; HARD: more snakes), `Custom` (counts or exact (start,end) pairs).
- **Strategy: rules** `SnakeAndLadderRules`: `isValidMove` (`pos + dice ≤ size`), `calculateNewPosition` (`pos + dice`, then if an entity is there return `entity->end`), `checkWinCondition` (`pos == size`).
- **Observer** (`ConsoleNotifier`) and **Factory** (`SnakeAndLadderGameFactory` wires Board, Dice, Strategy, Rules for standard/random/custom games).



Snake & Ladder UML: Game, Board with BoardEntity (Snake/Ladder), SetupStrategy family, Rules, Observer, Dice, Player, GameFactory.

```
play(): require players >= 2
  while (!isGameOver):
     p = players.front();  d = dice.roll()
     if !rules.isValidMove(p.pos, d)  → skip turn, rotate
     pos = rules.calculateNewPosition(p.pos, d)     // snake/ladder teleport via map lookup
     if rules.checkWin(pos) → declare winner, end
     rotate deque
```

23 Chess (online, multi-user)

▶ Video 37 of the playlist

Requirements

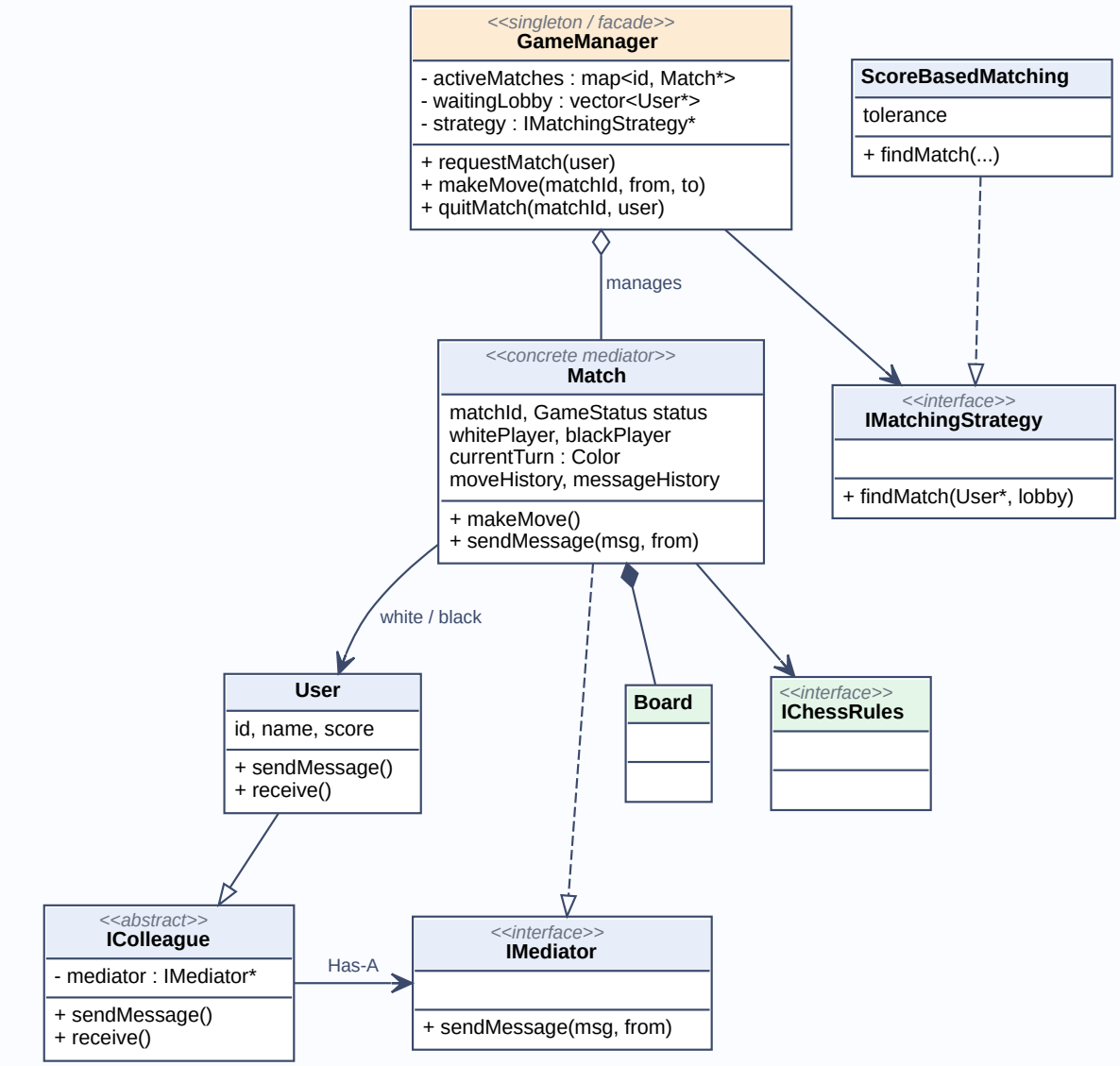
- An **online** platform (chess.com-style) with many simultaneous matches.
- **Score-based matchmaking** from a lobby within a tolerance.
- Standard rules, **extensible** for variants (OCP).
- In-match **chat**.
- **Quit handling:** quitter's score −20, winner +30.

Patterns

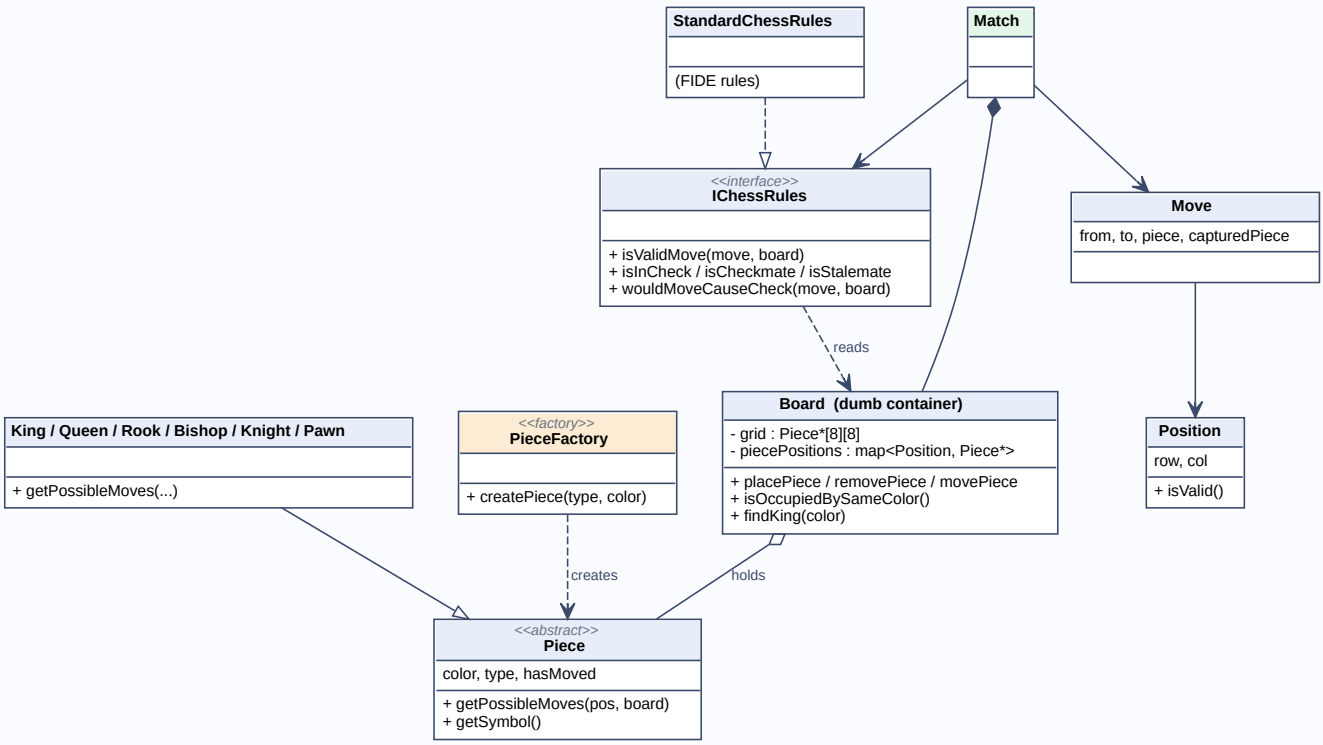
Pattern	Design
Mediator	`Match` is the concrete mediator for the chat (`sendMessage(msg, from)`); `User` is the colleague. For two players `Match` holds exact `whitePlayer` / `blackPlayer` and routes to the opponent.
Strategy	`IMatchingStrategy::findMatch(user, lobby)` (`ScoreBasedMatching`) and `IChessRules` (`isValidMove, isInCheck, isCheckmate, isStalemate, wouldMoveCauseCheck` ; `StandardChessRules` = FIDE).
Factory	`PieceFactory` creates King/Queen/Rook/Bishop/Knight/Pawn from `PieceType` + `Color` .
Singleton + Facade	`GameManager` : `activeMatches` , `waitingLobby` , strategy; `requestMatch, makeMove, quitMatch` .

Models

- `Match` : id, players, Board, rules, `GameStatus` (WAITING / IN_PROGRESS / COMPLETED / ABORTED), `currentTurn` , message and move histories.
- `Board` : **dumb container**: 8×8 grid + `map<Position, Piece*>` ; `placePiece, removePiece, movePiece, isOccupiedBySameColor, findKing` . **No rule knowledge**, so it's reusable for other board games.
- `Piece` (abstract): color, type, `hasMoved` (pawn's first double-step); `getPossibleMoves(pos, board)` , `getSymbol()` .
- `Position` (row, col, `isValid()` , operators for map keys) and `Move` (from, to, piece, capturedPiece): value classes.



Chess UML (redrawn), part 1: GameManager facade, score-based matching strategy and Match acting as the chat Mediator.



Chess UML (redrawn), part 2: the rule-free Board, Piece hierarchy with PieceFactory, and the IChessRules strategy.

`makeMove` pipeline

```
1. validate match status and whose turn it is
2. build tentative Move(from, to, piece, capturedPiece)
3. rules.isValidMove(move, board):
      target ∈ piece.getPossibleMoves() ?
      wouldMoveCauseCheck(): apply move temporarily → isInCheck(myColor)? → UNDO move
   invalid → reject, retry
4. board.movePiece(move)
5. evaluate opponent:  checkmate → end, winner +30 / loser −20
                       stalemate → end as draw
                       in check  → alert opponent
6. switch currentTurn; append to moveHistory
```

KEY INTERVIEW POINTS

Rules out of the Board (dumb container) → reuse across variants. **Pins and checks** are handled by *simulate, test, roll back* in `wouldMoveCauseCheck()` . **Mediator adaptation:** classic Mediator manages an arbitrary list of colleagues; a 2-player match narrows it to two exact references.

PART 4

Part 4 · Interview Playbook

A repeatable script for any LLD problem you have never seen. If you only have 15 minutes before an interview, read this part.

24 The 9-step approach to any LLD problem

Follow the steps in order and say each one out loud. The order matters: design **structure first, algorithms last**.



The 9-step flow. Steps 5 and 6 are where patterns enter; step 9 is where you earn senior-level credit.

Step 1 · Clarify and scope

- Split requirements into **Functional** (the user journey) and **Non-Functional** (scalability, concurrency, extensibility/OCP, reliability).
- **Narrow the scope aggressively:** you can't build a product in an hour. State what is **in** (the happy flow) and what is **out** (e.g. delivery-agent flow).
- Treat heavy subsystems (payments, notifications, maps) as **third-party interfaces**.
- Ask: *"Is this a reusable plug-and-play library or a full app?"* and *"Is RAM or concurrency a concern?"* (Flyweight, mutex).

Step 2 · Extract entities

- **Nouns → classes/models** (User, Cart, Order). **Verbs → methods.** Fixed sets → **enums** (`GameStatus`, `SplitType`, `DeviceType`).
- Model only what **this** application needs from each real-world object.
- Prefer small value classes (`Split`, `Position`, `Move`) over raw pairs or primitives.
- Keep models "dumb" (state + simple behaviour); put rules in strategies/services (`Board` doesn't know chess).

Step 3 · Choose the modelling direction

Bottom-up (default, ~80-90%)

Atomic models → relationships → managers → orchestrator. Use for service/commerce apps (Zomato, Spotify, Zepto, Splitwise).

Top-down

Orchestrator/container first, then components. Use for **games** and problems with **many small objects** (Tic-Tac-Toe, Snake & Ladder, Chess, Tinder).

Step 4 · Draw relationships

- **Is-A** (inheritance) only for a true substitutable subtype; otherwise **Has-A**.
- Choose aggregation vs composition by the **lifecycle question**, and **state your assumption**.
- Draw multiplicities (1..*) and visibility (`-` fields, `+` methods).

Step 5 · Spot the variability

Underline every phrase that means "this can change / there are several kinds". That is where a pattern goes (use the table in the next chapter). Ask yourself: *what will the interviewer ask me to add next?* Design that extension point.

Step 6 · Apply patterns and SOLID, then sanity-check

- Each pattern must **earn its place** by solving a named problem. Avoid gold plating.
- Check: **SRP** (one reason to change per class?) · **OCP** (new variant = new class, no edits?) · **LSP** (no child that throws "not supported"?) · **ISP** (no dead methods?) · **DIP** (high-level code holds interfaces, injected?).

Step 7 · Add the orchestrator / entry point

A **Facade / Manager / Controller / App** class gives the client one simple interface (`Tomato`, `MusicPlayerFacade`, `PaymentController`, `GameManager`). Make registries **Singletons** only when there is truly one global instance. Be ready to defend SRP/LoD violations (below).

Step 8 · Walk the happy flow

Run one scenario through your classes: a short `main()` or a sequence diagram (search → cart → order → pay; swipe → match → chat; move → validate → check-mate). It exposes missing methods and wrong ownership.

Step 9 · Defend trade-offs and offer extensions

- Name the principle you consciously bent and why (see the one-liners in Chapter 27).
- Offer 2 extensions from your pattern choices: new strategy, new factory case, retry with exponential backoff, a notification channel, thread-safety with a mutex.

- Mention what you deliberately left out of scope.

25 Recognising the pattern from the problem statement

Trigger phrase → pattern

If the problem says / implies…	Reach for	Typical shape
"Multiple ways to do X", payment modes, sort/discount/split/play modes, rule engines	**Strategy**	Context Has-A interface; concrete algorithms
"Create the right object by type", enum decides the class	**Factory** (Simple / Method / Abstract)	`XFactory::create(type)`
Product **families** that must match	**Abstract Factory**	One factory → several related products
Exactly one instance; manager, registry, logger, config	**Singleton**	Private ctor + static `getInstance()`
Object with many optional fields / immutable / validated on build	**Builder**	Fluent `.withX()` → `build()`
Creating objects is expensive; many similar copies	**Prototype**	`clone()` via copy ctor
"Notify / alert / subscribe / on change"	**Observer**	subscribe, notify, update
Undo/redo, queue or schedule actions, rebindable buttons	**Command**	execute/undo + history stack
Snapshot, rollback, checkpoint, transaction	**Memento**	Originator / Memento / Caretaker
Finite states, behaviour depends on state, lifecycle (order, vending, ATM)	**State**	Context delegates to current-state object
Same steps in fixed order, variants differ in details	**Template Method**	`final` template + abstract steps
Sequential checks that may stop early; approval levels; stacking rules; denominations	**Chain of Responsibility**	Handler with `next`
Combine features freely at run-time (add-ons, formatting, validators)	**Decorator**	Is-A + Has-A component
Third-party / legacy API with a different interface; "we may switch vendor"	**Adapter**	Target interface + wrapper
Complex subsystem, want one simple entry	**Facade**	Orchestrator class
Lazy load, permission check, remote call, retry/caching wrapper	**Proxy**	Same interface, holds real subject
Tree, part-whole, folders/files, menus, org chart	**Composite**	Leaf + Composite share a Component
Two dimensions that multiply (m × n classes)	**Bridge**	Abstraction Has-A implementor (m + n)
Millions of similar objects, RAM pressure	**Flyweight**	Shared intrinsic state + pooled factory
Many peers talking to each other; chat, lobby	**Mediator**	Hub that routes messages

Traverse a custom collection without exposing it	**Iterator**	`hasNext()` / `next()`
New operations over a stable set of element types	**Visitor**	`accept(v)` / `visit(this)` (double dispatch)
Null checks everywhere	**Null Object**	`NoOp` implementation

Code smell → fix

Smell in your design	Fix
`if/else` or `switch` on a type to choose **behaviour**	Strategy (or State if the type is the object's own lifecycle)
`if/else` or `switch` on a type to choose **which class to create**	Factory
Subclass count multiplies when adding a capability	Composition: Strategy / Decorator / Bridge
A subclass throws "not supported"	Re-split the hierarchy / interfaces (LSP, ISP); or Null Object
High-level class has `new MySqlDatabase()`	Depend on an interface and inject it (DIP)
One class changes for several unrelated reasons	Split by responsibility (SRP)
Client calls `a.getB().getC().x()`	Delegating method on `a` (Law of Demeter) or a Facade
Constructor with 8+ params / many overloads	Builder
Client loops over collection internals	Iterator
Objects hold references to many peers	Mediator

26 Decision cheat sheet

Relationship and ownership

Question	Rule of thumb
Does the part survive without the whole?	**Yes** → aggregation (restaurants in manager, song in playlist*). **No** → composition (cart in user, menu item in restaurant, inventory manager in dark store).
Is-A or Has-A?	Inheritance only if every parent guarantee holds for the child (LSP); otherwise Has-A + interface.
Singleton or not?	Singleton for global registries/services (Managers, Logger, Config, Factory). **Not** for things that exist per-entity (InventoryManager per DarkStore).
Where does the third-party call go?	Behind an interface via Adapter; wrap with Proxy for retry/auth.

*Spotify's notes treat Song–Playlist as a simple association; state your assumption either way.

Data-structure choices that appeared in the course

Need	Use	Where
O(1) lookup / update by key	`map` / hash map	PlaylistManager, DB inventory (SKU→qty, SKU→Product), Snake & Ladder cells, Splitwise 2-D balances, Flyweight pool
Round-robin turns	`deque` (front → back)	Tic-Tac-Toe, Snake & Ladder
History / undo / "previous"	`stack`	Command undo, Random & Custom play strategies
"Play next" insertion	`queue`	CustomQueueStrategy
Closest / best-first	min-heap / priority queue; sorted by distance	Nearest rider, nearby dark stores
Sequential handlers	linked `next` pointers	Chain of Responsibility, coupons
Geography	**Haversine** (not Euclidean)	Tinder
Concurrency	`mutex` / double-checked locking	CouponManager, Singleton
Auto-increment ids	`private static int nextId`	Model constructors
Float equality	tolerance (|x| > 0.01)	Splitwise leave check

27 Trade-offs, one-liners and the final checklist

Trade-off one-liners (memorise)

Situation	Say this
Orchestrator knows everything / deep call chains	"It's a facade for the client. In production I'd split into controllers and services. I accept the SRP/Demeter cost for a clear entry point."
LoD vs SRP (renderer fetching `getElements()`)	"LLD has no perfect solutions, only trade-offs. I'm choosing to violate the Law of Demeter to preserve SRP."
Observer manages observers and domain logic	"Strictly I'd lift subscriber management into a base/manager; I keep it here because the notification logic is static and it avoids class bloat."
Factory with `switch`	"I'm breaking OCP in one small place so the rest of the system never sees instantiation details."
Aggregation vs composition	"I assume X can't exist without Y, so composition. If menus outlive restaurants it'd be aggregation."
Strategy vs Bridge	"Strategy swaps behaviour at run-time; Bridge separates two hierarchies at design time."
Singleton drawbacks	"Global state hurts testability and adds thread-safety concerns, so I use it only for true single sources of truth."
Many small classes (Command, Strategy)	"More classes, but each is trivial and independently testable; the payoff is OCP."
Bending SOLID	"Principles are ideals not laws. Like time vs space in DSA, I choose consciously and minimise violations."

The final 10-point checklist

✓	Before you say "I'm done"
☐	Scope stated: what's in, what's out, which parts are third-party interfaces.
☐	Entities have one clear responsibility each; models are dumb, rules live in strategies/services.
☐	Relationships drawn (Is-A vs Has-A) and ownership assumptions spoken aloud.
☐	Every "many ways to do X" has an interface; no type-based `if/else` in business logic.
☐	High-level code depends on interfaces and receives them by injection.
☐	Object creation is isolated (Factory/Builder); singletons justified.
☐	One clean entry point (Facade/Manager/Controller).
☐	Happy flow walked end to end (main or sequence diagram).
☐	Concurrency and scale mentioned where relevant (mutex, O(1) lookups, Flyweight).
☐	Trade-offs named, 2 extensions offered, over-engineering avoided.

Suggested 45-minute pacing

Minutes	Focus
0 – 5	Clarify, scope, non-functional requirements (Step 1)
5 – 15	Entities, relationships, direction (Steps 2-4)
15 – 30	Patterns + class diagram on the board (Steps 5-7)
30 – 40	Key method pseudo-code + happy-flow walkthrough (Step 8)
40 – 45	Trade-offs and extensions (Step 9)

THE MINDSET IN ONE LINE

Model the domain, find what changes, hide it behind an interface, inject it, and be ready to explain the trade-off you made.